

Heverton Eduardo Peres

GRÁTIS: SUBSTITUA FERRAMENTAS PAGAS POR OPEN SOURCE

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P153g PERES, Heverton Eduardo

GRÁTIS: SUBSTITUA FERRAMENTAS PAGAS POR OPEN
SOURCE / Heverton Eduardo Peres. – São Paulo : Fábrica Agêntica de
Livros,
2026.

98 p. ; 21 cm.

ISBN 978-65-00000-63-4

1. GRÁTIS. 2. SUBSTITUA. 3. FERRAMENTAS. 4. PAGAS. I. Título.

CDD 005.74

Sumário

Capítulo 1: Anotações e Produtividade: Saindo do Aluguel do Notion e do Evernote	6
1. Introdução	6
2. Explica	6
3. Ilustra	7
4. Técnica	8
Preparando a Bancada: Requisitos Antes de Montar Qualquer Peça	9
Montando o AppFlowy Self-Hosted	9
As Outras Cinco Peças da Bancada	11
Tabela de Decisão: Qual Peça Escolher Primeiro	13
5. Aplica	14
6. Conclusão	15
7. Referências Bibliográficas	15
Capítulo 2: E-mail Marketing: Enviando Campanhas Sem Dependendo do Mailchimp	18
1. Introdução	18
2. Explica	18
3. Ilustra	19
4. Técnica	20
O selo de garantia: publicando SPF, DKIM e DMARC	20
Montando a bancada: Listmonk, Mautic e Postal — o que cada um resolve	22
Auditando a licença: o contraexemplo do EmailEngine e o alerta do Mailtrain	24
5. Aplica	25
Exercício	26
6. Conclusão	26
7. Referências Bibliográficas	27
Capítulo 3: CRM — Vendendo e Atendendo Sem Pagar Salesforce	29
1. Introdução	29
2. Explica	29
3. Ilustra	30
4. Técnica	32
O que a gaveta de automação já resolve sem você escrever uma linha de código	32
Colocando o EspoCRM de pé com Docker	33

SuiteCRM: a bancada mais testada, com o preço da idade	34
Twenty: a bancada mais nova, com a fatura de infraestrutura mais alta ...	34
Tabela de decisão: qual ferramenta cabe na sua bancada agora	35
Odoo e Krayin: as peças que ficam no catálogo, não na bancada deste capítulo	35
5. Aplica	36
6. Conclusão	38
7. Referências Bibliográficas	38
Capítulo 4: Automação de Fluxos: Trocando o Zapier pelo n8n	41
1. Introdução	41
2. Explica	41
3. Ilustra	42
4. Técnica	43
Por que Docker e por que PostgreSQL desde o primeiro dia	44
Subindo a bancada e testando o primeiro gatilho	45
O fluxo de ponta a ponta: gatilho, nó, credencial e deploy	45
Escolhendo a peça certa: tabela de decisão	47
Backup da bancada: o script que evita perder o fluxo montado	48
5. Aplica	48
Exercício	49
6. Conclusão	50
7. Referências Bibliográficas	50
Capítulo 5: Design Gráfico e Edição de Imagem: Sem Canva, Sem Photoshop, Sem Illustrator	52
1. Introdução	52
2. Explica	52
3. Ilustra	54
4. Técnica	55
Instalando as Peças Desktop: GIMP, Inkscape e Krita	55
Subindo o Penpot com Docker Compose	55
Do Zero a uma Peça de Marketing Pronta	57
Tabela de Decisão: Qual Peça Pegar na Prateleira	58
A Automação da Bancada: Scripts que Preparam e Exportam	58
A Curva de Aprendizado: O Que Realmente Custava Caro	59
5. Aplica	59
6. Conclusão	61
7. Referências Bibliográficas	61
Capítulo 6: Vídeo e Nuvem: Editando e Guardando Seus Arquivos Sem Assinatura	64
1. Introdução	64
2. Explica	64
3. Ilustra	66

4. Técnica	66
Montando a Bancada de Vídeo	67
A Estante Central: Nextcloud Self-Hosted	67
A Prateleira Externa: Backup com a Regra 3-2-1	69
A Estante Leve: Seafile para o Autônomo	69
Vídeo Não É Só Cortar: O Fluxo de Produção Completo	70
A Rotina de Manutenção da Estante: Sincronização e Saúde do Servidor ..	70
5. Aplica	71
6. Conclusão	72
7. Referências Bibliográficas	73
Capítulo 7: Análise de Dados e Gestão de Projetos: BI e Kanban Sem Licença por Usuário ..	75
1. Introdução	75
2. Explica	75
3. Ilustra	76
4. Técnica	77
Subindo o Metabase	77
A Bancada de Gestão: OpenProject	79
O Caminho do Superset: Quando o Analytics Precisa de Profundidade ...	79
Tabela de Decisão: Qual Bancada de Gestão Pendurar	80
O Quadro Pessoal: Focalboard para o Artesão Solo	81
O Ciclo de Vida de um Relatório: Do Banco ao E-mail Agendado	81
5. Aplica	82
6. Conclusão	83
7. Referências Bibliográficas	83
Capítulo 8: Comunicação em Equipe e a Infraestrutura Que Sustenta Tudo	86
1. Introdução	86
2. Explica	86
3. Ilustra	88
4. Técnica	88
A Tabela de Decisão da Sala de Equipe	89
A Bancada Que Hospeda Tudo: Mattermost + Docker	89
O Deploy na Camada PaaS: Dokku	90
O Armário com Pannel: Coolify para Quem Quer Ver Tudo	91
A Segurança da Oficina: Atualização e Acesso	92
O Ritual de Rollback: Quando a Atualização Quebra a Bancada	92
5. Aplica	93
6. Conclusão	94
7. Referências Bibliográficas	95

Capítulo 1: Anotações e Produtividade: Saindo do Aluguel do Notion e do Evernote

1. Introdução

Toda oficina começa com a primeira ferramenta comprada — e para o Artesão Digital que está deixando de alugar software, essa primeira peça quase sempre é o lugar onde ele anota, organiza e planeja o próprio trabalho. Notion e Evernote resolveram isso tão bem, por tanto tempo, que a maioria de nós nunca parou para calcular quanto já pagou por eles em assinaturas mensais. Este capítulo existe para fazer essa conta e, em seguida, mostrar exatamente como substituir essa peça alugada por uma peça própria, montada e mantida na sua própria bancada.

Você vai aprender três coisas, nesta ordem: o que realmente muda — na prática, não na teoria — quando você troca uma ferramenta de anotações paga por uma auto-hospedada; como instalar e configurar as seis alternativas mais maduras do mercado (AppFlowy, Logseq, Joplin, Outline, Trilium Notes e Standard Notes); e onde essa troca ainda não vale a pena, para que você decida com os olhos abertos, não com fé em promessas de marketing. Ao final, você terá pelo menos uma ferramenta rodando na sua própria infraestrutura e um critério claro para repetir essa decisão em qualquer categoria de software que vier depois.

2. Explica

Comece pela pergunta que a maioria pula: o que exatamente você compra quando assina o Notion ou o Evernote? Não é o software — é o *acesso* ao software, hospedado em servidores que você nunca vê, com dados que ficam sob custódia de outra empresa, entregue enquanto o pagamento mensal continuar entrando. É o mesmo modelo de uma loja de aluguel de ferramentas: você paga para usar a furadeira, mas a furadeira nunca é sua, e no dia em que parar de pagar, ela some da sua bancada — no caso do software, “some” significa que sua conta é suspensa e seu acesso aos dados originais pode ficar condicionado a exportações limitadas ou a prazos de carência.

“Open source” resolve uma parte específica desse problema, e é importante ser preciso sobre qual parte. Código aberto significa que o código-fonte do programa está publicamente disponível para leitura, modificação e redistribuição, sob uma licença que garante esses direitos — o AppFlowy, por exemplo, é distribuído como projeto aberto sob termos que permitem exatamente isso [4], e o Logseq segue licença AGPL-3.0, que impõe até a obrigação de manter aberto qualquer serviço derivado que ofereça o software pela rede [13]. Isso é diferente de “gratuito”: muita ferramenta gratuita não é aberta (você não pode ler nem alterar o código), e o inverso também existe — projeto aberto pode ter uma camada paga (suporte, hospedagem gerenciada) por cima do núcleo livre.

A segunda peça de vocabulário que você precisa é “self-hosted” versus “local-first”. Self-hosted significa que você mesmo hospeda o servidor da aplicação — em uma VPS, num Raspberry Pi, num NAS doméstico — em vez de depender do servidor da empresa que criou o software. Local-first é uma escolha de arquitetura diferente e complementar: o aplicativo funciona primeiro a partir dos dados salvos localmente no seu dispositivo, com sincronização como recurso opcional por cima, não como dependência obrigatória para o programa nem abrir. Logseq, Trilium Notes e Standard Notes seguem essa filosofia local-first; AppFlowy e Outline são desenhados para rodar como serviço cloud-capable, mesmo quando você mesmo hospeda esse serviço [1][3].

Essa distinção importa porque ela determina onde o “custo do aluguel” reaparece depois que você troca de ferramenta. Ao trocar um serviço SaaS por uma alternativa auto-hospedada, você não elimina custo — você o transforma. Deixa de pagar uma mensalidade previsível e passa a pagar, em vez disso, com tempo de administração: manter o servidor no ar, aplicar atualizações de segurança, fazer backup, configurar domínio e certificado, e resolver o que quebrar às 22h de uma sexta-feira. Esse padrão se repete em praticamente toda categoria de substituição SaaS por open source, e por isso ele abre este primeiro capítulo em vez de ficar escondido numa nota de rodapé.

3. Ilustra

Pense na diferença entre alugar uma furadeira numa loja de equipamentos e comprar a sua própria. Alugada, você paga por hora ou por dia, devolve limpa, e a loja é responsável por manutenção, calibração e substituição se ela quebrar — você nunca vê o motor por dentro. Comprada, a furadeira é sua para sempre: você decide quando usá-la, pode abri-la e trocar uma peça se quiser, e ninguém pode tirá-la de você por falta de pagamento. Mas agora *você* é quem precisa guardá-la em local seco, lubrificar as partes móveis e substituir a escova de carvão quando gastar. É exatamente essa troca que acontece quando você sai do Notion (a furadeira alugada, sempre pronta, sempre mantida por outra pessoa) para o AppFlowy auto-hospedado (a furadeira comprada, sua para sempre, mas sua responsabilidade).

Essa primeira analogia explica a mecânica geral — posse contra aluguel — mas ela ainda deixa escondido o ponto mais contraintuitivo da troca, que merece uma segunda lente. Pense agora não na furadeira em si, mas na *garantia* que vinha embutida no aluguel. Quando você aluga, a loja garante que a ferramenta funciona: se travar, eles trocam; se você não souber usar, eles orientam. Ao comprar sua própria peça para a bancada, essa garantia desaparece — e é substituída pela documentação do fabricante, pelo fórum da comunidade e pela sua própria capacidade de diagnosticar o problema. É exatamente o que acontece quando o Logseq trava ao abrir um grafo de notas grande demais: não existe suporte pago te chamando de volta, existe uma issue pública no rastreador do próprio projeto, ainda aberta, documentando exatamente esse limite de desempenho [12]. A “garantia” virou “transparência” — você vê o problema real, em vez de recebê-lo maquiado por trás de um SLA.

Como Artesão Digital, é essa segunda lente que você vai treinar ao longo de todo este livro: perguntar não apenas “essa peça própria funciona?”, mas “que garantia eu perco ao trocar, e estou disposto a assumir essa responsabilidade sozinho?”.

O diagrama abaixo resume o fluxo de decisão que guia o restante deste capítulo — da ferramenta alugada até a peça montada e funcionando na sua própria bancada.

```
%% legenda: Fluxo de decisão na Oficina Digital: da ferramenta alugada à
peça própria montada na bancada
flowchart TB
    A[Ferramenta alugada: Notion ou Evernote] --> B{Colaboracao tempo real ou
app mobile e critico?}
    B -->|Sim, indispensavel hoje| C[Mantenha o aluguel por enquanto]
    B -->|Nao, ou aceita o contorno| D[Escolha a peca para a bancada]
    D --> E[AppFlowy: tudo em um]
    D --> F[Logseq ou Trilium: notas locais]
    D --> G[Joplin: sync proprio]
    D --> H[Outline: wiki de equipe]
    D --> I[Standard Notes: seguranca auditada]
    E --> J[Monte via Docker Compose na sua VPS]
    F --> J
    G --> J
    H --> J
    I --> J
    J --> K[Peca propria: bancada, manutencao e garantia sao suas]
```

4. Técnica

Esta seção é onde a Oficina Digital sai do papel. Ela segue o padrão do material-fonte: configuração real em `docker-compose.yml`, sessões de terminal mostrando o comando e a saída

esperada, e uma tabela de decisão ao final — sem código de programação, porque o que você precisa aqui é operar a ferramenta, não desenvolvê-la.

Preparando a Bancada: Requisitos Antes de Montar Qualquer Peça

Antes de escolher qual alternativa instalar, verifique se a sua bancada tem a base mínima. A stack self-hosted completa do AppFlowy exige Docker Engine na versão 24.0 ou superior, Docker Compose na versão 2.20 ou superior, um domínio próprio configurado e no mínimo cinco serviços rodando em paralelo: banco de dados Postgres, cache Redis, armazenamento de objetos MinIO, autenticação GoTrue e a API principal [1][4]. Essa é a exigência mais pesada entre as seis ferramentas deste capítulo — o Outline pede uma pilha parecida (Node.js, Postgres 9.5 ou superior, Redis 4 ou superior e um bucket compatível com S3 como o MinIO para anexos) [15][14], enquanto Standard Notes, Joplin, Logseq e Trilium Notes rodam com bem menos.

Confirme a versão do Docker instalada antes de seguir:

```
$ docker --version
Docker version 24.0.7, build afdd53b
$ docker compose version
Docker Compose version v2.23.0
```

Se a versão do seu servidor for anterior a essas, atualize antes de continuar — os manifestos abaixo assumem a sintaxe do Compose V2 (`docker compose` , sem hífen).

Montando o AppFlowy Self-Hosted

AppFlowy é o caso de referência deste capítulo porque cobre a fatia mais ampla dos casos de uso do Notion: documentos, bases de dados relacionais e quadros kanban, tudo a custo de licença zero [1][3]. Comparativos técnicos independentes estimam que ele já reproduz cerca de 85% dos fluxos de trabalho centrais de um workspace típico do Notion [19] — o que sobra fora desse percentual é justamente o assunto da seção 5 deste capítulo.

O manifesto abaixo é a base mínima de produção recomendada pela documentação oficial, adaptada para uma única VPS:

```
# docker-compose.yml – AppFlowy self-hosted (referência mínima de produção)
version: "3.9"

services:
  appflowy_postgres:
    image: postgres:15-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: appflowy
```

```

    POSTGRES_PASSWORD: "troque-esta-senha"
    POSTGRES_DB: appflowy
volumes:
  - appflowy_pg_data:/var/lib/postgresql/data

appflowy_redis:
  image: redis:7-alpine
  restart: unless-stopped
volumes:
  - appflowy_redis_data:/data

appflowy_minio:
  image: minio/minio:latest
  restart: unless-stopped
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: appflowy
    MINIO_ROOT_PASSWORD: "troque-esta-senha-tambem"
volumes:
  - appflowy_minio_data:/data

appflowy_gottrue:
  image: appflowyinc/gottrue:latest
  restart: unless-stopped
  depends_on:
    - appflowy_postgres
  environment:
    GOTTRUE_SITE_URL: "https://notas.suaoficina.dev"
    GOTTRUE_DB_DRIVER: postgres

appflowy_cloud:
  image: appflowyinc/appflowy_cloud:latest
  restart: unless-stopped
  depends_on:
    - appflowy_postgres
    - appflowy_redis
    - appflowy_minio
    - appflowy_gottrue
  ports:
    - "8000:8000"
  environment:
    APPFLOWY_DATABASE_URL: "postgres://appflowy:troque-esta-senha@
appflowy_postgres/appflowy"
    APPFLOWY_REDIS_URI: "redis://appflowy_redis:6379"
    APPFLOWY_S3_ENDPOINT: "http://appflowy_minio:9000"

volumes:

```

```
appflowy_pg_data:
appflowy_redis_data:
appflowy_minio_data:
```

Suba a stack e confirme que todos os serviços entraram no ar antes de apontar o domínio para o servidor:

```
$ docker compose up -d
[+] Running 5/5
✓ Container appflowy_postgres Started
✓ Container appflowy_redis Started
✓ Container appflowy_minio Started
✓ Container appflowy_gottrue Started
✓ Container appflowy_cloud Started

$ docker compose ps
NAME                STATUS
appflowy_postgres   Up 12 seconds (healthy)
appflowy_redis      Up 12 seconds (healthy)
appflowy_minio      Up 11 seconds (healthy)
appflowy_gottrue    Up 9 seconds
appflowy_cloud      Up 8 seconds
```

Depois desse ponto, o passo seguinte é o mesmo descrito na documentação oficial de instalação: apontar um reverse proxy (Traefik, Caddy ou Nginx) com TLS para a porta 8000, criar a primeira conta de administrador pela interface e importar seu workspace exportado do Notion.

As Outras Cinco Peças da Bancada

Nem toda alternativa exige uma stack tão pesada quanto a do AppFlowy. Abaixo, o que muda em cada uma na hora de montar:

Logseq — não roda como serviço web por padrão: é um aplicativo local-first, distribuído como executável de desktop que lê e grava arquivos Markdown/Org-mode direto no seu disco [13][11]. Não há `docker-compose.yml` para “instalar o servidor” porque não existe servidor — a sincronização entre dispositivos depende de uma pasta compartilhada via Git ou de um serviço de arquivos externo [12]. É a peça mais simples de montar e a mais limitada em sincronização nativa.

Joplin — para ter paridade com a sincronização em nuvem do Evernote, você precisa rodar o Joplin Server, cuja rota de produção recomendada é Docker Compose com PostgreSQL como banco:

```
# docker-compose.yml – Joplin Server
version: "3.9"
```

```

services:
  joplin_db:
    image: postgres:15-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: joplin
      POSTGRES_PASSWORD: "troque-esta-senha"
      POSTGRES_DB: joplin
    volumes:
      - joplin_pg_data:/var/lib/postgresql/data

  joplin_server:
    image: joplin/server:latest
    restart: unless-stopped
    depends_on:
      - joplin_db
    ports:
      - "22300:22300"
    environment:
      APP_BASE_URL: "https://notas.suaoficina.dev"
      DB_CLIENT: pg
      POSTGRES_HOST: joplin_db
      POSTGRES_USER: joplin
      POSTGRES_PASSWORD: "troque-esta-senha"
      POSTGRES_DATABASE: joplin

volumes:
  joplin_pg_data:

```

Instalado o servidor, cada cliente Joplin (desktop, mobile) aponta para essa URL nas configurações de sincronização [8]. Um plugin de OCR está disponível desde a versão 2.14, mas seu desempenho em imagens complexas ainda é fraco, e a busca full-text em anexos não chega ao nível do Evernote [10][9].

Outline — exige obrigatoriamente autenticação via OAuth (Slack, Google ou OIDC): não existe login usuário/senha nativo [14]. A stack mínima documentada é Node.js, Postgres 9.5+, Redis 4+ e um bucket compatível com S3 para anexos — normalmente MinIO em instalações auto-hospedadas [14]. O limite padrão de upload de arquivo é de apenas 1 MB e precisa ser reconfigurado manualmente na variável de ambiente correspondente antes de qualquer uso real com anexos maiores [14][6]. Guias independentes de instalação passo a passo, como o de Karan Sharma, documentam esse ajuste como um dos primeiros passos pós-instalação [17].

Trilium Notes — como o Logseq, é local-first e roda como aplicativo desktop ou servidor pessoal leve; a diferença é que a colaboração multiusuário foi explicitamente recusada pelo mantenedor do projeto TriliumNext como complexidade fora do escopo — não é uma limitação técnica temporária, é uma decisão de design permanente [26][27]. Não há app mobile

oficial [26]. Se sua bancada é individual, essa peça é uma das mais estáveis da lista; se você precisa de colaboração em equipe, ela está fora de questão [16].

Standard Notes — o setup self-hosted de referência (v2) roda em uma VPS pequena, de apenas 2 GB de RAM e 1 vCPU, graças a uma reescrita anunciada pela própria equipe como 70% mais eficiente em memória que a versão anterior [20][21]. É a única ferramenta deste capítulo com auditoria de segurança independente publicada: um pentest e uma revisão de criptografia conduzidos pela Cure53 [23]. Em troca dessa robustez de segurança, o foco é texto simples criptografado ponta a ponta — não espere dele o mesmo suporte a anexos multimídia ricos e clipper web que o Evernote oferece [22].

Tabela de Decisão: Qual Peça Escolher Primeiro

Ferramenta	Licença	Modelo	Melhor uso	Evite se...
AppFlowy	Aberta [4]	Cloud-capable auto-hospedado	Substituir o Notion quase por completo	Equipe é majoritariamente mobile-first
Logseq	AGPL-3.0 [13]	Local-first, sem servidor	Notas pessoais em grafo/Markdown	Grafo vai crescer além de ~300 MB [12]
Joplin	Aberta [8]	Local-first + servidor de sync opcional	Substituir o Evernote com sync próprio	Precisa de OCR ou busca robusta em anexos [10]
Outline	Aberta [15]	Cloud-capable auto-hospedado	Wiki de equipe com OAuth corporativo	Não tem provedor OAuth disponível [14]
Trilium Notes	Aberta [27]	Local-first, mantenedor único	Base de conhecimento pessoal e estável	Precisa de colaboração multiusuário [26]
Standard Notes	Aberta [22]	Local-first criptografado	Texto simples com segurança auditada	Depende de anexos multimídia ricos [22]

Use essa tabela como ponto de partida, não como veredito final — ela resume o que a documentação oficial e os comparativos independentes desta seção já mostraram em detalhe. A ferramenta certa para a sua bancada depende do que você prioriza: abrangência de recursos (AppFlowy), simplicidade local (Logseq/Trilium), paridade de sincronização (Joplin), colaboração de equipe (Outline) ou segurança auditada (Standard Notes).

5. Aplica

Imagine a cena: você acabou de instalar o AppFlowy seguindo os passos da seção anterior, ficou satisfeito com o resultado no seu próprio teste solo, e decide migrar o workspace inteiro da sua equipe de oito pessoas no mesmo fim de semana — afinal, a ferramenta “cobrir 85% do Notion”, então por que esperar? Na segunda-feira, três colegas abrem o mesmo documento de planejamento ao mesmo tempo para revisar a pauta da reunião das 9h. As edições de um sobrescrevem silenciosamente as do outro. Ninguém percebe até a reunião já ter começado com informação desatualizada na tela.

O diagnóstico está na teoria da seção 2: você tratou uma ferramenta cloud-capable como se ela tivesse a mesma maturidade de colaboração em tempo real do Notion, mas “cobrir 85% dos casos de uso” não significa “cobrir 85% de cada caso de uso individualmente” — os 15% que faltam se concentram justamente em colaboração simultânea polida, exatamente o ponto que os comparativos técnicos já sinalizavam como fraqueza antes mesmo da instalação [19][5]. Você comprou a peça certa para a bancada errada: uma peça pensada para uso individual ou colaboração assíncrona, aplicada a um fluxo que exigia edição síncrona em tempo real.

A correção não é abandonar o AppFlowy — é sequenciar a migração. Rode um piloto com uma pessoa ou uma dupla por duas semanas, documentando explicitamente quais fluxos dependem de edição simultânea. Só depois disso migre o restante da equipe, e mantenha, para os poucos documentos que realmente exigem edição ao vivo por várias pessoas, uma ferramenta híbrida ou um fluxo assíncrono acordado (“um edita, avisa no chat, o próximo edita depois”). É um ajuste de processo barato que evita o prejuízo caro de uma decisão tomada em uma reunião sem a informação certa.

Esse tipo de armadilha se repete, com pequenas variações, nas outras cinco ferramentas deste capítulo. Vale sintetizar os limites honestos antes de fechar o capítulo:

- **Colaboração em tempo real polida** é onde toda a categoria perde para o Notion. AppFlowy até cobre parte disso, mas Logseq, Trilium Notes e Standard Notes são desenhados como local-first e não escalam para edição multiusuário simultânea — no caso do Trilium, essa limitação nem é um roadmap futuro, é uma decisão de design definitiva do mantenedor [26]. Não force esse uso: equipes que dependem de colaboração ao vivo devem manter a ferramenta paga para esse fluxo específico, mesmo adotando alternativas abertas para o resto [5].
- **Apps mobile maduros** são o segundo limite real. O comparativo entre AppFlowy e Notion aponta especificamente o aplicativo mobile como sensivelmente menos polido que o do concorrente pago [19], e o Trilium Notes simplesmente não tem app mobile oficial [26]. Se sua equipe trabalha primariamente em celular, teste esse fluxo específico antes de migrar — não assuma que a experiência desktop se repete lá.
- **OCR e busca em anexos de nível Evernote** ainda não têm equivalente maduro nesta lista. O Joplin oferece um plugin de OCR desde a versão 2.14, mas com desempenho fraco em imagens complexas, e sem a busca full-text robusta em anexos que o Evernote entrega

nativamente [10][9]. Quem depende de digitalizar recibos, cartões de visita ou documentos escaneados como parte do fluxo de trabalho principal ainda sente essa lacuna.

Curadorias independentes de projetos auto-hospedáveis, como a lista `awesome-selfhosted` [5] e a lista específica de alternativas a SaaS populares [18], confirmam esse mesmo padrão em outras categorias além de notas — e é por isso que este capítulo trata esses três limites como regra geral de como avaliar qualquer substituição futura, não como exceção isolada das ferramentas de anotação.

6. Conclusão

Você chegou ao final deste primeiro capítulo com três peças montadas na sua bancada mental: primeiro, que trocar SaaS por open source não elimina custo, apenas o transforma de assinatura mensal em tempo de manutenção — e essa conta precisa ser feita antes da migração, não depois. Segundo, que a instalação real dessas ferramentas é acessível com Docker Compose e algumas horas de atenção, com o AppFlowy cobrindo a fatia mais ampla dos casos de uso do Notion e as outras cinco peças (Logseq, Joplin, Outline, Trilium Notes e Standard Notes) preenchendo nichos específicos de sincronização, segurança e simplicidade. Terceiro, que colaboração em tempo real, apps mobile maduros e OCR de qualidade ainda são pontos honestos de fraqueza — e ignorar isso é o erro mais caro que você pode cometer nesta categoria.

Como desafio prático, antes de seguir para o próximo capítulo: suba o `docker-compose.yml` do AppFlowy (ou do Joplin Server, se preferir sincronização mais simples) em uma VPS real ou até numa máquina local, e migre para lá apenas as suas próprias notas pessoais — não as da sua equipe ainda. Use por uma semana inteira antes de considerar qualquer migração coletiva.

No próximo capítulo, a Oficina Digital ganha sua segunda peça: você vai aprender a montar sua própria stack de e-mail marketing e vai descobrir por que essa categoria escondeu, até agora, a conta mais traiçoeira de toda a substituição SaaS — a reputação de domínio e a deliverability, o motivo pelo qual “montar o servidor” é, ali, apenas o primeiro passo de um processo muito mais longo do que parece.

7. Referências Bibliográficas

[1] APPFLOWY. *Docker | AppFlowy Docs*. Disponível em: <https://docs.appflowy.io/docs/appflowy/install-appflowy/installation-methods/installing-with-docker>. Acesso em: 19 ago. 2026.

- [2] APPFLOWY. *Self-Hosting AppFlowy*. Disponível em: <https://docs.appflowy.io/docs/guides/appflowy>. Acesso em: 19 ago. 2026.
- [3] APPFLOWY-IO. *AppFlowy-Cloud*. Disponível em: <https://github.com/AppFlowy-IO/AppFlowy-Cloud>. Acesso em: 19 ago. 2026.
- [4] APPFLOWY-IO. *AppFlowy: Bring projects, wikis, and teams together with AI*. Disponível em: <https://github.com/AppFlowy-IO/AppFlowy>. Acesso em: 19 ago. 2026.
- [5] AWESOME-SELFHOSTED. *A list of Free Software network services and web applications which can be hosted on your own servers*. Disponível em: <https://github.com/awesome-selfhosted/awesome-selfhosted>. Acesso em: 19 ago. 2026.
- [6] CONTABO. *Docmost vs Outline vs Notion: Which Wiki Should You Self-Host?*. Disponível em: <https://contabo.com/blog/docmost-vs-outline-vs-notion/>. Acesso em: 19 ago. 2026.
- [7] DASROOT. *Obsidian vs Logseq vs Notion: PKM Systems Compared 2026*. Disponível em: <https://dasroot.net/posts/2026/03/obsidian-logseq-notion-pkm-systems-compared-2026/>. Acesso em: 19 ago. 2026.
- [8] JOPLIN. *laurent22/joplin*. Disponível em: <https://github.com/laurent22/joplin>. Acesso em: 19 ago. 2026.
- [9] JOPLIN. *Plugins — documentação oficial*. Disponível em: <https://joplinapp.org/plugins/>. Acesso em: 19 ago. 2026.
- [10] JOPLIN FORUM. *Still some way to go for OCR*. Disponível em: <https://discourse.joplinapp.org/t/still-some-way-to-go-for-ocr/42964>. Acesso em: 19 ago. 2026.
- [11] LOGSEQ. *Documentation*. Disponível em: <https://docs.logseq.com/>. Acesso em: 19 ago. 2026.
- [12] LOGSEQ. *Issue #8544: Logseq locks up trying to load large graphs*. Disponível em: <https://github.com/logseq/logseq/issues/8544>. Acesso em: 19 ago. 2026.
- [13] LOGSEQ. *logseq/logseq: A privacy-first, open-source platform for knowledge management and collaboration*. Disponível em: <https://github.com/logseq/logseq>. Acesso em: 19 ago. 2026.
- [14] OUTLINE. *outline/outline: The fastest knowledge base for growing teams*. Disponível em: <https://github.com/outline/outline>. Acesso em: 19 ago. 2026.
- [15] OUTLINE. *Outline – Team knowledge base & wiki*. Disponível em: <https://www.getoutline.com/>. Acesso em: 19 ago. 2026.
- [16] PISTACK. *Joplin vs Trilium Notes vs AFFiNE: Best Self-Hosted Note-Taking 2026*. Disponível em: <https://www.pistack.xyz/posts/2026-04-21-joplin-vs-trilium-vs-affine-self-hosted-note-taking-guide-2026/>. Acesso em: 19 ago. 2026.
- [17] SHARMA, Karan. *Self Hosting Outline Wiki*. Disponível em: <https://mrkaran.dev/posts/setting-outline/>. Acesso em: 19 ago. 2026.

[18] SOLVOHQ. *awesome-self-host-saas-alternatives: Curated list of strictly self-hostable open-source alternatives to popular SaaS*. Disponível em: <https://github.com/SolvoHQ/awesome-self-host-saas-alternatives>. Acesso em: 19 ago. 2026.

[19] STACKALTS. *AppFlowy vs Notion (2026): The Local-First Workspace That Owns Your Data*. Disponível em: <https://stackalts.com/appflowy-vs-notion.html>. Acesso em: 19 ago. 2026.

[20] STANDARD NOTES. *Introducing our new self-hosting setup with 70% more memory efficiency — Decrypted*. Disponível em: <https://standardnotes.com/blog/introducing-self-hosting-v2>. Acesso em: 19 ago. 2026.

[21] STANDARD NOTES. *Self-hosting with Docker*. Disponível em: <https://standardnotes.com/help/self-hosting/docker>. Acesso em: 19 ago. 2026.

[22] STANDARD NOTES. *standardnotes/docs: Documentation for Standard Notes users and developers*. Disponível em: <https://github.com/standardnotes/docs>. Acesso em: 19 ago. 2026.

[23] STANDARD NOTES. *Standard Notes Completes Penetration Test and Cryptography Audit (Cure53) — Decrypted*. Disponível em: <https://standardnotes.com/blog/standard-notes-security-audits-2021>. Acesso em: 19 ago. 2026.

[24] TECHJOCKEY. *Compare Evernote VS Joplin*. Disponível em: <https://www.techjockey.com/us/compare/evernote-vs-joplin>. Acesso em: 19 ago. 2026.

[25] TRILIUMNEXT. *Docs*. Disponível em: <https://docs.triliumnotes.org/>. Acesso em: 19 ago. 2026.

[26] TRILIUMNEXT. *FAQ – User Guide*. Disponível em: <https://docs.triliumnotes.org/user-guide/faq>. Acesso em: 19 ago. 2026.

[27] TRILIUMNEXT. *Trilium: Build your personal knowledge base with Trilium Notes*. Disponível em: <https://github.com/TriliumNext/Trilium>. Acesso em: 19 ago. 2026.

Capítulo 2: E-mail Marketing: Enviando Campanhas Sem Dependendo do Mailchimp

1. Introdução

No Capítulo 1, você trocou o aluguel do Notion e do Evernote pela posse de uma ferramenta própria — uma peça que você mesmo escolheu, instalou e passou a manter na sua oficina. A régua ali era relativamente simples: instalar, configurar, usar, e a peça funcionava exatamente como prometido, sem depender de nada fora da sua bancada. O e-mail marketing rasga essa régua. Você pode instalar a melhor ferramenta open source do mercado, configurá-la com perfeição e ainda assim ver suas campanhas caírem inteiras na caixa de spam — porque, diferente de uma nota ou um quadro kanban, um e-mail só existe de verdade quando atravessa a porta de outra empresa e passa pelo crivo de um algoritmo de reputação que você não controla.

Neste capítulo você vai entender por que “enviar e-mail” e “ferramenta de e-mail” são duas peças diferentes do catálogo — a confusão mais cara que existe nessa categoria —, vai montar sua própria bancada de campanhas com Listmonk, Mautic e Postal, e vai aprender a auditoria que separa uma peça de código aberto de verdade de uma imitação comercial disfarçada de grátis, usando o EmailEngine como o contraexemplo que ensina essa lição para o resto da vida.

2. Explica

Uma campanha de e-mail marketing tem duas responsabilidades que o Mailchimp empacota tão bem que a maioria dos usuários nunca percebe que são coisas distintas. A primeira é a **composição da campanha**: montar a lista de contatos, desenhar o template, agendar o disparo, medir abertura e clique. A segunda é a **entrega**: fazer o e-mail realmente atravessar a internet e chegar à caixa de entrada do destinatário sem ser classificado como spam. Quando você paga o Mailchimp, você paga pelas duas coisas ao mesmo tempo, numa única mensalidade. Quando

you monta sua própria stack, precisa resolver as duas separadamente — e é exatamente aí que a maioria das trocas para open source falha.

A parte técnica da entrega chama-se **relay SMTP**: um servidor que aceita o e-mail da sua ferramenta de disparo e o retransmite para o servidor de destino usando o protocolo SMTP (Simple Mail Transfer Protocol). Ferramentas de newsletter como o Listmonk **não incluem um motor SMTP próprio confiável** — elas dependem de um relay externo, seja um serviço comercial (Amazon SES, Postmark, Mailgun) seja uma infraestrutura própria como o Postal [6] [9]. Isso já é a primeira quebra de expectativa de quem vem do mundo SaaS: instalar o Listmonk não é o fim da jornada, é o meio.

A segunda peça teórica é a **reputação de domínio**, sustentada por três registros DNS que funcionam como um selo de garantia read-only pelos provedores de e-mail do mundo todo: SPF (Sender Policy Framework, que declara quais servidores têm permissão de enviar em nome do seu domínio), DKIM (DomainKeys Identified Mail, uma assinatura criptográfica que garante que o conteúdo não foi alterado no caminho) e DMARC (Domain-based Message Authentication, Reporting and Conformance, que diz ao provedor de destino o que fazer quando SPF ou DKIM falham). Sem esses três registros configurados corretamente, e-mails caem no spam independentemente da qualidade do conteúdo ou da ferramenta usada para o disparo [6] [5]. Um relatório de benchmark de 2025 dedicado inteiramente a esse tema mede, entre outras coisas, a taxa de colocação de e-mails legítimos na caixa de entrada — e é essa métrica, não o número de recursos de uma ferramenta, que decide se sua campanha “funcionou” [1].

A terceira peça, mais sutil, é a diferença entre **código aberto de verdade** e **software gratuito por tempo limitado**. Uma ferramenta pode estar hospedada publicamente no GitHub, ter código totalmente visível e ainda assim não ser open source — o critério não é “posso ler o código”, é “a licença me permite rodar, modificar e redistribuir em produção sem pagar”. Essa distinção parece jurídica, mas tem consequência prática direta: decide se você está comprando uma peça ou alugando uma peça disfarçada de compra. Você vai ver esse contraste de forma concreta ainda neste capítulo.

3. Ilustra

Pense na sua oficina como uma metalúrgica pequena que aceita encomendas por correspondência. A **bancada de campanha** (Listmonk, Mautic) é onde você desenha a peça: escreve a carta, escolhe o papel, decide para quem vai. Mas a bancada não tem uma frota de entregadores — para isso existe uma **esteira de entrega** por trás da parede da oficina (o relay SMTP, como o Postal), que empilha as cartas nos caminhões e as leva até a porta do destinatário. E na porta de cada destinatário existe um porteiro rigoroso, o filtro anti-spam do provedor de e-mail, que só deixa entrar caminhões com um **selo de garantia** válido no para-choque — o conjunto SPF/

DKIM/DMARC. Um caminhão sem selo é mandado de volta para o depósito de spam, mesmo que a carta dentro dele seja impecável.

Como Artesão Digital, você já entende que toda peça da sua oficina exige manutenção — mas a esteira de entrega tem uma particularidade que nenhuma outra ferramenta deste livro tem até agora: sua reputação não é sua, é **emprestada e reconstruída a cada envio**. Um caminhão com histórico limpo de entregas ganha passagem livre pelos postos de fiscalização; um caminhão novo, ou um que já levou spam antes, é parado e revistado — mesmo que hoje esteja carregando cartas legítimas. É por isso que a primeira campanha enviada por um domínio novo quase sempre tem taxa de entrega pior do que a décima: a reputação se constrói com o tempo, não se compra com a licença da ferramenta.

Esse é o ponto mais difícil de aceitar para quem vem do SaaS: no Mailchimp, a reputação do domínio de envio pertence, em parte, à infraestrutura compartilhada da própria empresa — você “aluga” um pedaço da reputação coletiva deles junto com a ferramenta. Ao montar sua própria esteira de entrega, você assume a responsabilidade inteira por aquele selo de garantia, sozinho. É a mesma lógica de posse versus aluguel do Capítulo 1, só que agora a peça que você “compra” carrega uma obrigação contínua de manutenção que uma nota do Logseq nunca teve.

```
%% legenda: Trajeto de uma campanha desde a bancada de disparo até a caixa
de entrada, passando pelo relay SMTP e pelo selo de garantia SPF/DKIM/DMARC
flowchart LR
    A[Bancada de campanha] --> B[Relay SMTP]
    F[Reputacao do dominio] --> C
    B --> C{Selo SPF DKIM DMARC ok?}
    C -->|sim| D[Caixa de entrada]
    C -->|nao| E[Pasta de spam]
```

4. Técnica

O selo de garantia: publicando SPF, DKIM e DMARC

Antes de instalar qualquer ferramenta, resolva a esteira de entrega — é a parte que, se pulada, faz toda a bancada parecer quebrada mesmo estando perfeita. Os três registros vivem no DNS do seu domínio como entradas do tipo TXT. O exemplo abaixo mostra a estrutura mínima que você publicaria no painel do seu provedor de DNS para um domínio de envio dedicado, prática recomendada para não misturar a reputação do e-mail transacional com a do e-mail de marketing [6][9]:

```
# registros-dns-campanhas.yaml
dominio: campanhas.suaoficina.com.br
registros:
  - tipo: TXT
    nome: campanhas.suaoficina.com.br
    valor: "v=spf1 include:_spf.postal.suaoficina.com.br ~all"
  - tipo: TXT
    nome: postal-dkim._domainkey.campanhas.suaoficina.com.br
    valor: "v=DKIM1; k=rsa; p=MIGfMA0GCsqGSIb3DQEBAQUAA4GNADCBiQKBgQC"
  - tipo: TXT
    nome: _dmarc.campanhas.suaoficina.com.br
    valor: "v=DMARC1; p=quarantine; rua=mailto:dmarc@suaoficina.com.br"
```

Depois de publicar, verifique com uma consulta DNS direta — nunca confie só no painel do provedor, confirme que o registro realmente propagou:

```
$ dig TXT campanhas.suaoficina.com.br +short
"v=spf1 include:_spf.postal.suaoficina.com.br ~all"

$ dig TXT _dmarc.campanhas.suaoficina.com.br +short
"v=DMARC1; p=quarantine; rua=mailto:dmarc@suaoficina.com.br"
```

Para quem prefere automatizar essa checagem em vez de repetir `dig` manualmente antes de cada campanha, um script pequeno já resolve a parte mais comum de erro humano — esquecer a política de falha no final do registro SPF:

```
def validar_spf(registro):
    """Confere se um registro SPF tem a forma minima esperada.

    Um registro SPF valido começa com 'v=spf1' e termina com uma
    politica de falha: '~all' (soft fail, recomendado ao começar)
    ou '-all' (hard fail, para dominios com reputacao ja madura).
    Sem essa politica, o registro fica incompleto e provedores mais
    rigorosos podem simplesmente ignorar a autorizacao do relay.
    """
    registro = registro.strip()
    if not registro.startswith("v=spf1"):
        return False, "registro nao começa com 'v=spf1'"
    if not (registro.endswith("~all") or registro.endswith("-all")):
        return False, "registro sem politica de falha (~all ou -all)"
    return True, "registro SPF valido"

if __name__ == "__main__":
    exemplos = [
        "v=spf1 include:_spf.postal.suaoficina.com.br ~all",
```

```

    "v=spf1 include:_spf.postal.suaoficina.com.br",
]
for spf in exemplos:
    ok, motivo = validar_spf(spf)
    print(f"{spf!r} -> {ok} ({motivo})")

```

Rodar esse script antes de cada campanha grande é uma daquelas peças de manutenção barata que evita um prejuízo caro: uma campanha inteira devolvida pelo provedor de destino por um `~all` esquecido.

Montando a bancada: Listmonk, Mautic e Postal — o que cada um resolve

Com o selo de garantia publicado, é hora de escolher as peças da bancada propriamente dita. O mercado de alternativas ao Mailchimp/ActiveCampaign se divide em duas famílias bem distintas, e confundi-las é o erro mais comum de quem começa essa troca.

A primeira família são ferramentas de **newsletter e mailing enxutas**. O **Listmonk** é a referência aqui: um binário único escrito em Go, licenciado sob AGPLv3, sem dependências pesadas além de um banco Postgres [9][6]. A instalação não roda migração de banco automaticamente — é preciso um passo explícito de setup antes do primeiro uso [9][6]. Ele não trata bounces de forma inteligente por padrão e é mais fraco em automações visuais estilo funil do que uma ferramenta de automação completa [5][10], mas é imbatível em simplicidade operacional para quem só precisa disparar newsletters em volume.

A segunda família é a de **automação de marketing full-stack**, cujo representante mais maduro é o **Mautic**. Diferente do Listmonk, o Mautic tenta competir diretamente com o ActiveCampaign: segmentação avançada, jornadas automatizadas, pontuação de leads. O preço dessa ambição é a infraestrutura: os requisitos mínimos reais de produção são 4 GB de RAM, 2 vCPU e 40 GB de disco [13]. Há também um teto conhecido de escala que pega muita gente desprevenida — a partir de segmentos com mais de 50.000 contatos, o processo de construção do segmento estoura o limite padrão de memória do PHP [13][14]. As imagens Docker oficiais da versão 5 adotaram uma abordagem de “microserviços” que a própria comunidade considera desnecessariamente complexa para instalações pequenas; a recomendação prática é rodar só dois containers — Mautic e MariaDB — em vez do conjunto completo sugerido pela documentação oficial [14]. Um detalhe operacional que quebra silenciosamente: os cron jobs do Mautic são críticos, e se pararem de rodar, campanhas e segmentos “morrem” sem aviso nenhum na interface [14].

A terceira peça — a que a maioria dos tutoriais de internet esquece de mencionar — é a **infraestrutura de envio** propriamente dita. O **Postal** não é uma ferramenta de campanha, é o equivalente self-hosted a um Mailgun ou SendGrid: construído em Rails com MySQL e RabbitMQ, exigindo uma máquina virtual dedicada só para ele [16][17]. A combinação recomendada pela própria comunidade técnica é usar Postal como camada de envio e Listmonk

como camada de campanha — cada ferramenta na função para a qual foi desenhada [5]. A deliverability real do Postal depende de fatores que nenhuma ferramenta resolve por você: IP dedicado com reputação própria, porta 25 liberada no provedor de VPS, registros MX/SPF/DKIM/rDNS corretos, e um período de “aquecimento” gradual do IP antes de disparar grandes volumes [17].

A tabela abaixo resume a decisão que você precisa tomar antes de instalar qualquer coisa:

Sua necessidade	Peça recomendada	O que ela NÃO resolve
Só disparar newsletter/mailling em volume	Listmonk	Relay SMTP próprio, automação avançada
Automação de marketing com jornadas e pontuação	Mautic	Infraestrutura de envio própria; exige servidor robusto
Controle total da entrega, IP e reputação	Postal	Interface de campanha; não substitui Listmonk/Mautic
Máximo de controle ponta a ponta	Postal + Listmonk juntos	Tempo de configuração inicial e de aquecimento de IP

Na prática, subir o Listmonk com Postgres via Docker Compose é o primeiro passo mais direto da bancada:

```
# docker-compose.yml
services:
  listmonk-db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: listmonk
      POSTGRES_PASSWORD: "<senha-forte-aqui>"
      POSTGRES_DB: listmonk
    volumes:
      - listmonk-data:/var/lib/postgresql/data
  listmonk:
    image: listmonk/listmonk:latest
    restart: unless-stopped
    ports:
      - "9000:9000"
    environment:
      LISTMONK_db__user: listmonk
      LISTMONK_db__password: "<senha-forte-aqui>"
      LISTMONK_db__database: listmonk
      LISTMONK_db__host: listmonk-db
    depends_on:
      - listmonk-db
```

```
volumes:
  listmonk-data:
```

O passo que a maioria esquece — e que gera o primeiro chamado de suporte de quem está começando — é que o Listmonk não cria o esquema do banco sozinho no primeiro `up`. É preciso rodar o instalador explicitamente antes de subir o serviço em modo normal [9][6]:

```
$ docker compose run --rm listmonk ./listmonk --install
* Verificando conexao com o banco de dados... ok
* Criando esquema do banco de dados
* Configurando definicoes iniciais
* Criando usuario administrador
Setup concluido. Acesse http://localhost:9000/admin

$ docker compose up -d
[+] Running 2/2
- Container listmonk-db Started
- Container listmonk Started
```

Auditando a licença: o contraexemplo do EmailEngine e o alerta do Mailtrain

A última peça deste capítulo não é uma ferramenta para instalar, é um hábito para adquirir: **antes de apostar sua oficina em qualquer projeto do GitHub, audite a licença e a cadência de manutenção, não só a estrela e a lista de recursos**. Dois casos reais desta categoria ilustram por que esse hábito compensa.

O primeiro é o **EmailEngine**, um cliente de e-mail “headless” frequentemente citado em listas de “alternativas grátis” de automação de e-mail. O código está publicamente visível no GitHub, o que faz muita gente assumir, de cara, que é open source [8]. Não é. A documentação oficial de licenciamento é explícita: o uso em produção exige uma licença comercial paga depois de um trial de 14 dias [18]. Código visível não é sinônimo de licença livre — é a diferença entre “posso olhar a peça na vitrine” e “posso levar a peça pra casa sem pagar”. Trate qualquer lista de “alternativas open source” que inclua o EmailEngine sem essa ressalva como uma lista que não foi auditada de verdade.

O segundo caso é diferente: o **Mailtrain** é, de fato, open source — grátis, sem limite de assinantes, código sob licença livre [12]. O alerta aqui não é de licença, é de **manutenção**: apesar de reunir cerca de 5.700 estrelas no GitHub, o último release público do projeto é de junho de 2021 [12][11]. Uma peça sem manutenção nova por anos acumula risco silencioso — vulnerabilidades de segurança conhecidas não são corrigidas, e a compatibilidade com versões novas de Node.js e MySQL/MariaDB vai se degradando [11]. Isso não significa “nunca use”, significa “se usar, saiba que está sozinho na manutenção — o mesmo tipo de responsabilidade que discutimos na esteira de entrega, mas agora aplicada ao próprio código”.

Um checklist rápido de auditoria, na forma de um registro que você pode manter por ferramenta candidata à sua oficina:

```
{  
  "ferramenta": "EmailEngine",  
  "licenca": "source-available (nao FOSS)",  
  "trial_dias": 14,  
  "ultimo_release": "ativo, mas sob licenca comercial apos o trial",  
  "veredito": "nao usar como base gratuita de producao"  
}
```

Antes de instalar qualquer peça nova nesta oficina — não só de e-mail, qualquer categoria deste livro —, confira estes quatro pontos, nesta ordem: (1) qual é a licença exata, lida na documentação oficial e não em blog de terceiros; (2) quando foi o último release com tag; (3) quantas issues abertas existem em relação a fechadas; (4) se existe comunidade ativa respondendo perguntas recentes. É o mesmo teste que separa uma peça compensadora de um aluguel disfarçado.

5. Aplica

Você acabou de subir o Listmonk, importou sua lista de 8.000 contatos que estava presa no Mailchimp há dois anos e disparou a primeira campanha de boas-vindas direto pela porta 25 padrão do seu próprio VPS, sem configurar nenhum relay dedicado. Meia hora depois, o painel de estatísticas mostra uma taxa de abertura de 3% — um número que seria motivo de alarme mesmo para os padrões modestos do e-mail marketing. O instinto nesse momento é culpar o conteúdo do e-mail, reescrever o assunto, trocar a imagem do cabeçalho. Esse é o erro: você está tentando resolver um problema de conteúdo quando o problema real é de reputação. Um domínio novo, sem SPF, sem DKIM, sem DMARC, disparando pela porta 25 de um provedor de VPS genérico — que a esta altura você já sabe que costuma ser bloqueada ou fortemente limitada pelos grandes provedores justamente para conter spam — está sendo silenciosamente filtrado antes mesmo de o destinatário abrir a caixa de entrada. O diagnóstico correto, voltando à seção Explica deste capítulo, é que a bancada de campanha (o Listmonk) fez exatamente o que devia fazer; quem faltou foi a esteira de entrega com o selo de garantia validado. A correção não é reescrever o e-mail — é publicar SPF/DKIM/DMARC, configurar um relay dedicado (Postal ou um serviço comercial de reputação já estabelecida) e reenviar a mesma campanha, sem alterar uma linha do conteúdo, para um grupo de teste menor primeiro.

Como Artesão Digital, essa cena deveria mudar permanentemente a ordem em que você monta qualquer stack de e-mail: primeiro a esteira de entrega, só depois a bancada de campanha — nunca o contrário.

Armadilhas comuns que a cena acima resume, na prática do dia a dia:

- Instalar a ferramenta de disparo e pular a configuração de SPF/DKIM/DMARC, tratando-a como “opcional” ou “para depois”.
- Disparar grandes volumes no primeiro dia com um domínio novo, sem período de aquecimento de IP — o equivalente a mandar um caminhão sem histórico de entregas direto para a rota mais concorrida.
- Confiar em uma lista de “melhores alternativas ao Mailchimp” sem verificar se cada item da lista é, de fato, open source (o caso EmailEngine) ou está tecnicamente abandonado (o caso Mailtrain).
- Tratar Mautic como “só mais uma instalação Docker”, ignorando que os cron jobs precisam de monitoramento contínuo — quando param, campanhas morrem em silêncio, sem erro visível no painel.

Em termos de escala, vale registrar onde cada peça desta bancada aguenta e onde quebra: o Listmonk, sendo um binário leve em Go, sustenta volumes de disparo grandes sem esforço de infraestrutura adicional — o gargalo nunca é o Listmonk, é sempre a reputação do caminho de entrega [9]. O Mautic, por outro lado, tem teto de escala concreto e documentado: acima de 50.000 contatos por segmento, o processo de segmentação estoura a memória padrão do PHP e passa a exigir ajuste manual de recursos do servidor [13][14]. Nenhuma das duas peças escala “infinitamente sem contrapartida” — e um relatório de benchmark dedicado ao tema mostra que a taxa de colocação na caixa de entrada, não o número de contatos, é a métrica que de fato separa uma operação de e-mail marketing saudável de uma fracassada [1].

Exercício

- ☐ Publique os três registros DNS (SPF, DKIM, DMARC) do seu domínio de envio e confirme a propagação com `dig` ou `nslookup`
- ☐ Suba o Listmonk com Docker Compose, rode o instalador e envie uma campanha de teste para uma lista pequena (menos de 20 contatos)
- ☐ Rode o script `validar_spf` deste capítulo contra o registro SPF real do seu domínio
- ☐ Monte o checklist de auditoria de licença (formato JSON deste capítulo) para duas ferramentas “open source” que você usa ou pretende usar

6. Conclusão

Este capítulo separou o e-mail marketing em três peças que o Mailchimp costuma empacotar como uma só: a esteira de entrega com seu selo de garantia SPF/DKIM/DMARC, a bancada de campanha propriamente dita — Listmonk para newsletter enxuta, Mautic para automação completa, Postal como infraestrutura de envio dedicada —, e o hábito de auditoria de licença que separa código aberto de verdade (Mailtrain, com ressalva de manutenção) de software comercial

disfarçado (EmailEngine). Domine essas três peças e você deixa de ser alguém que “instala uma ferramenta de e-mail” e passa a ser alguém que entende por que uma campanha chega — ou não chega — à caixa de entrada.

No Capítulo 3, essa mesma disciplina de separar “o que a ferramenta promete” de “o que ela realmente entrega sem apoio externo” volta a valer, com uma pergunta parecida: até onde um CRM open source cobre pipeline e atendimento antes de você sentir a falta de integrações e suporte formal de um Salesforce. Antes de virar a página, garanta que sua esteira de entrega está publicada e validada — é a fundação sobre a qual qualquer campanha futura, deste capítulo em diante, vai depender.

7. Referências Bibliográficas

[1] VALIDITY. *2025 Email Deliverability Benchmark Report*. Disponível em: <https://www.validity.com/resource-center/2025-email-deliverability-benchmark-report/>. Acesso em: 19 ago. 2026.

[2] AWESOME-SELFHOSTED. *A list of Free Software network services and web applications which can be hosted on your own servers*. Disponível em: <https://github.com/awesome-selfhosted/awesome-selfhosted>. Acesso em: 19 ago. 2026.

[3] 6SENSE. *ActiveCampaign vs Mautic: Marketing Automation Platforms Comparison*. Disponível em: <https://6sense.com/tech/marketing-automation/activecampaign-vs-mautic>. Acesso em: 19 ago. 2026.

[4] SOLVOHQ. *awesome-self-host-saas-alternatives: Curated list of strictly self-hostable open-source alternatives to popular SaaS*. Disponível em: <https://github.com/SolvoHQ/awesome-self-host-saas-alternatives>. Acesso em: 19 ago. 2026.

[5] OCTABYTE. *Best Open Source Alternatives to Mailchimp: Listmonk vs Postal vs Mautic*. Disponível em: <https://blog.octabyte.io/posts/open-source-alternative-to-mailchimp/>. Acesso em: 19 ago. 2026.

[6] LISTMONK. *Documentation*. Disponível em: <https://listmonk.app/docs/>. Acesso em: 19 ago. 2026.

[7] POSTAL. *Documentation*. Disponível em: <https://docs.postalserver.io/>. Acesso em: 19 ago. 2026.

[8] POSTALSYS. *emailengine: Headless email client*. Disponível em: <https://github.com/postalsys/emailengine>. Acesso em: 19 ago. 2026.

[9] KNADH. *listmonk: High performance, self-hosted, newsletter and mailing list manager*. Disponível em: <https://github.com/knadh/listmonk>. Acesso em: 19 ago. 2026.

[10] LEINSS, Tobias. *Listmonk vs Mailchimp: the newsletter I self-host instead*. Disponível em: <https://leinss.xyz/blog/en/listmonk-vs-mailchimp/>. Acesso em: 19 ago. 2026.

[11] SAASHUB. *Mailtrain VS MailChimp - compare differences & reviews*. Disponível em: <https://www.saashub.com/compare-mailtrain-vs-mailchimp>. Acesso em: 19 ago. 2026.

[12] MAILTRAIN-ORG. *mailtrain: Self hosted newsletter app*. Disponível em: <https://github.com/Mailtrain-org/mailtrain>. Acesso em: 19 ago. 2026.

[13] MAUTIC. *Mautic Requirements*. Disponível em: <https://mautic.org/mautic-requirements/>. Acesso em: 19 ago. 2026.

[14] MAUTEAM.ORG. *Mautic Self-Hosted Best Practices*. Disponível em: <https://mauteam.org/mautic/mautic-admins/mautic-hosting-mautic-self-hosted-best-practices/>. Acesso em: 19 ago. 2026.

[15] MAUTIC. *mautic/mautic: Open Source Marketing Automation Software*. Disponível em: <https://github.com/mautic/mautic>. Acesso em: 19 ago. 2026.

[16] POSTALSERVER. *postal: A fully featured open source mail delivery platform for incoming & outgoing e-mail*. Disponível em: <https://github.com/postalserver/postal>. Acesso em: 19 ago. 2026.

[17] POSTAL. *Pre-requisites*. Disponível em: <https://docs.postalserver.io/getting-started/prerequisites/>. Acesso em: 19 ago. 2026.

[18] EMAILENGINE. *Pricing & Licensing*. Disponível em: <https://learn.emailengine.app/docs/licensing>. Acesso em: 19 ago. 2026.

Capítulo 3: CRM — Vendendo e Atendendo Sem Pagar Salesforce

1. Introdução

No Capítulo 2, você descobriu a conta oculta do e-mail marketing “grátis”: reputação de domínio e deliverability, dois custos que o Mailchimp escondia dentro da mensalidade e que passaram a ser seu problema no momento em que você montou a própria stack de envio. O CRM guarda a mesma lição, só que com outro nome. Aqui, a conta oculta não é reputação de domínio — é tuning de servidor e maturidade de integração. E, como toda peça boa de oficina, ela vem com uma etiqueta de manutenção que o vendedor da loja de aluguel nunca mostra na vitrine.

Este capítulo resolve uma pergunta prática: até onde um CRM open source substitui de verdade o Salesforce, o HubSpot ou o Pipedrive — e onde a comparação simplesmente para. Você vai instalar e comparar três ferramentas em produção (EspoCRM, Twenty e SuiteCRM), entender o que cada uma cobre bem hoje e sair com um critério de decisão para saber quando vale trocar o aluguel pela posse, e quando a mensalidade ainda compra algo que sua bancada não tem.

2. Explica

Um CRM, reduzido ao essencial, é composto por quatro peças: um funil de vendas (pipeline) com estágios nomeados, uma ficha de contato com histórico de interação, uma camada de automação que reage a mudanças de estágio, e uma superfície de relatório que resume tudo isso para quem toma decisão. Toda a mitologia em torno de “inteligência artificial nativa” e “score preditivo” que os grandes fornecedores vendem como diferencial vem depois dessas quatro peças, não no lugar delas.

O mercado de CRM open source maduro se divide em duas gerações claramente distintas. De um lado estão sistemas PHP completos e antigos, como o SuiteCRM e o EspoCRM, com anos de desenvolvimento acumulado e cobertura funcional ampla. Do outro está uma

nova geração — o Twenty é o expoente mais visível — escrita em TypeScript, com interface moderna, mas cobertura de funcionalidades ainda incompleta [4]. Essa divisão não é só estética: ela determina o que cada ferramenta já entrega de fábrica e o que ainda exige que você monte na mão.

SuiteCRM nasceu como um fork robusto do extinto SugarCRM e hoje soma 5,7 mil estrelas no GitHub, com a versão 7.15.2 em produção estável [1]. É um projeto maduro no sentido literal da palavra: testado em escala, com muita gente já batendo a cabeça nos mesmos problemas que você vai encontrar. O EspoCRM, por sua vez, tem cadência de release mais rápida e manutenção mais ativa no dia a dia [2] — o tipo de sinal que importa quando você está decidindo em qual “fornecedor” (mesmo sendo open source, você ainda depende de alguém mantendo o projeto vivo) apostar sua operação de vendas.

O ponto cego dos dois é o mesmo: automação de marketing e IA nativa continuam rasas comparadas ao HubSpot. O EspoCRM já inclui campanhas de e-mail na base gratuita — algo que o Salesforce cobra separadamente através do Pardot [5] — mas o motor de regras é mais simples que o de uma plataforma de automação dedicada. Reporting nativo também é um ponto fraco recorrente: uma avaliação independente de 2026 descreve a interface do SuiteCRM como datada e sua automação de workflow como limitada [6].

Isso não é motivo para desistir — é motivo para saber exatamente qual das quatro peças do CRM você está comprando, e qual ainda precisa ser fabricada por você. É essa distinção, peça por peça, que separa quem escolhe ferramenta open source por ideologia de quem escolhe por engenharia.

3. Ilustra

Pense na sua operação de vendas como a bancada de uma oficina. Cada lead que chega é um pedido de peça: ele entra por uma porta, passa por gavetas etiquetadas (“Novo”, “Em negociação”, “Fechado”) e, se tudo correr bem, sai como uma peça montada e entregue — um cliente atendido, guardado na prateleira de relacionamento contínuo. Um CRM é literalmente o organizador dessas gavetas. Quando uma gaveta fica parada tempo demais — um negócio sem contato há uma semana —, a automação básica dispara um alerta para o vendedor responsável, do mesmo jeito que uma oficina bem organizada tem um aviso sonoro para peça esquecida na bancada.

%% legenda: Fluxo de um lead pelas gavetas de status do funil de vendas, com o ponto de decisão de alerta automático

flowchart LR

A[Pedido de peça chega] --> B{Gaveta: Novo}

B --> C{Gaveta: Em negociacao}

C --> D{Gaveta: Fechado}

```
D --> E[Prateleira de clientes atendidos]
C -->|parado 7 dias sem toque| F[Alerta automatico ao vendedor]
```

Como Artesão Digital, você já sabe que toda ferramenta que entra na sua bancada carrega dois custos: o de comprar (ou instalar) e o de manter afiada. Escolher entre EspoCRM, Twenty e SuiteCRM é escolher qual kit de ferramentas cabe no espaço físico — leia-se, no servidor — que você tem disponível. Aqui o conceito é denso o bastante para merecer duas camadas de analogia, porque a decisão não é só “qual ferramenta é melhor”, e sim “qual ferramenta o meu galpão suporta hoje”.

A primeira camada é a mecânica geral: cada ferramenta chega numa caixa com seu próprio kit de montagem. EspoCRM e SuiteCRM pedem o kit “PHP + banco relacional” — o equivalente a uma bancada com tomada de 110V, que qualquer galpão já tem. O Twenty pede o kit “contêineres Docker + banco de dados + cache” — uma bancada de 220V trifásico, que exige uma instalação elétrica mais robusta antes mesmo de a primeira peça ser montada.

```
%% legenda: Comparacao dos tres kits de montagem de servidor exigidos por
EspoCRM, SuiteCRM e Twenty antes de chegarem ao mesmo balcao de atendimento
flowchart TB
    A[Servidor disponivel] --> B{Qual kit de ferramentas a bancada exige}
    B -->|PHP mais banco relacional leve| C[EspoCRM]
    B -->|PHP mais banco relacional legado| D[SuiteCRM]
    B -->|Docker mais Postgres mais Redis| E[Twenty]
    C --> F[Balcao de atendimento em producao]
    D --> F
    E --> F
```

A segunda camada — o ponto mais contraintuitivo — é que “mais moderno” não é sinônimo de “mais completo”. O Twenty tem um kit de montagem exigente e uma comunidade grande (55,1 mil estrelas e cerca de 14,5 mil commits no repositório) [3], mas um relato técnico público resume a limitação com honestidade: falta “99% do que empresas precisam” em recursos como e-mail nativo robusto, dashboards avançados, autenticação SAML corporativa e campanhas de marketing [11]. Ou seja: o kit mais novo e mais bonito nem sempre é o mais equipado. Isso é o oposto do que a intuição de “tecnologia nova = tecnologia melhor” sugere — e é exatamente por isso que essa segunda analogia existe: para você não confundir bancada elegante com bancada completa.

4. Técnica

O que a gaveta de automação já resolve sem você escrever uma linha de código

Antes de instalar qualquer coisa, vale mapear o que a automação básica de um CRM open source já cobre — porque é aqui que a maioria das operações pequenas encontra tudo que precisa, sem gastar um centavo de licença.

Estágio da gaveta	Sinal observado	Ação automática esperada	Ferramenta que já cobre isso hoje
Novo	Lead sem contato em 48h	Notificação ao vendedor responsável	EspoCRM, SuiteCRM
Em negociação	Negócio parado 7 dias	Alerta + reabertura de tarefa	EspoCRM, SuiteCRM, Twenty (parcial)
Fechado — ganho	Mudança de estágio	Disparo de e-mail de boas-vindas	EspoCRM (campanha nativa) [5]
Fechado — perdido	Mudança de estágio	Tag de motivo de perda + relatório	EspoCRM, SuiteCRM

Essa tabela de decisão já resolve o pilar mais comum de reclamação sobre CRM pago: “eu pago R\$ 200 por usuário/mês e só uso 20% dos recursos”. A automação de estágio é justamente esse 20% essencial — e ele já está na prateleira das ferramentas livres.

Boa parte dos CRMs open source modernos expõe essa mudança de estágio como um evento de webhook, o que permite ligar o CRM a qualquer outra peça da sua oficina (um bot de WhatsApp, uma planilha, um sistema de emissão de nota fiscal) sem depender de um marketplace de integrações prontas. Um payload típico de evento de mudança de estágio se parece com isto:

```
{
  "event": "Opportunity.stageChanged",
  "entityId": "64f21a3b9c7d4",
  "previousStage": "Negotiation",
  "currentStage": "ClosedWon",
  "daysInPreviousStage": 9,
  "ownerId": "usr-0231"
}
```

Esse é o tipo de peça que você mesmo fabrica quando decide não alugar: em vez de esperar o fornecedor abrir uma integração pronta, você escreve um pequeno receptor de webhook e conecta ao resto da sua oficina.

Colocando o EspoCRM de pé com Docker

O EspoCRM é, para a maioria das operações pequenas e médias, o ponto de partida mais equilibrado: requisitos modernos (PHP 8.3–8.5, MySQL 8.0+, MariaDB 10.3+ ou PostgreSQL 15) e uma imagem Docker oficial que simplifica bastante o deploy [9][2]. O arquivo de composição abaixo é o suficiente para colocar uma instância em produção leve:

```
version: "3.8"
services:
  espocrm:
    image: espocrm/espocrm:latest
    ports:
      - "8080:80"
    environment:
      ESPOCRM_DATABASE_HOST: espocrm-db
      ESPOCRM_DATABASE_NAME: espocrm
      ESPOCRM_DATABASE_USER: espocrm
      ESPOCRM_DATABASE_PASSWORD: "<sua-senha-forte>"
      ESPOCRM_ADMIN_USERNAME: admin
      ESPOCRM_ADMIN_PASSWORD: "<senha-do-admin>"
      ESPOCRM_SITE_URL: "http://localhost:8080"
    volumes:
      - espocrm-data:/var/www/html
    depends_on:
      - espocrm-db
  espocrm-db:
    image: mariadb:10.11
    environment:
      MARIADB_ROOT_PASSWORD: "<senha-root>"
      MARIADB_DATABASE: espocrm
      MARIADB_USER: espocrm
      MARIADB_PASSWORD: "<sua-senha-forte>"
    volumes:
      - espocrm-db-data:/var/lib/mysql
volumes:
  espocrm-data:
  espocrm-db-data:
```

Subindo o serviço, a sessão de terminal deve se parecer com isto:

```
$ docker compose up -d
[+] Running 3/3
- Network gratis-crm_default    Created
- Container espocrm-db         Started
- Container espocrm            Started
$ docker compose ps
NAME                IMAGE                                STATUS
espocrm             espocrm/espocrm:latest             Up 46 seconds (healthy)
espocrm-db          mariadb:10.11                      Up 49 seconds (healthy)
```

Depois disso, o assistente de instalação web do EspoCRM guia a criação do usuário administrador e a configuração inicial do funil — sem exigir edição manual de arquivo de configuração para o caso básico.

SuiteCRM: a bancada mais testada, com o preço da idade

O SuiteCRM pede uma pilha um pouco mais tradicional — PHP 8.1 a 8.4 e MariaDB nas versões 10.6, 10.11, 11.4 ou 11.8 [7][8]. Não há imagem Docker oficial mantida com o mesmo nível de suporte do EspoCRM, então a instalação em produção normalmente passa por um servidor LAMP configurado manualmente ou por uma imagem de terceiros. Isso já é, por si só, um dado de decisão: se você não tem tempo para configurar Apache e PHP-FPM manualmente, o SuiteCRM cobra esse pedágio antes mesmo da primeira venda registrada.

Em compensação, é a ferramenta com maior volume de testagem em campo — 5,7 mil estrelas e um histórico de produção de anos [1]. O preço dessa maturidade é um débito técnico visível: o projeto acumula 1.366 issues abertas no repositório [1], e uma avaliação independente de 2026 relata degradação de performance em relatórios pesados sobre bases volumosas, além de uma interface que carrega marcas visuais datadas [6].

Twenty: a bancada mais nova, com a fatura de infraestrutura mais alta

O Twenty é a aposta mais recente da lista, com Docker Compose oficial que exige ao menos 2GB de RAM disponível, além de PostgreSQL e Redis como dependências obrigatórias — não opcionais [10]. Antes de decidir por ele, vale checar se o servidor de destino realmente tem esse orçamento de memória; é comum operações pequenas tentarem economizar num VPS de 1GB e descobrirem o problema só depois do deploy.

```
#!/usr/bin/env bash
RAM_MB=$(free -m 2>/dev/null | awk '/^Mem:/ {print $2}')
RAM_MB=${RAM_MB:-0}
if [ "$RAM_MB" -ge 2048 ]; then
    echo "RAM suficiente para o Twenty (Docker Compose oficial)"
else
```

```
echo "RAM insuficiente para o Twenty - prefira EspoCRM ou SuiteCRM neste
servidor"
fi
```

O projeto tem tração real de comunidade (55,1 mil estrelas, cerca de 14,5 mil commits) [3] e um benchmark independente de CRMs open source lhe dá nota 9 em 10 pela qualidade do código [4]. Mas o mesmo benchmark aponta duas limitações concretas: a licença é AGPL-3.0, o que contamina produtos derivados fechados caso você pretenda revender uma versão modificada, e não há aplicativo mobile nativo [4]. A lista de issues abertas no repositório confirma que recursos considerados básicos em CRMs corporativos ainda estão em construção [12].

Tabela de decisão: qual ferramenta cabe na sua bancada agora

Critério	EspoCRM	SuiteCRM	Twenty
Stack exigida	PHP 8.3–8.5 + MySQL/MariaDB/PostgreSQL [9]	PHP 8.1–8.4 + MariaDB [7][8]	Docker + PostgreSQL + Redis, ≥2GB RAM [10]
Deploy mais simples	Sim — imagem Docker oficial [2]	Não — sem Docker oficial mantido	Sim — Docker Compose oficial [10]
Ponto forte	Campanha de e-mail nativa incluída [5]	Maturidade em campo, muito testado [1]	UX moderna, código limpo [4]
Ponto de atenção	Reporting nativo raso [5]	Débito técnico: 1.366 issues [1]	Falta cerca de 99% de recursos corporativos avançados, por relato técnico [11]

Odoo e Krayin: as peças que ficam no catálogo, não na bancada deste capítulo

Duas outras ferramentas aparecem com frequência em qualquer levantamento de CRM open source, mas não entram na instalação prática deste capítulo por razões específicas — vale registrar por quê, para você não gastar tempo instalando a ferramenta errada para o seu caso.

O Odoo é tecnicamente um ERP completo com um módulo de CRM embutido, exigindo Python 3.10+ e PostgreSQL 13+ [18][19]. Sua edição Community é licenciada em LGPLv3, mas os recursos mais avançados — Studio, Helpdesk avançado, automação de marketing e pontuação de leads por IA — ficam trancados na Enterprise Edition, que é paga [20]. Um benchmark independente avalia o Odoo com nota 4 em 10 como CRM autônomo, descrevendo-

-o como “excessivamente complexo para necessidades básicas de CRM” [4], e o próprio G2 mostra a distância de maturidade de mercado: 4,0 de 5 estrelas em 166 avaliações contra 4,4 de 5 em 25.397 avaliações do Salesforce Sales Cloud [21]. A avaliação independente do PeerSpot reforça o mesmo padrão de percepção [22]. Se sua necessidade é CRM puro, o Odoo é a ferramenta errada para o problema errado — ele resolve outra dor, de gestão empresarial ampla, não a dor específica de vendas e atendimento [23].

O Krayin CRM, construído em Laravel e licenciado em MIT — a licença mais permissiva da lista —, exige PHP 8.3+, Composer 2.5+, MySQL 8.0.32+ ou MariaDB 10.3+, e pelo menos 3GB de RAM [24][25]. O suporte a Docker é secundário na documentação oficial, o que torna o deploy menos direto que o do EspoCRM. Um benchmark independente dá nota 7 em 10, descrevendo a experiência de uso como “rudimentar” e apontando problemas de performance, ainda que reconheça o valor da licença permissiva [4]. Vale registrar também que uma versão recente do projeto corrigiu uma vulnerabilidade de bypass de autenticação no instalador [26] — um lembrete de que “open source” não significa “auditado por você”; significa “auditável por qualquer um”, o que só funciona se alguém de fato auditar.

5. Aplica

Imagine a seguinte cena. É sexta-feira, fim de tarde. Você decidiu trocar o Pipedrive pelo Twenty depois de ler sobre a nota alta de qualidade de código no benchmark independente [4], e resolve subir o Docker Compose oficial num VPS de 1GB de RAM que sobrou de outro projeto — “é só para testar, no fim de semana eu migro para um servidor maior”. O comando `docker compose up -d` roda, os contêineres sobem, mas em poucos minutos o container do banco de dados começa a reiniciar em loop. A interface do Twenty fica inacessível bem no momento em que você ia mostrar a nova ferramenta para o time de vendas na segunda-feira.

O diagnóstico é simples e está documentado: o Twenty exige explicitamente PostgreSQL e Redis rodando ao lado da aplicação, com um mínimo de 2GB de RAM disponível para o conjunto [10]. Um VPS de 1GB não sustenta os três serviços simultaneamente sob carga real — o kernel do Linux mata o processo que consome mais memória (o famoso OOM killer) para manter o sistema vivo, e esse processo quase sempre é o banco de dados. Não é um bug do Twenty; é o kit de ferramentas certo instalado na bancada errada.

A correção é igualmente simples, mas exige disciplina de checklist, não de instinto: antes de qualquer deploy de CRM em produção, confira os requisitos mínimos de RAM e dependências obrigatórias da ferramenta escolhida — e faça isso num ambiente de homologação, nunca direto na sexta-feira à tarde antes de uma demonstração para o time. Se o servidor disponível não comporta o Twenty, a escolha correta não é “forçar de qualquer jeito” — é migrar a escolha de ferramenta para o EspoCRM ou o SuiteCRM, que rodam confortavelmente numa pilha PHP tradicional com fração da memória exigida.

Como síntese rápida, além da cena acima, vale registrar outras armadilhas comuns nessa migração:

- Migrar contatos e negócios de uma planilha ou de outro CRM sem mapear os campos personalizados antes — o resultado é um funil com metade dos negócios “sem estágio definido”.
- Subestimar o tempo de configuração de automação de e-mail transacional: o EspoCRM já inclui campanhas de e-mail na base gratuita [5], mas a configuração de SMTP e autenticação de domínio (o mesmo tema do Capítulo 2) continua sendo seu trabalho.
- Escolher a ferramenta pela nota de benchmark de código sem checar a nota de maturidade de mercado: o Twenty tem ótimo código [4], mas ainda carece de recursos que uma equipe de vendas grande considera básicos [11].

Sobre o limite real de escala: a partir de um funil com muitos milhares de negócios ativos e relatórios cruzados pesados, tanto o SuiteCRM quanto o EspoCRM apresentam degradação de performance segundo avaliação independente de mercado [6][14] — esse é o contorno honesto da comparação. Para uma operação de vendas de porte pequeno a médio, com pipeline, contatos e automação básica, a paridade com Salesforce e HubSpot já é real hoje. Para analytics corporativo pesado, forecast de território multi-nível ou aprovações complexas em várias camadas, a resposta muda: aí entra a lacuna honesta.

É exatamente essa lacuna que separa o amador do profissional nessa decisão. O amador troca de ferramenta porque “open source é sempre melhor”. O profissional — o Artesão Digital que você está se tornando ao longo deste livro — troca de ferramenta depois de comparar dado com dado: a nota de mercado do Odoo (4,0 de 5 em 166 avaliações) frente à do Salesforce (4,4 de 5 em 25.397 avaliações) [21] não é motivo para desistir do open source; é motivo para saber, categoria por categoria, onde o catálogo pronto de integrações (o AppExchange do Salesforce, o marketplace do HubSpot) ainda pesa mais do que o custo da mensalidade que você está tentando eliminar.

%% legenda: Decisao entre integracao pronta de mercado (Salesforce/HubSpot) e fabricacao propria via API em CRM open source

flowchart LR

A[Necessidade de integracao aparece] --> B{Existe peca pronta na prateleira do fornecedor}

B -->|Sim, AppExchange ou marketplace| C[Salesforce ou HubSpot resolvem na hora]

B -->|Nao, e preciso fabricar| D[CRM open source mais API propria]

C --> E[Aluguel continua, integracao imediata]

D --> F[Posse total, manutencao sob sua conta]

6. Conclusão

Este capítulo respondeu a três perguntas na ordem certa: o que um CRM open source já cobre bem (pipeline, contatos e automação básica de estágio), como colocar essa cobertura de pé em produção (EspoCRM como ponto de partida equilibrado, SuiteCRM como a bancada mais testada e mais pesada de manter, Twenty como a aposta moderna que cobra RAM e maturidade de recursos em troca de código limpo) e onde a comparação com Salesforce e HubSpot ainda para — analytics corporativo pesado, catálogo de integrações prontas e suporte formal com SLA.

Como desafio para fixar o aprendizado, escolha uma das três ferramentas instaladas neste capítulo, suba-a num ambiente de teste seguindo o `docker-compose.yml` apresentado, e migre manualmente cinco negócios fictícios pelas quatro gavetas do funil — Novo, Em negociação, Fechado-ganho, Fechado-perdido — cronometrando quanto tempo a configuração de automação de alerta leva para funcionar. Esse é o mesmo exercício de calibração que você vai repetir, com outra ferramenta, no próximo capítulo.

No Capítulo 4, a bancada ganha uma peça nova: a automação de fluxos entre ferramentas, trocando o Zapier pelo n8n. Você vai perceber que boa parte da automação “básica” de estágio que configurou aqui no CRM é só a ponta de um fio — e que esse fio pode ser esticado para conectar toda a oficina digital que você está montando, capítulo por capítulo.

7. Referências Bibliográficas

[1] SUITECRM/SALESAGILITY. *SuiteCRM/SuiteCRM: SuiteCRM - Open source CRM for the world*. Disponível em: <https://github.com/SuiteCRM/SuiteCRM>. Acesso em: 19 ago. 2026.

[2] ESPOCRM. *espocrm/espocrm: EspoCRM – Open Source CRM Application*. Disponível em: <https://github.com/espocrm/espocrm>. Acesso em: 19 ago. 2026.

[3] TWENTYHQ. *twenty: The open alternative to Salesforce, designed for AI*. Disponível em: <https://github.com/twentyhq/twenty>. Acesso em: 19 ago. 2026.

[4] MARMELAB. *Best Open Source CRM for 2026*. Disponível em: <https://marmelab.com/blog/2026/01/09/open-source-crm-benchmark-2026.html>. Acesso em: 19 ago. 2026.

[5] ESPOCRM. *Server Configuration — EspoCRM Documentation*. Disponível em: <https://docs.espocrm.com/administration/server-configuration/>. Acesso em: 19 ago. 2026.

[6] CRM.ORG. *SuiteCRM Review 2026: Features, Limits, and Real-World Use*. Disponível em: <https://crm.org/news/suitecrm-review>. Acesso em: 19 ago. 2026.

- [7] SUITECRM. *Downloading & Installing — SuiteCRM Documentation*. Disponível em: <https://docs.suitecrm.com/admin/installation-guide/downloading-installing/>. Acesso em: 19 ago. 2026.
- [8] SUITECRM. *Compatibility Matrix — SuiteCRM Documentation*. Disponível em: <https://docs.suitecrm.com/admin/compatibility-matrix/>. Acesso em: 19 ago. 2026.
- [9] ESPOCRM. *Installation — EspoCRM Documentation*. Disponível em: <https://docs.espcrm.com/administration/installation/>. Acesso em: 19 ago. 2026.
- [10] TWENTY. *Docker Compose — Twenty Documentation*. Disponível em: <https://docs.twenty.com/developers/self-host/capabilities/docker-compose>. Acesso em: 19 ago. 2026.
- [11] HACKER NEWS (Y COMBINATOR). *Twenty: A Modern open-source CRM*. Disponível em: <https://news.ycombinator.com/item?id=37805520>. Acesso em: 19 ago. 2026.
- [12] TWENTYHQ. *Issues · twentyhq/twenty*. Disponível em: <https://github.com/twentyhq/twenty/issues>. Acesso em: 19 ago. 2026.
- [13] TWENTY. *Twenty Documentation*. Disponível em: <https://docs.twenty.com/>. Acesso em: 19 ago. 2026.
- [14] G2. *EspoCRM vs. HubSpot Sales Hub Comparison 2026*. Disponível em: <https://www.g2.com/compare/espocrm-vs-hubspot-sales-hub>. Acesso em: 19 ago. 2026.
- [15] CAPTERRA. *Salesforce Sales Cloud Software Pricing, Alternatives & More — vs. EspoCRM*. Disponível em: <https://www.capterra.com/compare/61368-136101/Salesforce-vs-EspoCRM>. Acesso em: 19 ago. 2026.
- [16] CAPTERRA. *Compare Salesforce Sales Cloud vs SuiteCRM*. Disponível em: <https://www.capterra.com/compare/61368-136373/Salesforce-vs-SuiteCRM>. Acesso em: 19 ago. 2026.
- [17] G2. *SuiteCRM Reviews 2026: Details, Pricing, & Features*. Disponível em: <https://www.g2.com/products/suitecrm/reviews>. Acesso em: 19 ago. 2026.
- [18] ODOO S.A. *odoo: Odoo is a suite of web based open source business apps*. Disponível em: <https://github.com/odoo/odoo>. Acesso em: 19 ago. 2026.
- [19] ODOO S.A. *CRM — Odoo 19.0 documentation*. Disponível em: <https://www.odoo.com/documentation/19.0/applications/sales/crm.html>. Acesso em: 19 ago. 2026.
- [20] ODOO S.A. *Licenses — Odoo 19.0 documentation*. Disponível em: <https://www.odoo.com/documentation/19.0/legal/licenses.html>. Acesso em: 19 ago. 2026.
- [21] G2.COM. *Odoo CRM vs. Salesforce Sales Cloud Comparison*. Disponível em: <https://www.g2.com/compare/odoo-crm-vs-salesforce-salesforce-sales-cloud>. Acesso em: 19 ago. 2026.
- [22] PEERSPOT. *Odoo Reviews, Competitors and Pricing*. Disponível em: <https://www.peerspot.com/products/odoo-reviews>. Acesso em: 19 ago. 2026.

[23] ODOO S.A. *Odoo vs Salesforce: A CRM software comparison*. Disponível em: <https://www.odoo.com/page/odoo-vs-salesforce-crm>. Acesso em: 19 ago. 2026.

[24] KRAYIN. *laravel-crm: Krayin CRM is Free & Open Source CRM Built with Laravel*. Disponível em: <https://github.com/krayin/laravel-crm>. Acesso em: 19 ago. 2026.

[25] KRAYIN. *Requirements | Krayin CRM Developer Portal*. Disponível em: <https://devdocs.krayincrm.com/2.2/introduction/requirements.html>. Acesso em: 19 ago. 2026.

[26] KRAYIN. *Releases · krayin/laravel-crm*. Disponível em: <https://github.com/krayin/laravel-crm/releases>. Acesso em: 19 ago. 2026.

Capítulo 4: Automação de Fluxos: Trocando o Zapier pelo n8n

1. Introdução

No Capítulo 3, você mapeou a Lacuna Honesta do CRM open source — o ponto exato em que EspoCRM, Twenty e SuiteCRM cobrem o essencial, mas Salesforce e HubSpot ainda vencem por integração e maturidade de mercado. Neste capítulo essa mesma lente vai apontar para uma categoria diferente, e o resultado da medição muda: automação de fluxos é, entre tudo o que este livro cobre, o lugar onde a Lacuna Honesta encolhe até quase desaparecer. n8n, Node-RED, Huginn e Activepieces não são substitutos capengas de Zapier e Make — são ferramentas que times reais colocam em produção todos os dias, com trade-offs específicos que você vai aprender a pesar.

Ao final deste capítulo você vai saber qual dessas quatro peças pegar na prateleira da sua oficina para cada cenário, vai ter montado seu primeiro fluxo de ponta a ponta — gatilho, transformação, credencial e publicação — e vai entender exatamente que conta chega depois que a mensalidade do Zapier para de aparecer no seu cartão. Essa última parte é a que mais separa quem brinca de automação de quem administra automação de verdade, e é para lá que a segunda metade do capítulo aponta.

2. Explica

Automação de fluxos resolve um problema estrutural simples: sistemas diferentes não falam a mesma língua, e alguém precisa traduzir e disparar essa conversa sem intervenção manual repetida. Zapier e Make cobram justamente por terem feito esse trabalho de tradução uma vez, para milhares de combinações, e revendido o resultado como serviço. A pergunta que este capítulo responde é até onde um motor de automação self-hosted reproduz esse trabalho de tradução sem depender do catálogo proprietário de nenhum dos dois.

A resposta, apoiada no material técnico levantado para este livro, é que a resposta muda por ferramenta — e a maturidade de cada uma se mede por três eixos concretos: tamanho do catálogo de integrações, cadência de manutenção do projeto e capacidade de isolar execuções (multi-tenancy). n8n lidera nos três eixos ao mesmo tempo: é fair-code, tem cerca de 400 integrações nativas e embarca capacidades de IA, com um projeto que já soma mais de 201 mil estrelas no GitHub — um sinal direto de adoção real, não de hype passageiro [1]. Para comparação, o catálogo do Zapier passa de 9.000 aplicativos [2], o que deixa claro que “paridade real” não significa “catálogo idêntico” — significa que os casos de uso mais comuns (webhooks, APIs REST, planilhas, e-mail, bancos de dados) já estão cobertos, e o restante é excepcional, não a regra.

Node-RED nasceu para outro problema: orquestrar sensores e dispositivos IoT, não sincronizar CRMs com ferramentas de e-mail marketing [3]. Herda dessa origem uma limitação arquitetural que pesa: é single-threaded e não isola execuções por cliente ou por fluxo, então um node mal escrito pode travar o event loop inteiro para todos os fluxos que rodam na mesma instância [4]. Isso não o desqualifica — só define o perímetro correto de uso, que é diferente do de um substituto de Zapier.

Huginn é o mais antigo dos quatro e o mais flexível conceitualmente: modela cada tarefa como um “agente” que observa e reage a eventos, em um grafo configurável. Também é o que carrega mais sinais de manutenção incerta — a última versão com tag oficial de release remonta a agosto de 2022, mesmo com atividade contínua de commits [5], e o projeto acumula hoje cerca de 600 issues abertas e 91 pull requests pendentes de revisão [6]. Já Activepieces é o mais jovem: nasceu com arquitetura moderna (workers isolados em sandbox, fila baseada em Redis) e cresceu rápido — mais de 23 mil estrelas em pouco tempo de vida [7] — mas ainda carrega uma biblioteca pequena, algo entre 300 e 500 “pieces”, contra os milhares de apps do concorrente pago [8].

3. Ilustra

Pense na sua oficina como um estoque com quatro peças específicas na prateleira de automação, cada uma com uma etiqueta de garantia diferente. n8n é a peça com garantia mais longa e mais completa: veio testada por uma comunidade enorme, tem manual de instruções atualizado e um catálogo de acessórios (as integrações) que cobre praticamente toda tarefa comum de bancada. Node-RED é uma peça excelente, mas com etiqueta de garantia que avisa “projetada para eletrônica e sensores” — funciona bem fora dessa faixa, mas o fabricante nunca prometeu isolamento entre projetos diferentes rodando ao mesmo tempo na mesma bancada. Huginn é aquela ferramenta robusta, comprada de um fabricante que parou de responder e-mail — ainda funciona, mas você está por conta própria se quebrar. Activepieces é a ferramenta nova na caixa, bem construída, mas com o catálogo de acessórios ainda enxuto.

%% legenda: Comparativo das quatro pecas de automacao da prateleira frente a bancada Zapier/Make

flowchart LR

N8N[n8n - maior garantia] --> BANCADA{Bancada de comparacao}

NR[Node-RED - IoT e sensores] --> BANCADA

HG[Huginn - sem suporte do fabricante] --> BANCADA

AP[Activepieces - novo, catalogo enxuto] --> BANCADA

BANCADA --> ZAP[Zapier / Make - referencia paga]

ZAP -.compara-se com.-> BANCADA

Agora, como Artesão Digital, você precisa entender a peça mais importante que vai montar nesta oficina: a esteira de montagem de um fluxo de automação. Toda automação — seja no n8n, no Zapier ou em qualquer motor do mercado — segue a mesma esteira de quatro estações: um **gatilho** dispara o movimento (um webhook chega, um horário bate, um e-mail entra), a peça passa por um ou mais **nós** que transformam ou movem os dados, ela pega uma **credencial** guardada no cofre da oficina para acessar um sistema externo, e finalmente o fluxo é **publicado** (deploy) para rodar sozinho, sem você por perto.

A parte que costuma confundir quem está começando é a estação da credencial, então vale uma segunda analogia, mais específica: pense na credencial não como uma chave qualquer que fica solta na bancada, mas como um crachá de acesso guardado num cofre trancado dentro da própria oficina — o n8n nunca expõe a senha ou o token em texto puro dentro do fluxo, ele guarda o crachá no cofre (o *Credential Store* interno) e só entrega para o nó certo, na hora certa, sem que o restante da esteira veja o conteúdo. Entender essa separação — o fluxo pede acesso, o cofre concede, o fluxo nunca vê a chave crua — é o que separa quem monta uma automação funcional de quem monta uma automação que um dia vaza uma credencial de produção sem perceber.

%% legenda: Esteira de montagem de um fluxo n8n do gatilho ao deploy

flowchart TB

A[Gatilho: Webhook] --> B[Nó de transformacao]

B --> C{Precisa de credencial?}

C -->|sim| D[Cofre de credenciais]

D --> E[No autenticado]

C -->|nao| E

E --> F[Deploy: fluxo ativo]

F -->|falha| G[Alerta de execucao]

4. Técnica

Chegou a hora de sair da prateleira e montar a peça na bancada. Vamos subir o n8n com Docker, configurar o banco de dados que ele exige em qualquer cenário sério e publicar um fluxo real:

um webhook recebe um pedido, um nó transforma o dado e um nó autenticado envia o resultado para um serviço externo usando uma credencial guardada no cofre.

Por que Docker e por que PostgreSQL desde o primeiro dia

A documentação oficial recomenda Docker com volume persistente e trata o PostgreSQL como obrigatório a partir do momento em que o fluxo sai do “só para testar” — o SQLite embutido (usado por padrão sem configuração) não sobrevive bem a cargas concorrentes [9]. Um benchmark de produção publicado recentemente identificou o banco de dados, o event loop do próprio Node.js e a ausência de isolamento de execução entre workflows como os três gargalos reais quando a instância cresce [10] — não são gargalos hipotéticos, são os pontos que quebram primeiro.

O guia de dimensionamento de infraestrutura para n8n é direto sobre o tamanho da bancada que você precisa: uma instância de desenvolvimento ou teste roda confortavelmente com 2 GB de RAM e 2 vCPUs, mas uma instância de produção — com fila de execução ativa (queue mode) e volume real de fluxos — exige entre 8 e 16 GB de RAM e 4 ou mais vCPUs [11]. Essa diferença de uma ordem de grandeza é exatamente o tipo de custo que a assinatura de SaaS escondia dentro da mensalidade, e ele vai reaparecer com força na seção Aplica.

```
# docker-compose.yml – bancada mínima para produção do n8n
version: "3.8"

services:
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: n8n
      POSTGRES_PASSWORD: "troque-esta-senha-no-deploy"
      POSTGRES_DB: n8n
  volumes:
    - postgres_data:/var/lib/postgresql/data

n8n:
  image: docker.n8n.io/n8nio/n8n:latest
  restart: unless-stopped
  ports:
    - "5678:5678"
  environment:
    DB_TYPE: postgresdb
    DB_POSTGRESDB_HOST: postgres
    DB_POSTGRESDB_DATABASE: n8n
    DB_POSTGRESDB_USER: n8n
    DB_POSTGRESDB_PASSWORD: "troque-esta-senha-no-deploy"
```

```

N8N_ENCRYPTION_KEY: "<chave-de-32-caracteres-gerada-no-deploy>"
WEBHOOK_URL: "https://automacao.suaoficina.com.br/"
GENERIC_TIMEZONE: "America/Sao_Paulo"
volumes:
  - n8n_data:/home/node/.n8n
depends_on:
  - postgres

volumes:
  postgres_data:
  n8n_data:

```

Subindo a bancada e testando o primeiro gatilho

Com o `docker-compose.yml` salvo, a montagem começa com um único comando. A sessão abaixo mostra a subida dos dois serviços e a verificação de que o n8n respondeu na porta esperada.

```

$ docker compose up -d
[+] Running 2/2
  - Container oficina-postgres-1   Started
  - Container oficina-n8n-1        Started

$ docker compose ps
NAME                IMAGE                                STATUS
oficina-postgres-1  postgres:16                        Up 12 seconds (healthy)
oficina-n8n-1       docker.n8n.io/n8nio/n8n:latest    Up 10 seconds

$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:5678/healthz
200

```

Com a instância no ar, a interface web em `http://localhost:5678` guia a criação de uma conta de administrador local — esse é o único cadastro que existe: nenhum dado do seu fluxo passa por servidor de terceiros a partir daqui.

O fluxo de ponta a ponta: gatilho, nó, credencial e deploy

O JSON abaixo é a exportação real de um fluxo mínimo: um gatilho `Webhook`, um nó `Set` que normaliza o payload recebido e um nó `HTTP Request` que usa uma credencial de cabeçalho (header auth) para reenviar o dado a um serviço externo — o mesmo desenho de esteira ilustrado na seção anterior, só que agora como artefato real, importável no próprio n8n.

```

{
  "name": "Pedido recebido -> normaliza -> reenvia",
  "nodes": [
    {
      "id": "gatilho-webhook",
      "name": "Gatilho: Webhook",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 2,
      "position": [240, 300],
      "parameters": {
        "path": "pedido-novo",
        "httpMethod": "POST",
        "responseMode": "lastNode"
      }
    },
    {
      "id": "no-transformacao",
      "name": "Normaliza payload",
      "type": "n8n-nodes-base.set",
      "typeVersion": 3,
      "position": [460, 300],
      "parameters": {
        "assignments": {
          "assignments": [
            { "name": "cliente", "type": "string", "value":
"={{$json.nome}}" },
            { "name": "valor_pedido", "type": "number", "value":
"={{$json.total}}" }
          ]
        }
      }
    },
    {
      "id": "no-autenticado",
      "name": "Reenvia com credencial",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
      "position": [680, 300],
      "parameters": {
        "method": "POST",
        "url": "https://api.parceiro-externo.com/pedidos",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpHeaderAuth"
      },
      "credentials": {
        "httpHeaderAuth": {
          "id": "1",

```

```

        "name": "Cofre - Parceiro Externo"
      }
    }
  ],
  "connections": {
    "Gatilho: Webhook": {
      "main": [[{ "node": "Normaliza payload", "type": "main", "index":
0 }]]
    },
    "Normaliza payload": {
      "main": [[{ "node": "Reenvia com credencial", "type": "main",
"index": 0 }]]
    }
  }
}

```

Note que o bloco `credentials` do último nó só guarda um `id` e um `name` — a chave real (o token de autenticação do parceiro) nunca aparece neste arquivo. Ela vive isolada no cofre interno do n8n, cifrada com a `N8N_ENCRYPTION_KEY` definida no `docker-compose.yml`, exatamente como a analogia do crachá previu: o fluxo pede acesso, o cofre concede, o arquivo do fluxo nunca vê a chave crua.

Depois de importar esse JSON pela interface e clicar em “Activate”, o fluxo passa a escutar de verdade. A sessão a seguir mostra o teste do gatilho com uma chamada real, simulando o sistema que dispararia esse pedido em produção:

```

$ curl -X POST https://automacao.suaoficina.com.br/webhook/pedido-novo \
-H "Content-Type: application/json" \
-d '{"nome": "Ana Ferreira", "total": 189.90}'

{"reenviado": true, "cliente": "Ana Ferreira", "valor_pedido": 189.9}

```

Escolhendo a peça certa: tabela de decisão

Antes de sair da bancada, vale consolidar a comparação da seção Explica em uma tabela de decisão prática — o tipo de referência rápida que você vai consultar de novo antes de montar o próximo fluxo:

Cenário do leitor	Peça recomendada	Por quê
Preciso do catálogo mais amplo e ativo hoje	n8n	400+ integrações nativas, 201 mil+ estrelas, IA embutida [1]
Automação de sensores, dispositivos, IoT doméstico	Node-RED	Nasceu para esse domínio; não é multi-tenant [4]
Aceito manter Ruby/DevOps sem suporte comercial	Huginn	Flexível, mas sem release tagueado desde 2022 [5]
Quero arquitetura moderna e aceito catálogo menor	Activepieces	Workers isolados, mas só 300-500 pieces [8]
Dependo de uma integração nichada específica	Nenhuma das quatro — avalie manter o Zapier	Nenhuma cobre o catálogo de 9.000+ apps [2]

Backup da bancada: o script que evita perder o fluxo montado

Uma última peça técnica, curta e direta: como qualquer ferramenta da oficina, a instância self-hosted não tem backup automático de fábrica — isso é responsabilidade de quem monta. O script abaixo, agendável via `cron`, exporta o banco PostgreSQL do n8n para um arquivo comprimido diário.

```
#!/usr/bin/env bash
set -euo pipefail

DATA=$(date +%F)
DESTINO="/backups/n8n/n8n-`${DATA}`.sql.gz"

mkdir -p /backups/n8n

docker exec oficina-postgres-1 pg_dump -U n8n n8n | gzip > "${DESTINO}"

echo "Backup gravado em `${DESTINO}`"

find /backups/n8n -name "n8n-*.sql.gz" -mtime +30 -delete
```

5. Aplica

Imagine a cena: seu fluxo está pronto, testado, publicado. Ele reenvia pedidos para o sistema do parceiro há duas semanas sem incidente. Numa sexta-feira, você decide “otimizar” e edita o nó de credencial direto na tela, copiando o token novo do parceiro e colando-o, por pressa, dentro

do campo de texto do nó `HTTP Request` — não no campo de credencial dedicado, mas como um cabeçalho fixo, “só para testar rápido”. Funciona. Você esquece de voltar e corrigir depois. Três meses depois, esse fluxo é exportado como JSON para ser versionado no Git do time — e o token do parceiro vai embutido, em texto puro, para o repositório.

O diagnóstico é exatamente o ponto que a seção Ilustra separou com a segunda analogia: o cofre de credenciais existe para que o *arquivo do fluxo* nunca carregue a chave crua — e colar um token dentro de um parâmetro comum, em vez de usar o sistema de credenciais dedicado, quebra essa garantia de forma silenciosa. Nada trava, nada avisa, porque tecnicamente o fluxo continua funcionando. A correção é sempre a mesma: qualquer segredo (token, senha, chave de API) entra pelo tipo de nó de credencial correto, nunca por um campo de texto livre, e uma revisão de fluxo antes de qualquer exportação para controle de versão deveria varrer justamente por esse padrão.

Esse tipo de deslize é comum o suficiente para aparecer como armadilha recorrente entre equipes que adotam automação self-hosted pela primeira vez:

- Colar segredos em campos de texto comuns em vez do nó de credencial dedicado.
- Deixar a instância em modo de desenvolvimento (SQLite, sem fila) rodando fluxos de produção real.
- Nunca testar o que acontece quando o serviço externo cai — sem retry configurado, o fluxo simplesmente perde a execução.
- Achar que “está funcionando” substitui monitoramento — sem alerta de falha, ninguém percebe a automação parada até o cliente reclamar.

No mercado, o profissional que sabe montar essa esteira sozinho — do Docker Compose ao cofre de credenciais — já entrega mais valor do que quem apenas clica em templates prontos de SaaS: ele consegue adaptar o fluxo ao sistema legado da empresa, algo que nenhum catálogo fechado de integrações prevê. É esse domínio de bastidores, e não a interface visual em si, que separa quem opera automação de quem apenas a consome.

A adoção de open source em geral ainda tropeça em processo, não em tecnologia: apenas 34% das organizações têm hoje uma estratégia formal para adotar e manter esse tipo de ferramenta [12]. E a fração que vai além da estratégia informal e mantém um escritório dedicado ao tema (um OSPO) cai para 26% [13] — o que ajuda a explicar por que tanta automação self-hosted é montada por uma única pessoa, sem plano de sucessão, e quebra silenciosamente quando essa pessoa sai de férias ou troca de emprego.

Exercício

- ☐ Suba o `docker-compose.yml` desta seção em uma VPS de teste (2 GB RAM é suficiente para começar).
- ☐ Importe o fluxo JSON do webhook e ative-o.

- ☐ Dispare o webhook com `curl` usando um payload diferente do exemplo e confirme a resposta.
- ☐ Crie a credencial do nó `HTTP Request` pelo painel de credenciais do n8n — nunca em texto livre no nó.
- ☐ Configure o script de backup via `cron` e confirme que o arquivo `.sql.gz` é gerado no dia seguinte.
- ☐ Escreva, em uma frase, quem na sua equipe (ou você mesmo) fica de guarda se esse fluxo falhar às 3h da manhã.

6. Conclusão

Três ideias sustentam este capítulo: automação de fluxos é, dos temas deste livro, o mais próximo de paridade real com o SaaS pago — mas “próximo” não é “idêntico”, e a tabela de decisão desta seção Técnica existe justamente para você escolher a peça certa em vez da mais hype; toda automação, em qualquer motor, segue a mesma esteira de gatilho, nó, credencial e deploy, e dominar essa esteira — sobretudo a disciplina do cofre de credenciais — é o que separa uma automação funcional de uma automação segura; e o custo que a mensalidade do Zapier escondia não desapareceu ao trocar de ferramenta, apenas migrou de forma para VPS, backup e, principalmente, para a pergunta que fechou a seção Aplica: quem fica de guarda quando o fluxo quebra.

Como Artesão Digital, você já tem agora duas peças montadas na sua oficina — anotações e automação — e sabe pesar, categoria por categoria, quando a troca vale o esforço. No Capítulo 5, a bancada muda de assunto: design gráfico e edição de imagem, onde GIMP, Inkscape, Krita e Penpot substituem Canva, Photoshop e Illustrator com uma curva de aprendizado bem menos gentil do que a que você acabou de vencer aqui — e que vale conhecer antes de prometer para o seu time que a troca sai de graça.

7. Referências Bibliográficas

[1] ACTIVEPIECES. *activepieces/activepieces: AI Agents & MCPs & AI Workflow Automation*. Disponível em: <https://github.com/activepieces/activepieces>. Acesso em: 19 ago. 2026.

[2] ACTIVEPIECES. *Architecture Overview — Activepieces Docs*. Disponível em: <https://www.activepieces.com/docs/install/architecture/overview>. Acesso em: 19 ago. 2026.

[3] AUFRANC, Jean-Luc (CNX SOFTWARE). *Huginn is a self-hosted, open-source alternative to IFTTT and Zapier*. Disponível em: <https://www.cnx-software.com/2025/04/05/huginn-is-a-self-hosted-open-source-alternative-to-ifttt-and-zapier/>. Acesso em: 19 ago. 2026.

- [4] CHERRY SERVERS. *n8n Self-Hosting Requirements Guide (2026)*. Disponível em: <https://www.cherryservers.com/blog/n8n-self-hosting-requirements>. Acesso em: 19 ago. 2026.
- [5] CODEREVUE. *Using Node-RED as an open-source alternative to Zapier for workflow automation*. Disponível em: <https://coderevue.net/posts/zapier-alternative-node-red/>. Acesso em: 19 ago. 2026.
- [6] HARDILL, Ben. *Multi Tenant Node-RED*. Disponível em: <https://blog.hardill.me.uk/2020/10/01/multi-tenant-node-red/>. Acesso em: 19 ago. 2026.
- [7] HUGINN. *huginn/huginn: Create agents that monitor and act on your behalf*. Disponível em: <https://github.com/huginn/huginn>. Acesso em: 19 ago. 2026.
- [8] HUGINN. *Releases · huginn/huginn*. Disponível em: <https://github.com/huginn/huginn/releases>. Acesso em: 19 ago. 2026.
- [9] LINUX FOUNDATION RESEARCH. *The State of Global Open Source 2025*. Disponível em: <https://www.linuxfoundation.org/blog/the-state-of-open-source-software-in-2025>. Acesso em: 19 ago. 2026.
- [10] N8N. *n8n Documentation*. Disponível em: <https://docs.n8n.io/>. Acesso em: 19 ago. 2026.
- [11] N8N. *n8n-io/n8n: Fair-code workflow automation platform with native AI capabilities*. Disponível em: <https://github.com/n8n-io/n8n>. Acesso em: 19 ago. 2026.
- [12] OPENLOGIC. *2026 State of Open Source Report*. Disponível em: <https://www.openlogic.com/resources/state-of-open-source-report>. Acesso em: 19 ago. 2026.
- [13] OPENJS FOUNDATION. *node-red/node-red*. Disponível em: <https://github.com/node-red/node-red>. Acesso em: 19 ago. 2026.
- [14] N8N.REVIEWS. *n8n at Scale: Performance Benchmarks Under Real Production Load*. Disponível em: <https://n8n.reviews/n8n-at-scale/>. Acesso em: 19 ago. 2026.
- [15] BETTER STACK COMMUNITY. *Open-Source Workflow Automation with Activepieces*. Disponível em: <https://betterstack.com/community/guides/ai/activepieces-workflow-automation/>. Acesso em: 19 ago. 2026.
- [16] ZAPIER. *n8n vs. Zapier: Which Automation Tool is Right For You?*. Disponível em: <https://zapier.com/blog/n8n-vs-zapier/>. Acesso em: 19 ago. 2026.

Capítulo 5: Design Gráfico e Edição de Imagem: Sem Canva, Sem Photoshop, Sem Illustrator

1. Introdução

No Capítulo 4, a Oficina Digital ganhou a primeira peça que trabalha sozinha: o n8n, montado na sua bancada, passou a mover dados e disparar fluxos sem assinatura mensal. Agora chegou a vez das peças que produzem as artes. Canva, Photoshop e Illustrator são, hoje, as ferramentas mais populares de design e edição de imagem do planeta — e também três das assinaturas mais caras que um Artesão Digital pode carregar no cartão, todo mês, sem nunca ter parado para somar o total. Este capítulo faz essa conta e monta a substituição peça por peça: GIMP no lugar do Photoshop, Inkscape no lugar do Illustrator, Krita para pintura digital e Penpot no lugar do Figma e do Canva colaborativo.

A promessa aqui não é a mesma dos vídeos de marketing que mostram “photoshop grátis em 5 minutos”. A promessa é mais honesta e mais útil: você vai aprender exatamente o que cada ferramenta cobre bem, onde a curva de aprendizado assusta o iniciante, como instalar tudo sem pagar um centavo de licença e como percorrer um fluxo de trabalho completo — do zero até uma peça de marketing pronta. Ao final, a pergunta “qual ferramenta eu uso para isso?” terá uma resposta sua, baseada em dado, não em hábito.

2. Explica

Comece pela pergunta que a maioria pula: o que exatamente você compra quando assina o Canva ou a Creative Cloud da Adobe? Não é o software — é o acesso ao software, hospedado ou licenciado sob um contrato que acaba quando o pagamento para. É o mesmo modelo da loja de equipamentos que aluga furadeiras: você paga para usar a peça, mas ela nunca é sua, e no dia em que o aluguel vence sem renovação, a peça some da sua bancada. No caso do Canva, os seus projetos ficam sob custódia da empresa; no caso da Creative Cloud, os programas param de abrir. A mensalidade não compra posse — compra permanência temporária.

Open source resolve exatamente essa parte do problema, e é importante ser preciso sobre o que isso significa. Código aberto quer dizer que o código-fonte do programa é público, licenciado para leitura, modificação e redistribuição. O Penpot, por exemplo, é mantido pela Kaleidos sob licença MPL-2.0, com um repositório ativo que já soma 58,9 mil estrelas no GitHub [1] — você pode ler o código, contribuir e, se quiser, hospedar a sua própria instância. GIMP, Inkscape e Krita seguem o mesmo princípio, com repositórios oficiais abertos em GitLab e GitHub [10] [15][19][20]. Isso é diferente de “gratuito”: muita ferramenta gratuita não é aberta, e projeto aberto pode ter camada paga por cima — o que importa é que o núcleo é seu.

Agora, a parte que ninguém conta nos vídeos de comparação: a curva de aprendizado. GIMP, Inkscape e Krita são aplicativos desktop nativos, gratuitos e sem modelo de assinatura, que cobrem boa parte do fluxo raster, vetor e pintura do dia a dia [18]. A cobertura é real — reviews independentes estimam que o trio cobre cerca de 85% do fluxo vetor e raster diário sem custo [18]. Mas a interface não é o Canva: não existe caixa de pesquisa que monta o layout para você. No GIMP você trabalha com camadas, canais e máscaras; no Inkscape, com nós, caminhos e curvas de Bézier; no Krita, com pincéis e motores de pintura. Cada um desses conceitos tem um custo de aprendizado inicial — e é exatamente esse custo que assusta o iniciante na primeira hora de uso.

A curva assusta, mas a direção dela é a certa: ela sobe. O GIMP 3.0, lançado vinte anos depois do 2.0, foi recebido por reviews independentes com nota de evolução de interface, embora o port para Windows tenha sido historicamente inferior ao do Linux [11][12]. O Krita 5.2.6 foi classificado por uma review dedicada como “finalmente uma alternativa real” ao Photoshop — no seu nicho, pintura digital [23]. O Inkscape segue como a referência livre de desenho vetorial, com instalador MSI oficial para Windows [14][16]. O que separa quem desiste na primeira semana de quem domina a ferramenta não é talento: é entender que essas peças exigem montagem — instalação, configuração de espaço de trabalho, atualizações — e que essa responsabilidade, antes embutida no aluguel da loja, agora é sua.

Vale também ser preciso sobre as lacunas, porque escondê-las seria desonesto e inútil para você. As três lacunas reais da categoria, confirmadas por reviews independentes, são: produção profissional para impressão (GIMP e Inkscape não têm suporte nativo a CMYK, o que afeta quem precisa de saída certificada para gráfica) [11][13][17]; edição de RAW fotográfico (o GIMP não tem suporte nativo, exigindo ferramenta auxiliar como darktable); e IA generativa (nenhuma das quatro ferramentas tem equivalente maduro ao Generative Fill do Photoshop) [11][13]. Entender essas fronteiras antes de trocar é o que separa o Artesão Digital do amador que troca por fé — e é o assunto da seção Aplica deste capítulo.

3. Ilustra

Pense na sua oficina montando uma mesa de trabalho de design. Na loja de aluguel, você recebia três peças emprestadas, sempre prontas, sempre atualizadas: a bancada de fotografia (Photoshop), a bancada de desenho técnico (Illustrator) e a bancada de montagem rápida (Canva). O aluguel cobria manutenção, garantia e um atendente que resolvia quando algo travava. Ao comprar as suas próprias peças, você ganha posse — mas ganha também a responsabilidade de montar cada bancada, ler o manual, calibrar as ferramentas e manter a garantia por conta própria, na documentação e nos fóruns da comunidade.

Agora dê um nome às peças compradas. A bancada de fotografia é o GIMP: raster, edição de imagem, retoque, composição de camadas — o equivalente direto do Photoshop para o fluxo do dia a dia [8][9]. A bancada de desenho técnico é o Inkscape: vetor, logotipos, ícones, diagramas e qualquer arte que precise ser redimensionada sem perder qualidade [14] [15]. A bancada de pintura é o Krita: desenho digital, ilustração e concept art, com mais de 120 pincéis com emulação realista [22][23]. E a bancada colaborativa é o Penpot: design de interface rodando no navegador, com edição em equipe em tempo real — o equivalente aberto do Figma, com a diferença crucial de que o Inspect Mode, pago no Figma, é gratuito nele [5].

É exatamente essa troca que acontece quando você sai do aluguel do Canva para o Penpot: a peça deixa de ser um serviço na nuvem de outra empresa e passa a ser uma ferramenta que você pode hospedar onde quiser, ou usar na nuvem oficial gratuita [2]. Como Artesão Digital, é essa segunda lente que você vai treinar no restante do livro: perguntar não apenas “essa peça funciona?”, mas “que garantia eu perco ao trocar, e estou disposto a assumir essa manutenção?”.

O diagrama abaixo resume o fluxo de decisão que guia este capítulo — da ideia da peça de marketing até a peça pronta na bancada.

%% legenda: Rota da peça de marketing na Oficina Digital: do vetor no Inkscape ao raster no GIMP, com o Krita como opção de pintura e o Penpot na colaboração de interface

flowchart TB

```

A[Ideia da peça de marketing] --> B{0 que a peça precisa}
B -->|Vetor, logo ou ícone| C[Inkscape: desenho vetorial]
B -->|Foto, retoque ou composição| D[GIMP: edição raster]
B -->|Ilustração ou pintura digital| E[Krita: pintura]
B -->|Interface ou colaboração em equipe| F[Penpot: design no navegador]
C --> G[Exportar SVG ou PNG]
D --> H[Exportar PNG ou JPG]
E --> H
F --> I[Inspect Mode gratuito para o dev]
G --> J[Revisar e publicar a peça]
H --> J

```

```
I --> J
J --> K[Peca pronta na bancada do Artesao Digital]
```

4. Técnica

Esta seção é onde a Oficina Digital sai do papel. Você vai instalar as quatro peças, subir o Penpot com Docker Compose e montar o fluxo real de produção de uma peça de marketing — tudo com comandos que pode copiar e colar, sem código de programação, porque o que você precisa aqui é operar as ferramentas, não desenvolvê-las.

Instalando as Peças Desktop: GIMP, Inkscape e Krita

Os três aplicativos são nativos para Windows, macOS e Linux [9][14][22]. No Windows, o caminho mais rápido é o instalador oficial: o Inkscape oferece MSI oficial [16], o GIMP e o Krita distribuem instaladores próprios a partir dos sites oficiais [8][21]. Se você prefere gerenciar tudo por linha de comando, o winget resolve as três instalações:

```
$ winget install GIMP.GIMP
$ winget install Inkscape.Inkscape
$ winget install KDE.Krita
```

No Linux (Ubuntu/Debian), os três estão nos repositórios oficiais e a instalação é um único comando por ferramenta:

```
sudo apt update
sudo apt install -y gimp inkscape krita # exemplo didatico: requer um
ambiente Linux com sudo
```

Subindo o Penpot com Docker Compose

O Penpot é a peça colaborativa da bancada: design de interface no navegador, com edição em equipe em tempo real. A rota self-hosted recomendada é Docker Compose, com PostgreSQL como banco e um volume persistente para os assets [3][4]. O manifesto abaixo é a base mínima documentada:

```
# docker-compose.yml – Penpot self-hosted (referência mínima de produção)
services:
  penpot_postgres:
```

```

image: postgres:15-alpine
restart: unless-stopped
environment:
  POSTGRES_INITDB_ARGS: "--data-checksums"
  POSTGRES_DB: penpot
  POSTGRES_USER: penpot
  POSTGRES_PASSWORD: "troque-esta-senha"
volumes:
  - penpot_pg_data:/var/lib/postgresql/data

penpot_backend:
  image: penpotapp/backend:latest
  restart: unless-stopped
  depends_on:
    - penpot_postgres
  environment:
    PENPOT_FLAGS: "enable-registration enable-login-with-password"
    PENPOT_DATABASE_URI: "postgresql://penpot:troque-esta-senha@penpot_
postgres/penpot"
    PENPOT_PUBLIC_URI: "https://design.suaoficina.dev"
    PENPOT_ASSETS_STORAGE_BACKEND: "assets-fs"
    PENPOT_ASSETS_STORAGE_FS_DIRECTORY: "/opt/data/assets"
    PENPOT_TELEMETRY_ENABLED: "false"
  volumes:
    - penpot_assets:/opt/data/assets

penpot_frontend:
  image: penpotapp/frontend:latest
  restart: unless-stopped
  depends_on:
    - penpot_backend
  ports:
    - "8080:8080"

volumes:
  penpot_pg_data:
  penpot_assets:

```

Suba a stack e confirme que os serviços entraram no ar:

```

$ docker compose up -d
[+] Running 3/3
✓ Container penpot_postgres Started
✓ Container penpot_backend Started
✓ Container penpot_frontend Started

$ docker compose ps

```

NAME	STATUS
penpot_postgres	Up 5 seconds (healthy)
penpot_backend	Up 4 seconds
penpot_frontend	Up 3 seconds

Depois disso, o passo seguinte é o mesmo da documentação oficial de self-hosting: apontar um reverse proxy com HTTPS para a porta 8080 e criar a primeira conta pela interface [3][4]. Se você não quer administrar servidor nenhum, a nuvem oficial gratuita do Penpot elimina essa etapa — o trade-off é exatamente o aluguel que você está tentando deixar para trás, então a escolha é sua [2].

Do Zero a uma Peça de Marketing Pronta

O fluxo completo de uma peça de marketing típica — um banner para redes sociais — usa as quatro bancadas em sequência:

1. **Inkscape**: desenhe o logotipo ou os ícones em vetor. Vetor significa que a arte é definida por caminhos matemáticos, não por pixels: você pode redimensionar para o tamanho de um outdoor sem perder qualidade [14]. Exporte como SVG.
2. **GIMP**: monte a composição final — fundo, foto, logotipo vetorial importado como camada e textos. Trabalhe sempre em camadas separadas; é o equivalente a guardar cada peça da sua bancada em uma gaveta própria [9].
3. **Krita**: se a peça pedir uma ilustração ou pintura original (um personagem, uma textura desenhada à mão), produza aqui, com os pincéis de emulação realista [23].
4. **Penpot**: se a peça for parte de uma interface maior — um banner de site, um card de produto, uma tela de app — monte a estrutura no Penpot e use o Inspect Mode gratuito para repassar medidas e cores a quem for implementar [5].

O ciclo de exportação fecha o fluxo: PNG para imagens com transparência, JPG para fotos e publicações que não precisam de transparência, SVG para vetores que vão ser reutilizados. Nenhuma das quatro ferramentas cobra um centavo por exportação — no Canva, recursos como fundo transparente e exportação em alta resolução ficam atrás do paywall; aqui, são botões padrão.

Tabela de Decisão: Qual Peça Pegar na Prateleira

Ferramenta	Licença	Modelo	Melhor uso	Evite se...
GIMP	Aberta [10]	Desktop nativo	Retoque, composição raster e edição de imagem [8]	Fluxo exige CMYK nativo ou RAW sem ferramenta auxiliar [11][13]
Inkscape	Aberta [15]	Desktop nativo	Vetor, logotipos, ícones e diagramas escaláveis [14]	Arquivos vetoriais muito grandes (instabilidade documentada) [17]
Krita	Aberta [19]	Desktop nativo	Pintura digital e ilustração (120+ pincéis) [22]	Uso principal é edição fotográfica, não pintura [23]
Penpot	MPL-2.0 [1]	Self-hosted ou nuvem gratuita	Design de interface colaborativo no navegador [2]	Time depende do marketplace de plugins do Figma ou de templates prontos do Canva [6] [7]

Use a tabela como ponto de partida, não como veredito final. Ela resume o que a documentação oficial e os reviews independentes desta seção já mostraram: o trio desktop cobre cerca de 85% do fluxo vetor e raster do dia a dia sem custo [18], e o Penpot já é considerado um substituto viável do Figma para design colaborativo — mas cada peça tem o seu perímetro exato de uso [5][6].

A Automação da Bancada: Scripts que Preparam e Exportam

A bancada de design ganha velocidade quando as tarefas repetitivas viram scripts — e o Artesão Digital que já conhece o básico de terminal do capítulo 4 pode automatizar a parte mais chata do fluxo: a preparação dos arquivos para publicação. O `ImageMagick`, companheiro de linha de comando do GIMP e do Inkscape, redimensiona, converte e aplica efeitos em lote com um comando — e a sessão abaixo mostra o caso mais comum, a geração de todas as variações de uma peça para as redes sociais a partir do arquivo mestre:

```
$ mkdir -p publicacao && cd publicacao
$ convert ../peca-mestre.png -resize 1080x1080^ -gravity center -extent 1080x1080 post-quadrado.png
$ convert ../peca-mestre.png -resize 1080x1920^ -gravity center -extent 1080x1920 story-vertical.png
$ convert ../peca-mestre.png -resize 1200x630^ -gravity center -extent 1200x630 link-preview.png
$ ls -lh
-rw-r--r-- 1 artesao artesao 412K ago 20 07:00 link-preview.png
-rw-r--r-- 1 artesao artesao 489K ago 20 07:00 post-quadrado.png
-rw-r--r-- 1 artesao artesao 812K ago 20 07:00 story-vertical.png
```

O comando `convert` do ImageMagick faz o redimensionamento com corte central (o trecho `^` combinado com `-extent`), mantendo o enquadramento da peça em cada formato — o mesmo efeito que o Canva faz por clique, mas em lote e sem depender de internet [8][12]. A linha de baixo mostra o resultado: três formatos prontos a partir de um único arquivo mestre, sem abrir nenhum programa gráfico. A automação não substitui o design — substitui a repetição, e libera o tempo do Artesão Digital para o que nenhum script faz: a decisão visual.

A Curva de Aprendizado: O Que Realmente Custava Caro

A fatura escondida do software de design nunca foi o preço da licença — foi o tempo para aprender a usar a ferramenta. O Photoshop e o Illustrator cobram mensalidade; o GIMP e o Inkscape cobram horas de estudo. A revisão da Creative Bloq sobre o Krita 5.2.6 resume a virada: depois de anos sendo tratado como alternativa “de nicho”, o Krita finalmente foi reconhecido como uma alternativa real ao Photoshop para pintura digital [23] — mas o reconhecimento não elimina a curva. Os requisitos são honestos: o Krita recomenda 8 GB de RAM, e 16 GB para telas 4K [22]; o GIMP e o Inkscape rodam em máquinas modestas, mas lidam mal com arquivos muito grandes, com instabilidade documentada em projetos pesados [17]. A régua da seção 5 decide quando a curva vale a pena: para o fluxo diário de marketing, a curva é curta e o retorno é imediato; para a produção profissional de impressão com saída CMYK certificada, a curva continua íngreme em qualquer ferramenta gratuita, e a troca não compensa [11][17]. A transparência dessa régua é o que separa a oficina séria da promessa de marketing: o open source é grátis, mas nada nele é instantâneo.

5. Aplica

Imagine a cena: você acabou de instalar as quatro peças, animado com a conta que vai economizar, e precisa entregar uma peça de marketing para a semana de lançamento — um banner de 1200x628 pixels para uma postagem patrocinada. Você abre o GIMP pela primeira vez e a

interface mostra dezenas de janelas flutuantes, painéis de camadas, canais, caminhos e histórico. Você tenta “simplesmente” recortar o produto da foto de fundo com a ferramenta de seleção livre, erra o contorno, tenta desfazer, o recorte fica serrilhado, e em vinte minutos você está convencido de que open source é impossível e de que “o Photoshop era melhor”.

Pare. Esse é o erro clássico, e o diagnóstico é simples: você está usando a bancada certa do jeito errado. A curva de aprendizado do GIMP não é sobre inteligência — é sobre conceito. Recortar um objeto de uma foto não se faz com seleção livre à mão livre; faz-se com o caminho de Bézier (a ferramenta “Caminhos”), que desenha curvas suaves ao redor do objeto, converte o caminho em seleção e aplica a máscara. É o mesmo método que os profissionais usam no Photoshop, e ele leva algumas horas de prática para fluir — não vinte minutos. A correção não é voltar para o aluguel: é aceitar que a primeira peça de marketing vai levar o dobro do tempo, e que isso é investimento, não prejuízo.

Além da cena acima, vale registrar as outras armadilhas comuns dessa migração:

- **Trocar a ferramenta pelo fluxo errado:** usar o GIMP para desenhar um logotipo que vai ser impresso em vários tamanhos. Logotipo é vetor — a peça certa é o Inkscape, que exporta SVG e escala sem perda [14]. O GIMP é raster; ampliar um logotipo em raster é pedir serrilhado.
- **Prometer paridade CMYK para a gráfica:** GIMP e Inkscape não têm suporte nativo a CMYK [11][13][17]. Se o seu fluxo principal é impressão profissional certificada, essa é a lacuna honesta — mantenha o aluguel para esse fluxo específico ou contorne com plugins e ferramentas auxiliares, e deixe isso combinado com a gráfica antes de prometer a troca para o time.
- **Depender de IA generativa:** o Generative Fill do Photoshop não tem equivalente maduro nas quatro ferramentas [11][13]. Se a sua rotina depende de preenchimento generativo de imagens, nenhuma das peças desta bancada substitui esse fluxo hoje — planeje o contorno (remover o elemento manualmente com carimbo e clonagem) antes de migrar.
- **Subestimar o ecossistema:** o Penpot tem base de usuários menor e menos plugins de terceiros que o Figma, e o Canva vende templates prontos que não têm equivalente direto [6][7]. Times que vivem de marketplace de plugins ou de templates prontos vão sentir a troca — o contorno é construir a sua própria biblioteca de componentes no Penpot, e isso também é trabalho de bancada.

O profissional — o Artesão Digital que você está se tornando — não mede a troca pelo preço, mede pelo fluxo. Antes de migrar qualquer equipe, rode o piloto sozinho: produza três peças reais do início ao fim, cronometre, anote onde cada ferramenta travou e onde fluiu. Só depois disso a decisão entre manter o aluguel para um fluxo específico e comprar a peça para o resto é uma decisão com dado na mesa — e é exatamente essa disciplina que separa quem brinca de open source de quem administra uma oficina.

6. Conclusão

Três ideias sustentam este capítulo. Primeira: o trio GIMP, Inkscape e Krita cobre cerca de 85% do fluxo vetor e raster diário sem custo [18], e o Penpot já é um substituto viável do Figma para design colaborativo [5] — a bancada completa de design da Oficina Digital existe, é gratuita e é sua. Segunda: a curva de aprendizado é real e assusta o iniciante, mas é uma curva de conceitos — camadas, caminhos, vetores — que se domina com prática deliberada, e o retorno é a posse definitiva da peça, sem mensalidade para sempre [11][12][23]. Terceira: as lacunas honestas — CMYK nativo, RAW, IA generativa e ecossistema de plugins — definem o perímetro da troca, e conhecê-las antes de prometer a migração para o seu time é o que separa o Artesão Digital do amador [6][11][13][17].

Como desafio prático: produza a sua primeira peça de marketing completa com as quatro bancadas — um vetor no Inkscape, uma composição no GIMP, uma ilustração no Krita (se a peça pedir) e uma tela no Penpot, hospedado onde você escolher. Cronometre o tempo total e anote as três dores que aparecerem. Esse registro é o seu mapa de manutenção da bancada, e ele vai servir para todas as trocas que ainda vêm.

No próximo capítulo, a bancada ganha uma peça de outro tipo: edição de vídeo e armazenamento em nuvem. Você vai descobrir por que o DaVinci Resolve — a ferramenta mais famosa da categoria — é, na verdade, uma armadilha de licença para quem busca open source de verdade, e como Shotcut, Kdenlive, Nextcloud e Seafile ocupam os espaços que ela não cobre.

7. Referências Bibliográficas

[1] AWESOME-SELFHOSTED. *A list of Free Software network services and web applications which can be hosted on your own servers*. Disponível em: <https://github.com/awesome-selfhosted/awesome-selfhosted>. Acesso em: 19 ago. 2026.

[2] PENPOT/KALEIDOS. *penpot: The open-source design platform for Product teams that need scalable collaboration*. Disponível em: <https://github.com/penpot/penpot>. Acesso em: 19 ago. 2026.

[3] PENPOT. *Self-Host Penpot: Deploy it anywhere*. Disponível em: <https://penpot.app/self-host>. Acesso em: 19 ago. 2026.

[4] PENPOT. *Self-hosting Guide — Help center*. Disponível em: <https://help.penpot.app/technical-guide/getting-started/>. Acesso em: 19 ago. 2026.

[5] PENPOT. *Install with Docker — Help center*. Disponível em: <https://help.penpot.app/technical-guide/getting-started/docker/>. Acesso em: 19 ago. 2026.

[6] JONKER, Nolen. *I tried a free, open-source, browser-based alternative to Figma, and it blew my mind*. XDA Developers. Disponível em: <https://www.xda-developers.com/tried-free-open-source-browser-based-alternative-figma/>. Acesso em: 19 ago. 2026.

[7] XDA DEVELOPERS. *I replaced Figma with four open-source alternatives for a week, and only one actually stuck*. Disponível em: <https://www.xda-developers.com/replaced-figma-with-open-source-alternatives-only-one-stuck/>. Acesso em: 19 ago. 2026.

[8] SLASHDOT. *Compare Canva vs. Figma vs. Penpot in 2026*. Disponível em: <https://slashdot.org/software/comparison/Canva-vs-Figma-vs-penpot/>. Acesso em: 19 ago. 2026.

[9] GIMP TEAM. *GIMP — GNU Image Manipulation Program*. Disponível em: <https://www.gimp.org/>. Acesso em: 19 ago. 2026.

[10] GIMP TEAM. *GIMP Documentation*. Disponível em: <https://docs.gimp.org/>. Acesso em: 19 ago. 2026.

[11] GNOME. *gimp: GNU Image Manipulation Program (source)*. GitLab. Disponível em: <https://gitlab.gnome.org/GNOME/gimp>. Acesso em: 19 ago. 2026.

[12] CREATIVE BLOQ. *GIMP 3.0 review: 20 years on from 2.0, has GIMP kept up with the times?*. Disponível em: <https://www.creativebloq.com/photography/photo-editing-software/gimp-3-0-review-20-years-on-from-2-0-has-gimp-kept-up-with-the-times>. Acesso em: 19 ago. 2026.

[13] CREATIVE BLOQ. *How I really feel about GIMP, now the dust has settled*. Disponível em: <https://www.creativebloq.com/photography/photo-editing-software/how-i-really-feel-about-gimp-now-the-dust-has-settled>. Acesso em: 19 ago. 2026.

[14] GEEKSFORGEES. *GIMP vs Photoshop*. Disponível em: <https://www.geeksforgeeks.org/blogs/gimp-vs-photoshop/>. Acesso em: 19 ago. 2026.

[15] INKSCAPE PROJECT. *Inkscape: Draw Freely*. Disponível em: <https://inkscape.org/>. Acesso em: 19 ago. 2026.

[16] INKSCAPE PROJECT. *inkscape/inkscape (repositório de código)*. GitLab. Disponível em: <https://gitlab.com/inkscape/inkscape>. Acesso em: 19 ago. 2026.

[17] INKSCAPE PROJECT. *Installing Inkscape — Inkscape Wiki*. Disponível em: https://wiki.inkscape.org/wiki/Installing_Inkscape. Acesso em: 19 ago. 2026.

[18] VECTEEZY. *Inkscape vs. Illustrator: A Head-to-Head Comparison*. Disponível em: <https://www.vecteezy.com/blog/design-tips/inkscape-vs-illustrator>. Acesso em: 19 ago. 2026.

[19] CREATIVE BLOQ. *Tired of Adobe and the big tech giants? Here's how I created my ideal Open Source Creative Workstation*. Disponível em: <https://www.creativebloq.com/art/digital-art-software/tired-of-adobe-and-the-big-tech-giants-i-created-my-ideal-open-source-creative-workstation-p-s-its-very-cheap>. Acesso em: 19 ago. 2026.

[20] KDE. *krita* (mirror). GitHub. Disponível em: <https://github.com/KDE/krita>. Acesso em: 19 ago. 2026.

[21] KDE. *Graphics / Krita* (repositório oficial). GitLab (invent.kde.org). Disponível em: <https://invent.kde.org/graphics/krita>. Acesso em: 19 ago. 2026.

[22] KRITA FOUNDATION. *Krita — Digital Painting. Creative Freedom*. Disponível em: <https://krita.org/en/>. Acesso em: 19 ago. 2026.

[23] KRITA FOUNDATION. *Installation — Krita Manual*. Disponível em: https://docs.krita.org/en/user_manual/getting_started/installation.html. Acesso em: 19 ago. 2026.

[24] CREATIVE BLOQ. *Krita 5.2.6 review: finally, a real Photoshop alternative*. Disponível em: <https://www.creativebloq.com/reviews/krita>. Acesso em: 19 ago. 2026.

[25] AWESOME-SELFHOSTED. *A list of Free Software network services and web applications which can be hosted on your own servers*. Disponível em: <https://github.com/awesome-selfhosted/awesome-selfhosted>. Acesso em: 19 ago. 2026.

[26] SOLVOHQ. *awesome-self-host-saas-alternatives: Curated list of strictly self-hostable open-source alternatives to popular SaaS*. Disponível em: <https://github.com/SolvoHQ/awesome-self-host-saas-alternatives>. Acesso em: 19 ago. 2026.

Capítulo 6: Vídeo e Nuvem: Editando e Guardando Seus Arquivos Sem Assinatura

1. Introdução

No capítulo anterior, a oficina aprendeu a produzir peças visuais — imagens, vetores e interfaces — sem pagar licença. Agora a bancada enfrenta dois dos aluguéis mais caros da lista: o editor de vídeo que cobra mensalidade ou esconde o preço na forma de uma versão gratuita que não é sua, e a nuvem que guarda seus arquivos enquanto o cartão de crédito continuar valendo. Este capítulo trata das duas trocas — e, como nos anteriores, com a honestidade que a Oficina Digital exige: existem categorias em que a substituição é madura, e existem casos em que o rótulo “open source” é usado de forma errada.

Você vai aprender, nesta ordem: como editar vídeo de verdade com ferramentas livres (Shotcut e Kdenlive) e por que o DaVinci Resolve — gratuito, mas proprietário — não pertence à sua prateleira, e sim à loja de aluguel; como montar a sua própria nuvem com Nextcloud e Seafile, devolvendo o controle dos dados à sua bancada; e o orçamento oculto que toda assinatura de armazenamento escondia dentro da mensalidade, com a matemática real de disco, backup e banda. Ao final, você terá um pipeline de vídeo funcionando e uma nuvem própria no ar — e saberá exatamente quanto cada um deles custa para manter.

2. Explica

A primeira distinção que este capítulo precisa fazer é a mais ignorada em listas de “alternativas open source”: grátis não é o mesmo que aberto, e aberto não é o mesmo que seu. O DaVinci Resolve é o exemplo perfeito. Ele é distribuído gratuitamente pela Blackmagic Design, mas é software proprietário, em modelo freemium: não existe repositório público de código-fonte, você não pode ler, modificar nem redistribuir nada [135]. A versão gratuita é uma vitrine cuidadosamente calibrada — ela cobre edição, correção de cor, efeitos e áudio até UHD 60fps sem marca d’água, o que é generoso, mas deixa de fora justamente os recursos que movem o

profissional para o pagamento: inteligência artificial, exportação DCI 4K/6K/8K, aceleração multi-GPU e HDR [135][137][138]. É o modelo clássico da loja de aluguel: você usa a furadeira de graça, mas os acessórios que fazem o serviço profissional custam, e o motor continua sendo da loja. Tratar o Resolve como “open source” na sua oficina seria o mesmo que pendurar na prateleira uma peça que continua alugada.

As ferramentas que realmente pertencem à sua bancada são Shotcut e Kdenlive — ambas 100% open source, sob licença GPLv3, e ambas construídas sobre o mesmo motor: o MLT Framework, uma biblioteca de edição não linear que faz o trabalho pesado de timeline, codecs e filtros [130][131]. Isso importa na prática porque os dois projetos herdam os mesmos pontos fortes e as mesmas limitações do MLT. Os requisitos de hardware são honestos e modestos: Windows 10 64-bit, macOS 12+ ou Linux, de 4 a 16 GB de RAM e uma GPU com suporte a OpenGL 2.0 — qualquer computador razoável monta a bancada de edição [126]. As reviews independentes confirmam o amadurecimento: a do Shotcut, na PCWorld, classifica o editor como “impressionante” apesar de uma interface que ainda parece datada [127], e a do TechRadar reforça o mesmo veredito [128]. A instabilidade histórica, porém, é real e documentada — há issues públicas no rastreador do MLT registrando crashes ao adicionar vídeos longos à timeline [129], e o Kdenlive sofreu por anos com cenários de travamento que só as releases recentes corrigiram em massa [132][133]. A verdade incômoda: para produções longas e pesadas, ou projetos que dependem do ecossistema Adobe, a troca ainda perde — e o capítulo 5 desta parte mostrou como reconhecer esse limite antes de migrar [134].

A segunda peça de vocabulário é a nuvem self-hosted. Nextcloud é um servidor completo de sincronização e compartilhamento: você instala em uma VPS ou num servidor caseiro, e os clientes de desktop e celular sincronizam pastas como o Google Drive ou o Dropbox fariam — mas os arquivos vivem em infraestrutura sua, sem cobrança por usuário [139][140][141]. O custo oficial mínimo é baixo, mas a prática recomenda 2 a 4 GB de RAM para uso doméstico e 8 GB ou mais para times [140]. A limitação honesta: a criptografia ponta-a-ponta ainda não está totalmente implementada — os arquivos são criptografados em trânsito e em repouso no servidor, mas o servidor, sendo seu, é o limite da sua confiança [142]. Seafile ocupa o mesmo nicho com uma arquitetura diferente — núcleo em C, sincronização por blocos — e exigência ainda mais modesta: 2 núcleos de CPU e 2 GB de RAM para a edição comunitária [144][145]. Sua criptografia client-side usa AES-256-CBC por biblioteca, mas não funciona no navegador nem nos clientes de nuvem — um limite que qualquer review independente acaba apontando [146]. E há ainda o Syncthing, que não é uma nuvem: é sincronização ponto a ponto, sem servidor central, sem limite de armazenamento e sem mensalidade [148][151] — mas a própria documentação oficial insiste que ele não substitui backup, e o capítulo 5 deste capítulo vai mostrar por quê [149][150].

3. Ilustra

A imagem mental deste capítulo é a de uma oficina que recebe matéria-prima bruta — o vídeo gravado no cartão de memória — e a transforma em peça acabada que vai para a prateleira. No modelo alugado, você entrega a matéria-prima a uma loja (a nuvem ou o editor do fabricante), recebe o produto pronto, e a loja fica com a peça, o molde e os lucros do serviço recorrente. Na sua oficina, o caminho é outro: o arquivo bruto entra pela bancada de montagem (Shotcut ou Kdenlive), sai como vídeo finalizado, e termina na estante central da sua própria nuvem — com uma cópia externa de segurança que é a única coisa que sobrevive a um desastre.

Essa primeira analogia resolve a mecânica de posse. Mas, como no capítulo anterior, a segunda lente é a que revela o custo escondido: a mensalidade da nuvem comprava, silenciosamente, um pacote completo de infraestrutura — disco, replicação em múltiplos data centers, banda de upload abundante e peças de reposição instantâneas quando um drive morria. Nenhuma dessas coisas é mágica; todas têm etiqueta de preço. Ao trocar o Google Drive pelo Nextcloud, você não elimina esses custos — você os separa em três contas visíveis que passam a ser suas: a conta do disco, a conta do backup e a conta da banda. O diagrama abaixo resume esse fluxo, da matéria-prima à prateleira com redundância.

%% legenda: Da materia-prima a prateleira: o caminho do arquivo na Oficina Digital

flowchart LR

```
A[Cartao de memoria: video bruto] --> B{Bancada de montagem}
B --> C[Shotcut: editor livre]
B --> D[Kdenlive: editor livre]
C --> E[Video finalizado]
D --> E
E --> F{Guarda onde?}
F --> G[Nextcloud: estante central propria]
F --> H[Seafile: estante leve]
G --> I[Backup externo 3-2-1]
H --> I
I --> J[Prateleira segura: o desastre nao leva tudo]
```

4. Técnica

Esta seção segue o padrão do material-fonte: verificação de hardware antes da instalação, sessões de terminal com comando e saída esperada, um manifesto Docker Compose completo para a nuvem própria, e um script de backup comentado linha a linha — sem código de programação, porque o que você precisa aqui é operar a oficina, não desenvolvê-la.

Montando a Bancada de Vídeo

Antes de instalar qualquer editor, confira se a bancada tem o mínimo que o MLT Framework exige. O comando abaixo mostra o sistema operacional, a memória e o suporte a OpenGL — os três requisitos do Shotcut e do Kdenlive [126][131]:

```
$ uname -a
Linux oficina 6.8.0-45-generic #45-Ubuntu SMP x86_64 GNU/Linux
$ free -h
```

	total	used	free
Mem:	15GiB	3,2GiB	11GiB

```
$ glxinfo | grep "OpenGL version"
OpenGL version string: 4.6 (Core Profile) Mesa 24.0.5
```

Com os requisitos confirmados, a instalação no Linux é um comando — o Kdenlive é empacotado pela KDE e disponível nos repositórios oficiais da maioria das distribuições:

```
$ sudo apt install kdenlive
$ kdenlive --version
kdenlive 25.08.1
```

O Shotcut, por sua vez, distribui um instalador para cada plataforma — no Windows, o arquivo MSI oficial instala e cria o atalho da bancada [127]:

```
C:\Users\artesaio> shotcut.exe --version
Shotcut version 25.05.30
```

A Estante Central: Nextcloud Self-Hosted

A nuvem da oficina começa com um manifesto Docker Compose. A base mínima de produção recomendada pela documentação oficial do Nextcloud usa três serviços: a aplicação, o banco de dados e um volume persistente — a prateleira física onde os dados duram [141]:

```
# docker-compose.yml – Nextcloud self-hosted (referência mínima)
version: "3.9"

services:
  db:
    image: mariadb:11
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud
      MYSQL_PASSWORD: troque-esta-senha
      MYSQL_ROOT_PASSWORD: troque-esta-root
```

```

volumes:
  - db:/var/lib/mysql

app:
  image: nextcloud:stable
  restart: unless-stopped
  ports:
    - "8080:80"
  environment:
    MYSQL_HOST: db
    MYSQL_DATABASE: nextcloud
    MYSQL_USER: nextcloud
    MYSQL_PASSWORD: troque-esta-senha
  volumes:
    - nextcloud:/var/www/html
  depends_on:
    - db

volumes:
  db:
  nextcloud:

```

Subir a estante é um comando, e o primeiro login define a conta de administrador — a chave da oficina:

```

$ docker compose up -d
[+] Running 3/3
✓ Container db Started
✓ Container app Started
$ curl -sI http://localhost:8080 | head -1
HTTP/1.1 302 Found

```

A manutenção da estante também passa pelo terminal: a interface de linha de comando do próprio Nextcloud (o comando `occ`) executa as tarefas de rotina — declaração de manutenção antes de atualizar, e a conferência de erros do sistema:

```

$ docker compose exec -u www-data app php occ maintenance:mode --on
Maintenance mode enabled
$ docker compose exec -u www-data app php occ status
- installed: true
- version: 30.0.1

```

A Prateleira Externa: Backup com a Regra 3-2-1

A lição mais cara deste capítulo é também a mais simples: estante não é backup. O Syncthing sincroniza, mas se um ransomware criptografar os arquivos em todos os dispositivos, a sincronização propaga o desastre [149][150]. Backup de verdade segue a regra 3-2-1 — três cópias, em dois meios diferentes, uma delas fora do ambiente — e o `restic` executa isso de forma automatizada e verificável:

```
# backup-oficina.sh – regra 3-2-1 na pratica (exemplo didatico: requer
restic instalado)
# Copia 1 e 2: estante Nextcloud + disco local (ja existem)
# Copia 3: repositório restic em disco externo, fora do ambiente

export RESTIC_REPOSITORY=/media/disco-externo/restic-oficina
export RESTIC_PASSWORD="senha-do-cofre"

# 1. Faz o snapshot criptografado de toda a prateleira
restic backup /srv/nextcloud/data

# 2. Verifica a integridade do cofre (nada de confiar cegamente)
restic check

# 3. Política de retenção: 7 diários, 4 semanais, 6 mensais
restic forget --keep-daily 7 --keep-weekly 4 --keep-monthly 6 --prune
```

A Estante Leve: Seafile para o Autônomo

O Nextcloud é a estante completa, mas nem toda oficina precisa de calendário, contatos e edição colaborativa embutidos. O Seafile ataca exatamente esse espaço: um núcleo em C focado em sincronização de blocos, rápido e econômico, com a edição comunitária exigindo apenas 2 núcleos de CPU e 2 GB de RAM [144][145]. A instalação via Docker segue o mesmo padrão dos manifestos anteriores, com um detalhe que vale registrar: o Seafile usa uma única imagem oficial que sobe o servidor e o banco interno juntos, e a configuração inicial é feita pelo assistente web na primeira execução [144]. A criptografia client-side existe e usa AES-256-CBC, mas a própria documentação e as reviews deixam claro o limite: ela não funciona no navegador nem nos clientes de nuvem — só no desktop —, então quem vive de acesso pelo celular acaba guardando os arquivos sem essa camada de proteção [146].

A escolha entre Nextcloud e Seafile é, portanto, a escolha entre a estante completa e a prateleira enxuta: o comparativo independente da CloudBasedBackup resume o veredito com uma frase — o Nextcloud vence em colaboração e ecossistema, o Seafile vence em velocidade e simplicidade de recursos [147]. O teste da Speed-Drain reforça a mesma conclusão para quem compara as três opções self-hosted do mercado: o Syncthing é o mais leve, o Seafile o mais rápido para pastas grandes e o Nextcloud o mais completo em funcionalidades [152]. Para o

Artesão Digital que roda um VPS de 2 GB, essa diferença de peso decide a escolha antes mesmo da interface.

Vídeo Não É Só Cortar: O Fluxo de Produção Completo

Editar vídeo na bancada própria vai além de arrastar clipes na timeline. O fluxo completo de produção tem etapas que o usuário de assinatura nunca enxergou porque a loja fazia tudo: capturar (o cartão de memória), transferir (a velocidade de leitura do leitor), organizar (a árvore de pastas do projeto), editar (o corte), exportar (o codec e o formato de destino) e distribuir (o upload para a plataforma). Cada uma dessas etapas cobra um recurso diferente da sua oficina — e a seção 5 deste capítulo mostrou como o orçamento aparece em cada uma. O que vale fixar aqui é a ordem: o iniciante quer pular direto para a edição e esquece que a maior parte do tempo real de um projeto está na captura e na organização. O Kdenlive facilita o trabalho com uma árvore de projeto clara, mas a disciplina de nomear arquivos, versões e cortes é sua [131][133]. O Shotcut, por sua vez, brilha na simplicidade do fluxo de exportação, com perfis prontos para YouTube e Vimeo que escondem a matemática de bitrate e resolução [127][128] — e esconder essa matemática é exatamente o que o iniciante precisa nos primeiros projetos, antes de aprender a calcular o tamanho de arquivo por minuto.

```
$ ffprobe -v error -show_entries format=duration,bit_rate -of
default=noprint_wrappers=1 projeto-final.mp4
duration=300.500000
bit_rate=8250000
$ ls -lh projeto-final.mp4
-rw-r--r-- 1 artesao artesao 313M ago 20 07:00 projeto-final.mp4
```

O comando acima — uma linha de `ffprobe` — responde à pergunta que todo editor precisa fazer antes de subir um vídeo: quanto pesa o arquivo, quanto dura e a que bitrate foi exportado. São as três informações que decidem se o upload para a plataforma vai levar cinco minutos ou cinco horas na banda da sua conexão [152]. E é também a diferença entre bater cabeça com o “vídeo traveou na timeline” e saber, antes de editar, que o arquivo de 60 minutos em 4K bruto vai exigir proxies ou cortes em partes: a instabilidade documentada do MLT em arquivos longos não é mistério — é uma propriedade mensurável do formato e do hardware [129][133]. E é por isso que a produção de vídeo na oficina própria é, antes de tudo, um exercício de medição: medir antes de editar, medir antes de exportar e medir antes de subir. Cada medição evita uma hora de retrabalho — e nenhuma delas depende de licença.

A Rotina de Manutenção da Estante: Sincronização e Saúde do Servidor

A estante própria não entrega valor apenas no dia da montagem — entrega na rotina. Depois do Nextcloud no ar, a manutenção tem um ritmo mensal com três frentes que a documentação oficial cobre e que a assinatura escondia na mensalidade: aplicar as atualizações do servidor,

verificar a saúde dos serviços e conferir o estado da sincronização [141][143]. A primeira frente é o comando `occ` — a linha de comando administrativa do Nextcloud — que já apareceu na seção de instalação e que também executa a verificação de integridade da instalação:

```
$ docker compose exec -u www-data app php occ maintenance:mode --off
Maintenance mode disabled
$ docker compose exec -u www-data app php occ app:update --all
[OK] Atualizacoes aplicadas para: 3 apps
$ docker compose exec -u www-data app php occ status
- installed: true
- version: 30.0.1
```

A segunda frente é a saúde visível no painel do servidor: uso de disco, memória e a fila de trabalhos agendados do Nextcloud (os cron jobs que processam notificações, uploads e expirações de compartilhamento). A sessão abaixo mostra como conferir a fila e o estado do disco de uma só vez — os dois números que decidem se a estante está saudável ou se está uma bomba-relógio:

```
$ docker compose exec -u www-data app php occ background:cron
$ df -h /srv/nextcloud
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda1       50G   31G   19G   62% /
$ docker compose logs --since 24h app | grep -c "ERROR"
0
```

O hábito que fechar a rotina é a conferência do log: zero erros nas últimas 24 horas é o sinal verde, e qualquer ERROR repetido é um alerta que a assinatura teria resolvido sem você ver — e que agora é problema da sua bancada [143]. É esse tripé — atualização, saúde e log — que transforma a estante de “projeto do fim de semana” em “infraestrutura de verdade”, e é exatamente o mesmo tripé que a seção 5 do capítulo 8 vai aplicar à oficina inteira, em escala.

5. Aplica

A jornada prática deste capítulo é a primeira semana do Artesão Digital migrando vídeo e arquivos. O cenário: você terminou de editar um vídeo curto para o canal da sua área — algo de cinco minutos, com cortes, texto sobreposto e áudio limpo — e gravou os arquivos brutos de um projeto inteiro na pasta sincronizada da nuvem.

Erro comum: você confia na sincronização como se ela fosse backup. Edita o vídeo direto da pasta sincronizada do Nextcloud em dois dispositivos, apaga os arquivos brutos “para liberar espaço”, e quando o drive da VPS morre — ou um erro de sincronização propaga a exclusão para todos os lados — descobre que o projeto inteiro existia em uma única cópia. O

diagnóstico é sempre o mesmo: sincronizar replica; backup preserva. São funções diferentes, e este capítulo mostrou que até a documentação do Syncthing avisa que ele não é ferramenta de backup [149][150].

Prática correta: a pasta sincronizada recebe apenas o material de trabalho corrente. O projeto bruto gravado no cartão vai direto para a estante — e o `backup-oficina.sh` roda na madrugada, criando o snapshot criptografado no disco externo. A rotina semanal inclui um `restic check` real, porque backup que nunca é verificado é apenas uma esperança. Quando o desastre finalmente acontece — e algum dia acontece — a restauração é um comando: `restic restore latest --target /restauracao`. Você perde horas, não anos.

A decisão que fecha a semana: com o pipeline montado, chega a hora de calcular a conta real — e comparar o que a assinatura do Drive/Dropbox escondia: o disco da estante, o disco externo do cofre e a banda de upload da sua conexão, que agora sobe arquivos de projeto inteiros, não apenas fotos comprimidas no celular. A revisão da seção 2 já deu os números de referência: o Nextcloud prático pede 2 a 4 GB de RAM doméstico [140], o Seafile roda com 2 núcleos e 2 GB [145], e a sua conexão residencial de upload assimétrico decide se a estante fica em casa ou numa VPS. Quando a soma dessas três contas ainda ficar abaixo do que a mensalidade cobrava, a troca está paga — e a peça é sua.

6. Conclusão

Este capítulo fechou duas das trocas mais emocionais da oficina. Na edição de vídeo, você aprendeu a diferença entre o gratuito que é seu (Shotcut e Kdenlive, GPLv3, com limites documentados e honestos) e o gratuito que é vitrine de outra loja (DaVinci Resolve, proprietário, com tetos calibrados para empurrar a versão paga) [135][137]. Na nuvem, você montou a estante central com Nextcloud ou Seafile, e aprendeu o orçamento oculto: disco, backup e banda que a mensalidade comprava silenciosamente agora são contas suas — mas são contas que você enxerga, mede e decide [141][145]. E descobriu a regra que vale para a oficina inteira: sincronizar replica, backup preserva, e a única cópia que sobrevive a um desastre é a que está fora do ambiente [149][150].

O Artesão Digital que sai deste capítulo não é o que acha que “open source é grátis”. É o que sabe orçar a manutenção, distingue posse real de vitrine gratuita, e calcula — ferramenta por ferramenta — quando a troca vale o esforço. Essa régua é exatamente a que o próximo capítulo vai usar para medir as peças que organizam e analisam o trabalho da oficina: BI e gestão de projetos sem licença por usuário.

7. Referências Bibliográficas

- [126] SHOTCUT.ORG. *Frequently Asked Questions*. Disponível em: <https://www.shotcut.org/FAQ/>. Acesso em: 19 ago. 2026. [127] AXON, Samuel. *Shotcut review: This open-source video editor is impressive*. PCWorld. Disponível em: <https://www.pcworld.com/article/407690/shotcut-review.html>. Acesso em: 19 ago. 2026. [128] TECHRADAR. *Shotcut review*. Disponível em: <https://www.techradar.com/reviews/shotcut>. Acesso em: 19 ago. 2026. [129] MLTFRAMEWORK. *Issue #1786: Shotcut crashes when adding or keeping a long video file on timeline*. Disponível em: <https://github.com/mltframework/shotcut/issues/1786>. Acesso em: 19 ago. 2026. [130] KDE. *kdenlive: Free and open source video editor, based on MLT Framework and KDE Frameworks*. Disponível em: <https://github.com/KDE/kdenlive>. Acesso em: 19 ago. 2026. [131] KDE. *Installation — Kdenlive 26.04 Manual*. Disponível em: https://docs.kdenlive.org/en/getting_started/installation.html. Acesso em: 19 ago. 2026. [132] KDENLIVE.ORG. *Kdenlive 25.08.1 released*. Disponível em: <https://kdenlive.org/news/releases/25.08.1/>. Acesso em: 19 ago. 2026. [133] KDE. *Bug 462777 — kdenlive crashes after a few clicks on the clips in the timeline*. Disponível em: https://bugs.kde.org/show_bug.cgi?id=462777. Acesso em: 19 ago. 2026. [134] FIXTHEPHOTO. *Kdenlive vs Adobe Premiere Pro: Which Is NOT Intuitive?*. Disponível em: <https://fixthephoto.com/kdenlive-vs-adobe-premiere-pro.html>. Acesso em: 19 ago. 2026. [135] BLACKMAGIC DESIGN. *DaVinci Resolve*. Disponível em: <https://www.blackmagicdesign.com/products/davinciresolve>. Acesso em: 19 ago. 2026. [136] TECHRADAR. *DaVinci Resolve 21 (2026) review: Our top free video editing app gets big improvements*. Disponível em: <https://www.techradar.com/pro/software-services/davinci-resolve-21-2026-video-editing-software-review>. Acesso em: 19 ago. 2026. [137] TOOLFARM. *In Depth: DaVinci Resolve Studio vs Free (Updated for 21)*. Disponível em: <https://www.toolfarm.com/tutorial/in-depth-davinci-resolve-studio-vs-the-free-version/>. Acesso em: 19 ago. 2026. [138] DIGITAL CAMERA WORLD. *DaVinci Resolve 18 free vs Resolve Studio 18: which is the best option for you?*. Disponível em: <https://www.digitalcameraworld.com/buying-guides/davinci-resolve-18-free-vs-resolve-studio-18-which-is-the-best-option-for-you>. Acesso em: 19 ago. 2026. [139] NEXTCLOUD. *server: Nextcloud server, a safe home for all your data*. Disponível em: <https://github.com/nextcloud/server>. Acesso em: 19 ago. 2026. [140] NEXTCLOUD. *System requirements — Nextcloud Administration Manual*. Disponível em: https://docs.nextcloud.com/server/stable/admin_manual/installation/system_requirements.html. Acesso em: 19 ago. 2026. [141] NEXTCLOUD. *Installation and server configuration — Nextcloud Administration Manual*. Disponível em: https://docs.nextcloud.com/server/stable/admin_manual/installation/index.html. Acesso em: 19 ago. 2026. [142] CYBERINSIDER. *Nextcloud Review (2026 Test Results)*. Disponível em: <https://cyberinsider.com/cloud-storage/reviews/nextcloud/>. Acesso em: 19 ago. 2026. [143] CONTABO. *Nextcloud vs. Competitors: A Deep Dive into Self-Hosted Alternatives*. Disponível em: <https://contabo.com/blog/nextcloud-vs-competitors/>. Acesso em: 19 ago. 2026. [144] HAIWEN. *seafiler-server: Seafiler Server Core*. Disponível em: <https://github.com/haiwen/seafiler-server>. Acesso em: 19 ago. 2026. [145] SEAFILER. *System require-*

ments — *Seafile Admin Manual*. Disponível em: https://manual.seafile.com/13.0/setup/system_requirements/. Acesso em: 19 ago. 2026. [146] PROPRIVACY. *Seafile Review*. Disponível em: <https://proprivacy.com/cloud/review/seafile>. Acesso em: 19 ago. 2026. [147] CLOUDBASED-BACKUP. *Nextcloud vs Seafile: Which Cloud Storage Is Better*. Disponível em: <https://cloudbasedbackup.com/en/blog/nextcloud-vs-seafile-which-cloud-storage-is-better>. Acesso em: 19 ago. 2026. [148] SYNCTHING. *syncthing: Open Source Continuous File Synchronization*. Disponível em: <https://github.com/syncthing/syncthing>. Acesso em: 19 ago. 2026. [149] SYNCTHING. *Getting Started — Syncthing documentation*. Disponível em: <https://docs.syncthing.net/intro/getting-started.html>. Acesso em: 19 ago. 2026. [150] SYNCTHING. *Security Principles — Syncthing documentation*. Disponível em: <https://docs.syncthing.net/users/security.html>. Acesso em: 19 ago. 2026. [151] XDA DEVELOPERS. *I replaced Dropbox with Syncthing, and learned these 5 things*. Disponível em: <https://www.xda-developers.com/replaced-dropbox-with-syncthing-learned-these-things/>. Acesso em: 19 ago. 2026. [152] SPEED-DRAIN. *Best Self-Hosted Cloud Storage 2026: Nextcloud vs Seafile vs Syncthing Comparison*. Disponível em: <https://speed-drain.com/blog/best-self-hosted-cloud-storage-2026-nextcloud-seafile-syncthing/>. Acesso em: 19 ago. 2026.

Capítulo 7: Análise de Dados e Gestão de Projetos: BI e Kanban Sem Licença por Usuário

1. Introdução

No capítulo anterior, a prateleira da oficina ganhou as estantes que guardam os arquivos da jornada — vídeo editado, nuvem própria, backup verificado. Agora a Oficina Digital precisa de duas peças que nenhuma bancada séria dispensa: a que transforma dados em decisão — o BI — e a que organiza o trabalho de quem está montando tudo — a gestão de projetos. E aqui o aluguel tem um formato especialmente traiçoeiro: a mensalidade é cobrada por assento. Tableau, Power BI, Asana, Monday.com e Trello não cobram da oficina — cobram de cada pessoa que senta na bancada. Dez pessoas no time é dez mensalidades, todos os meses, para sempre.

Este capítulo mostra o outro lado dessa conta. Você vai aprender, nesta ordem: como subir um painel de vendas próprio com Metabase ou Superset em uma tarde, e por que o Redash merece um alerta de longevidade antes de virar peça fixa da sua prateleira; como escolher entre OpenProject, Taiga e Focalboard a bancada de gestão certa para o tamanho e o ritmo do seu time; e onde essa comparação para — o custo real de operação, e os casos em que a mensalidade ainda compra o que a bancada não entrega. Ao final, você terá um painel de BI no ar e um critério honesto para decidir quando o “grátis por usuário” vale a manutenção.

2. Explica

Comece pelo que BI significa na prática, sem o jargão de vendedor: BI é o processo de conectar fontes de dados, escrever consultas, publicar dashboards e agendar o envio desses painéis para quem decide. Não é mágica, é uma rotina — e é exatamente essa rotina que as ferramentas open source já cobrem com maturidade. O Metabase é o caso de entrada mais suave: um repositório com 48,8 mil estrelas no GitHub, requisitos de produção de 2 vCPU, 2 a 4 GB de RAM e 10 GB ou mais de disco, e uma instalação Docker que sobe em minutos [153][154][155]. O Apache Superset tem perfil diferente — é o projeto da Apache Software Foundation, com instalação via

Docker Compose oficial e uma pegada mais voltada a analistas que escrevem SQL e exploram dados em profundidade [160][161][162]. As reviews comparativas independentes convergem num veredito: para a fatia de dashboards operacionais e relatórios de vendas, os dois já são páreo técnico para Tableau e Power BI em cenários de pequenas e médias empresas [156][157][163][164]. Há limites documentados de escala — o Superset impõe um teto de cerca de 800 KB por resultado de consulta [160], e o Metabase trava a interface em datasets gigantes, com issues públicas registrando exatamente o comportamento [158][159].

A segunda peça de vocabulário é a gestão de projetos, e aqui o modelo de assinatura por usuário fica ainda mais visível: o custo do Asana ou do Monday.com cresce a cada pessoa adicionada ao quadro. As alternativas da oficina cobrem o espectro com desenhos diferentes. O OpenProject é o mais completo — Gantt, portfólio, visão de PMO — e roda na Community Edition com requisitos honestos de 4 GB de RAM, 2 CPUs e 20 GB de disco para 10 a 20 usuários, exigindo PostgreSQL 16 ou superior [171][172][173]. O Taiga é o campeão do Scrum e do Kanban ágil, construído pela comunidade Kaleidos, com instalação Docker disponível e uma migração conhecida por perder comentários e anexos em planilhas CSV [176][177][178][179]. E o Focalboard, da Mattermost, é o kanban leve — a alternativa direta ao Trello para quem quer o quadro simples sem o peso de um sistema de portfólio [180]. A régua de escolha, como os comparativos mostram, é o formato do time: empresa consolidada com múltiplos projetos pendura o OpenProject na bancada; squad ágil enxuto prefere o Taiga; o autônomo que só quer o quadro na parede monta o Focalboard [175][180][185].

A terceira peça — e a mais honesta deste capítulo — é o risco de longevidade. Projeto open source não tem contrato de suporte: ele vive enquanto houver mantenedores. O Redash é o estudo de caso que todo Artesão Digital deveria conhecer antes de escolher BI: adquirido pela Databricks em 2020, teve a hospedagem paga descontinuada em 2021, com o fim de vida documentado oficialmente [168], e foi reiniciado em 2023 como esforço comunitário com cerca de sete mantenedores voluntários e ritmo lento de releases [165][169][170]. Nenhuma dessas informações é especulação — está no changelog, no FAQ de EOL e nas reviews de mercado. A lição não é “não use Redash”: é que a garantia da peça na sua prateleira precisa ser auditada na compra, e o custo da troca futura entra na conta de hoje.

3. Ilustra

A imagem mental deste capítulo é a parede de status da oficina — aquele quadro que mostra, em um relance, como anda cada bancada montada nos capítulos anteriores. Na loja de aluguel, essa parede é alugada peça por peça por assento: cada pessoa que olha o quadro paga uma mensalidade, e o quadro em si nunca sai da vitrine. Na sua oficina, a parede é sua: os dados vêm das fontes que você escolheu, o painel é montado com peças que você pode abrir e ajustar, e o custo não cresce quando o time cresce — cresce apenas quando a operação cresce.

A segunda lente, como nos capítulos anteriores, revela o custo escondido: o BI pago não vendia apenas o quadro — vendia a manutenção invisível. Quem atualiza o servidor, faz o backup do painel, renova o certificado TLS e responde quando a consulta quebra às 23h? Na assinatura, a resposta é “a loja”. Na oficina, a resposta é “você”. O diagrama abaixo mostra o caminho da decisão: três caixas de peça chegam da loja de aluguel, duas entram na prateleira de BI próprio e uma carrega a etiqueta de garantia vencida — não porque quebrou, mas porque ninguém assumiu o contrato de manutenção dela.

```
%% legenda: A parede de status da Oficina Digital: BI proprio, longevidade
auditada
flowchart LR
    A[Loja de aluguel: Tableau, Power BI, Asana, Monday] --> B{Oficina
precisa de painel?}
    B -->|Sim| C[Metabase: painel em uma tarde]
    B -->|Sim| D[Superset: exploracao profunda]
    B -->|Sim| E[Redash: etiqueta de garantia vencida]
    C --> F[Prateleira de BI proprio: sem licenca por assento]
    D --> F
    E --> G[Audite a longevidade antes de pendurar]
    G --> F
    F --> H[Decisao com dados, custo de operacao na conta]
```

4. Técnica

Esta seção segue o padrão do material-fonte: manifestos Docker Compose reais, sessões de terminal com comando e saída esperada, uma consulta SQL de exemplo e tabelas de decisão — sem código de programação, porque o que você precisa aqui é operar as peças, não desenvolvê-las.

Subindo o Metabase

O Metabase é a porta de entrada mais rápida para o BI próprio. A instalação oficial via Docker é um manifesto mínimo com um único serviço e um volume persistente — o banco de aplicação interno guarda as perguntas, os dashboards e as configurações [155]:

```
# docker-compose.yml – Metabase (referência mínima de produção)
version: "3.9"

services:
  metabase:
    image: metabase/metabase:latest
```

```

restart: unless-stopped
ports:
  - "3000:3000"
environment:
  MB_DB_TYPE: postgres
  MB_DB_DBNAME: metabase
  MB_DB_PORT: 5432
  MB_DB_USER: metabase
  MB_DB_PASS: troque-esta-senha
  MB_DB_HOST: db
volumes:
  - metabase-data:/metabase.db
depends_on:
  - db

db:
  image: postgres:15-alpine
  restart: unless-stopped
  environment:
    POSTGRES_DB: metabase
    POSTGRES_USER: metabase
    POSTGRES_PASSWORD: troque-esta-senha
  volumes:
    - db-data:/var/lib/postgresql/data

volumes:
  metabase-data:
  db-data:

```

Subir o painel é um comando, e o primeiro acesso configura o administrador da oficina:

```

$ docker compose up -d
[+] Running 2/2
 ✓ Container db          Started
 ✓ Container metabase    Started
$ curl -sI http://localhost:3000 | head -1
HTTP/1.1 200 OK

```

Com o painel no ar, a consulta que sustenta o primeiro dashboard do Artesão Digital pode começar a ser escrita — um relatório simples de vendas por período, pronto para virar gráfico na interface do Metabase:

```

-- relatorio_vendas.sql – base do primeiro dashboard
SELECT
  date_trunc('month', criado_em) AS mes,
  count(*)                      AS pedidos,
  sum(valor_total)              AS receita

```

```
FROM vendas
WHERE criado_em >= now() - interval '12 months'
GROUP BY 1
ORDER BY 1;
```

A Bancada de Gestão: OpenProject

Para o time que precisa de mais do que um quadro — cronograma, portfólio e papéis de PMO — o OpenProject é a peça da prateleira. A instalação Docker all-in-one oficial é um manifesto com dois serviços, aplicação e banco PostgreSQL 16 [173]:

```
# docker-compose.yml – OpenProject (Community Edition)
version: "3.9"

services:
  openproject:
    image: openproject/openproject:16
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      OPENPROJECT_HOST: projetos.suaoficina.com
      OPENPROJECT_SECRET_KEY_BASE: troque-esta-chave
    volumes:
      - op-pgdata:/var/lib/postgresql/14/main
      - op-assets:/var/db/openproject
    command: docker/production/install

volumes:
  op-pgdata:
  op-assets:
```

A subida e a checagem seguem o mesmo ritmo da estante do capítulo anterior:

```
$ docker compose up -d
[+] Running 1/1
✓ Container openproject Started
$ sleep 20 && curl -sI http://localhost:8080 | head -1
HTTP/1.1 302 Found
```

O Caminho do Superset: Quando o Analytics Precisa de Profundidade

O Metabase resolve a fatia operacional do BI com velocidade — conectar, consultar, publicar. O Apache Superset entra quando a análise pede profundidade: exploração de dados com SQL

avançado, visualizações mais sofisticadas e o ecossistema da Apache Foundation por trás [160]. A instalação oficial via Docker Compose traz um conjunto maior de serviços — o banco de metadados, o Redis de cache, o worker assíncrono e o próprio servidor — porque o Superset foi desenhado para cenários onde o painel serve dezenas de usuários e consultas pesadas [161] [162]. O preço dessa profundidade aparece em dois pontos que os comparativos medem com números: o teto de cerca de 800 KB por resultado de consulta, que corta queries indisciplinadas na origem [160], e a curva de configuração, que exige conhecer o modelo de dados antes de montar o primeiro dashboard [163][164].

```
$ docker compose -f docker-compose.yml up -d
[+] Running 4/4
✓ Container superset_db      Started
✓ Container superset_cache   Started
✓ Container superset_worker  Started
✓ Container superset_app     Started
$ docker compose exec superset_app superset fab create-admin \
  --username artesao --firstname Artesao --lastname Digital \
  --email artesao@oficina.com --password troque-esta-senha
$ docker compose exec superset_app superset init
```

A sessão acima mostra o ritual completo de primeira execução: subir os quatro serviços, criar o administrador com um comando e inicializar as tabelas internas de metadados com outro [161]. Depois disso, o Superset fica parecido com o Metabase no uso diário — conectar o banco de vendas, escrever a consulta e arrastar o gráfico para o painel — mas com o motor de exploração mais profundo por baixo [163]. A regra de bolso: Metabase para o painel rápido de decisão; Superset para a análise que precisa de fôlego; e nenhum dos dois quando o requisito é um relatório executivo pronto de fornecedor — esse é o assunto da tabela de decisão da seção 5 [164].

Tabela de Decisão: Qual Bancada de Gestão Pendurar

A escolha entre as três peças de gestão depende do formato do time, não do tamanho do orçamento. Os critérios abaixo consolidam o que a seção 2 levantou [171][175][180][185]:

Critério	OpenProject	Taiga	Focalboard
Melhor para	Múltiplos projetos, Gantt, PMO	Squad ágil, Scrum/Kanban	Autônomo, quadro leve
Requisitos	4 GB RAM / 2 CPU / 20 GB disco	Docker, leve	Muito leve (pode rodar em 1 GB)
Banco	PostgreSQL 16+ obrigatório	PostgreSQL	SQLite
Curva de aprendizado	Média	Baixa	Mínima
Quando evitar	Time de 2 pessoas, um quadro só	Portfólio pesado multi-projeto	Gestão de portfólio ou Gantt

A regra de bolso: o custo por assento some da conta, mas não some da planilha — ele vira custo de operação. A tabela da seção 3 do próximo capítulo fecha essa conta com os números da VPS.

O Quadro Pessoal: Focalboard para o Artesão Solo

O time inteiro nem sempre precisa do OpenProject ou do Taiga — e o Artesão Digital que trabalha sozinho ou com poucos projetos pendura um quadro mais leve na bancada. O Focalboard, criado pela mesma comunidade do Mattermost e hoje mantido como projeto aberto, entrega exatamente esse equilíbrio: quadros Kanban, listas e calendário com uma instalação que roda em minutos num container único, sem banco separado [180]. A simplicidade tem um preço que a própria comunidade documenta: o Focalboard é um quadro pessoal e de time pequeno, não um substituto do OpenProject — faltam cronogramas Gantt, dependências entre tarefas e relatórios de portfólio [171][180]. Por isso a tabela de decisão da seção 4 colocou cada peça no seu perímetro: o Wekan e o Planka brigam no mesmo segmento do quadro leve [185], e a escolha entre eles é mais estética do que técnica — todos entregam o Kanban sem assinatura.

A régua que fecha a seção é a mesma aplicada desde o capítulo 1: a peça certa é a que cabe no formato do trabalho, não a que tem mais estrelas no GitHub [180][185]. Time solo com um quadro: Focalboard. Time de produto com sprints: Taiga. Portfólio multi-projeto com cronograma: OpenProject. E, em qualquer um dos três, o dado que decide é o tamanho da operação — o mesmo dado que a conta final do próximo capítulo soma na planilha.

O Ciclo de Vida de um Relatório: Do Banco ao E-mail Agendado

A diferença entre usar BI de verdade e apenas instalar um painel está no ciclo de vida do relatório — e é o ciclo que a assinatura automatizava silenciosamente. O fluxo completo tem quatro

pontas: a fonte de dados (o banco de vendas), a consulta (a pergunta de negócio), o dashboard (a visualização) e o agendamento (o envio periódico do relatório para quem decide). O Metabase e o Superset cobrem as quatro pontas com maturidade — e o que separa um do outro é onde cada um prefere trabalhar [154][162]. O Metabase agrega valor na interface: qualquer pessoa do time monta dashboards sem escrever SQL, conectando o banco e clicando nas métricas [154][155]. O Superset assume que quem usa entende de dados — e devolve em troca um motor de exploração mais profundo [160][162].

A sessão abaixo mostra a ponta do agendamento no Metabase — o pulso que a assinatura fazia por você e que agora roda na sua infraestrutura:

```
$ curl -s -X POST http://localhost:3000/api/pulse \
-H "X-Metabase-Session: SESS_ID" \
-H "Content-Type: application/json" \
-d '{"name":"Relatorio semanal de vendas","channel_specs":{"email":
{"recipients":["gestor@oficina.com"]}},"cards":
[{"id":1,"include_csv":true},"schedule_type":"weekly","schedule_day":"mon","schedule_h
{"id":42,"name":"Relatorio semanal de vendas","schedule_type":"weekly"}'
```

O comando cria um envio semanal — toda segunda-feira às 8h, o painel de vendas chega na caixa de entrada do gestor com o CSV anexado, sem nenhum clique humano [155]. Na assinatura, esse ritual era um botão no plano pago; na oficina, é uma chamada de API que qualquer cron dispara. A regra que fecha o ciclo: a automatização do BI não é o diferencial do open source — é o pagamento da operação. Quem monta o relatório, o agendamento e o alerta assume o plantão de manter o pipeline de pé, e é isso que a conta final da seção 5 inclui na planilha.

5. Aplica

A jornada prática deste capítulo é a primeira semana do Artesão Digital medindo e organizando a própria oficina. O cenário: você tem vendas acontecendo — o produto digital que o livro deu condições de criar — e um time pequeno de duas ou três pessoas ajudando a montar as próximas peças. Os dados estão espalhados entre uma planilha, o banco do site e o e-mail de confirmação de venda.

Erro comum: você escolhe o BI pelo nome — “Tableau é o que as empresas usam” — ou pelo preço zero, sem auditar a longevidade. O primeiro erro paga mensalidade por assento para sempre; o segundo pendura na parede uma peça cuja manutenção morreu. É exatamente o que acontece com o Redash: quem montou a stack em 2019, antes da aquisição pela Databricks, acordou em 2021 com o serviço gerenciado descontinuado — o fim de vida está documentado no FAQ oficial [168] — e precisou migrar num prazo que não escolheu [165][169]. O diagnóstico: escolha de ferramenta sem checklist de garantia é compra no escuro.

Prática correta: a escolha começa pelo formato do time e pela necessidade real de relatório. Time de poucas pessoas com vendas num único banco sobe o Metabase na primeira tarde — o manifesto da seção 4 tem o painel no ar antes do café esfriar [153][154]. A régua de escala é conferida antes da decisão, não depois: o Superset corta resultados em ~800 KB por consulta, e datasets gigantes travam a interface do Metabase — limites documentados em issues públicas, lembrando que garantia de open source é a transparência dos rastreadores, não um SLA [158][159][160]. E a gestão do time entra na mesma planilha: OpenProject para o portfólio de projetos, Taiga para o sprint, Focalboard para o quadro pessoal — sem nenhuma mensalidade por assento [171][177][180].

A decisão que fecha a semana: com o painel no ar e o quadro organizado, a conta final compara o custo total de operação — a VPS do Metabase, o backup do PostgreSQL, o certificado TLS e as horas de manutenção — contra o que as assinaturas de BI e gestão cobrariam por assento. Quando a soma da operação fica abaixo, a peça é sua e a garantia é a sua auditoria. E quando a soma passa do aceitável, o próximo capítulo mostra o balanço final: a conta honesta de quando o aluguel ainda é a peça mais barata da oficina.

6. Conclusão

Este capítulo deu à oficina a parede de status que faltava. Você aprendeu que BI não é mistério — é conectar, consultar, publicar e agendar — e que o Metabase e o Superset já cobrem essa rotina com maturidade real, com limites de escala documentados em issues públicas, não escondidos atrás de SLA [153][158][160]. Aprendeu que gestão de projetos tem três desenhos diferentes — o portfólio do OpenProject, o sprint do Taiga, o quadro leve do Focalboard — e que a escolha certa nasce do formato do time, não do orçamento [171][177][180]. E aprendeu a lição mais valiosa do capítulo, encarnada no Redash: peça open source não vem com contrato de manutenção, e a sua garantia é a auditoria de longevidade que você faz na compra — repositório ativo, releases recentes, manutenção sustentada [165][168][169].

O Artesão Digital que sai deste capítulo sabe medir a própria oficina e organizar a própria bancada sem pagar por assento. Falta uma peça para a jornada fechar o ciclo: a bancada que hospeda todas as outras, e o balanço final que decide quando a soma de VPS, manutenção e risco ainda é mais barata que a assinatura — assunto do próximo capítulo.

7. Referências Bibliográficas

[153] METABASE. *metabase/metabase*. Disponível em: <https://github.com/metabase/metabase>. Acesso em: 19 ago. 2026. [154] METABASE. *Installing Metabase*. Disponí-

vel em: <https://www.metabase.com/docs/latest/installation-and-operation/installing-metabase>. Acesso em: 19 ago. 2026. [155] METABASE. *Running Metabase on Docker*. Disponível em: <https://www.metabase.com/docs/latest/installation-and-operation/running-metabase-on-docker>. Acesso em: 19 ago. 2026. [156] IKEMO. *Power BI vs Tableau vs Looker Studio vs Metabase: Which BI Platform Should You Choose?*. Disponível em: <https://ikemo.io/blog/power-bi-vs-tableau-vs-looker-vs-metabase>. Acesso em: 19 ago. 2026. [157] EDANA. *Business Intelligence: Comparison of Power BI, Tableau, Superset, Metabase*. Disponível em: <https://edana.ch/en/2025/04/20/business-intelligence-comparison-of-power-bi-tableau-superset-metabase/>. Acesso em: 19 ago. 2026. [158] METABASE (fórum oficial). *Metabase failing to fetch large dataset*. Disponível em: <https://discourse.metabase.com/t/metabase-failing-to-fetch-large-dataset/5789>. Acesso em: 19 ago. 2026. [159] METABASE. *Issue #21985 — Large databases can be very slow to view in Data Model*. Disponível em: <https://github.com/metabase/metabase/issues/21985>. Acesso em: 19 ago. 2026. [160] APACHE SOFTWARE FOUNDATION. *apache/superset*. Disponível em: <https://github.com/apache/superset>. Acesso em: 19 ago. 2026. [161] APACHE SOFTWARE FOUNDATION. *Using Docker Compose — Apache Superset*. Disponível em: <https://superset.apache.org/admin-docs/installation/docker-compose/>. Acesso em: 19 ago. 2026. [162] APACHE SOFTWARE FOUNDATION. *Installation Methods — Apache Superset*. Disponível em: <https://superset.apache.org/admin-docs/installation/installation-methods/>. Acesso em: 19 ago. 2026. [163] PRESET.IO. *Apache Superset vs Tableau: A Practical Comparison*. Disponível em: <https://preset.io/blog/apache-superset-vs-tableau/>. Acesso em: 19 ago. 2026. [164] PEERSPOT. *Compare Apache Superset vs Tableau Enterprise*. Disponível em: https://www.peerspot.com/products/comparisons/apache-superset_vs_tableau. Acesso em: 19 ago. 2026. [165] GETREDASH. *getredash/redash*. Disponível em: <https://github.com/getredash/redash>. Acesso em: 19 ago. 2026. [166] REDASH. *Setting up a Redash Instance*. Disponível em: <https://redash.io/help/open-source/setup/>. Acesso em: 19 ago. 2026. [167] GETREDASH. *getredash/setup — Setup scripts for Redash Cloud Images*. Disponível em: <https://github.com/getredash/setup>. Acesso em: 19 ago. 2026. [168] REDASH. *Hosted Redash End of Life*. Disponível em: <https://redash.io/help/faq/eol/>. Acesso em: 19 ago. 2026. [169] SAASRAT. *Redash Review 2026: Pricing, Features, and Open-Source SQL BI Buyer Guide*. Disponível em: <https://saasrat.com/products/redash>. Acesso em: 19 ago. 2026. [170] GETREDASH. *CHANGELOG.md — redash*. Disponível em: <https://github.com/getredash/redash/blob/master/CHANGELOG.md>. Acesso em: 19 ago. 2026. [171] OPENPROJECT (OPF). *opf/openproject*. Disponível em: <https://github.com/opf/openproject>. Acesso em: 19 ago. 2026. [172] OPENPROJECT. *System requirements*. Disponível em: <https://www.openproject.org/docs/installation-and-operations/system-requirements/>. Acesso em: 19 ago. 2026. [173] OPENPROJECT. *OpenProject on Docker all-in-one container*. Disponível em: <https://www.openproject.org/docs/installation-and-operations/installation/docker/>. Acesso em: 19 ago. 2026. [174] THE NEW STACK. *Install OpenProject with Linux and Docker*. Disponível em: <https://thenewstack.io/install-openproject-with-linux-and-docker/>. Acesso em: 19 ago. 2026. [175] STACKSHARE. *OpenProject vs monday.com*. Disponível em: <https://stackshare.io/stackups/monday-vs-openproject>. Acesso em: 19 ago. 2026. [176] TAIGA (KALEIDOS VEN-

TURES). *taigaio/taiga-back*. Disponível em: <https://github.com/taigaio/taiga-back>. Acesso em: 19 ago. 2026. [177] TAIGA. *taigaio/taiga-docker*. Disponível em: <https://github.com/taigaio/taiga-docker>. Acesso em: 19 ago. 2026. [178] TAIGA COMMUNITY. *Setting up Taiga (Self-Hosted) From Scratch*. Disponível em: <https://community.taiga.io/t/setting-up-taiga-self-hosted-from-scratch/893>. Acesso em: 19 ago. 2026. [179] WIKIPEDIA. *Taiga (project management)*. Disponível em: [https://en.wikipedia.org/wiki/Taiga_\(project_management\)](https://en.wikipedia.org/wiki/Taiga_(project_management)). Acesso em: 19 ago. 2026. [180] HOWTOGEEK. *3 open-source Trello alternatives you can self-host (and keep your data)*. Disponível em: <https://www.howtogeek.com/3-open-source-trello-alternatives-you-can-self-host-and-keep-your-data/>. Acesso em: 19 ago. 2026. [181] WEKAN (WEKAN TEAM). *wekan/wekan*. Disponível em: <https://github.com/wekan/wekan>. Acesso em: 19 ago. 2026. [182] WEKAN. *Install with Docker Compose | wekan-doc*. Disponível em: <https://wekan.github.io/wekan-doc/installation/docker-compose.html>. Acesso em: 19 ago. 2026. [183] WEKAN. *Install Wekan Docker in production — wekan/wekan Wiki*. Disponível em: <https://github.com/wekan/wekan/wiki/Install-Wekan-Docker-in-production>. Acesso em: 19 ago. 2026. [184] MEETRIX. *Wekan vs Trello: Self-Hosted Kanban vs SaaS*. Disponível em: <https://meetrix.io/blogs/wekan-vs-trello/>. Acesso em: 19 ago. 2026. [185] HOMELABCOMPASS. *Self-Hosted Alternative to Trello (2026): Planka, Wekan, Vikunja*. Disponível em: <https://homelabcompass.com/alternatives/self-hosted-alternative-to-trello>. Acesso em: 19 ago. 2026.

Capítulo 8: Comunicação em Equipe e a Infraestrutura Que Sustenta Tudo

1. Introdução

Sete capítulos. Sete categorias de aluguel transformadas em peças próprias na sua bancada: notas, e-mail, CRM, automação, design, vídeo e nuvem, BI e gestão. Cada troca seguiu a mesma régua da Oficina Digital — aluguel vira posse quando a soma de VPS, manutenção e risco fica abaixo da mensalidade. Mas falta a peça que sustenta todas as outras: a comunicação do time e a infraestrutura que hospeda cada serviço montado até aqui. Sem ela, a oficina é um monte de peças soltas na prateleira; com ela, cada ferramenta dos capítulos anteriores ganha um endereço, um backup e um dono.

Este capítulo encerra a jornada e fecha o ciclo. Você vai aprender, nesta ordem: como substituir o Slack por peças reais da sua prateleira — Mattermost, Rocket.Chat e Zulip — e onde a equipe vai sentir a diferença; como montar a bancada que hospeda tudo — Docker, Coolify, CapRover e Dokku, as camadas estilo Heroku da sua própria oficina; e o balanço final — a conta honesta de quando a soma de VPS, tempo de manutenção e risco de segurança ainda sai mais barata que a assinatura, e o checklist de decisão reutilizável para qualquer categoria futura. Ao final, você não terá apenas uma oficina montada: terá a régua para continuar montando.

2. Explica

Comece pela sala de equipe — a peça que todo mundo acha que só a loja de aluguel tem. O Slack domina o mercado de tal forma que a maioria dos times nunca considerou a alternativa: e a alternativa existe, é madura, e em alguns pontos específicos passa na frente. Três projetos dividem a prateleira da oficina com desenhos diferentes. O Mattermost é o mais próximo do Slack em experiência — Go e React num binário único, forte em segurança e adoção on-premise, com Docker Compose oficial e licença open source [186][187][188]. Requer 2 GB de RAM para times pequenos e 4 a 8 GB para times maiores, sempre em Linux em produção [188][189]. O

Rocket.Chat, em TypeScript e Node.js com MongoDB, tem foco em compliance e implantações grandes, com instalação Docker oficial [192][193][194]. E o Zulip, Apache 2.0, é o campeão de contribuição entre chats de grupo open source, com o diferencial real do threading por tópico dentro de streams — mas exige mais trabalho no self-hosting, com recomendação oficial de instalação nativa em vez de Docker [197][198][199][200].

A comparação honesta aparece quando o time sente a diferença: o Slack e o Teams ainda lideram em chamadas de vídeo nativas de alta qualidade e em integrações prontas — o Mattermost tem vídeo básico e menos integrações no ecossistema [190][191], o Rocket.Chat sofre com documentação inconsistente [195][196], e o Zulip não tem ferramentas robustas de moderação em nível de organização [201][202]. O ponto que os comparativos independentes reforçam: para comunicação diária de um time técnico, a paridade é real; para uma organização que vive do ecossistema de integrações do Slack, a lacuna também é real [190][195][200].

A segunda peça é a que muda a escala de tudo: a camada de hospedagem. Até aqui, cada capítulo instala um serviço numa VPS. Agora a oficina ganha o equivalente ao armário de ferramentas da loja de aluguel: a plataforma que gerencia todos os containers. O Docker é a base — o formato comum em que todas as peças dos capítulos anteriores já vieram. Sobre ele, quatro camadas disputam a bancada. O Portainer, apesar do visual amigável, é gestão de containers — não faz deploy automatizado a partir do código, então não é um PaaS [203][204][205][206]. Os três PaaS estilo Heroku de verdade são o Dokku, o CapRover e o Coolify. O Dokku é o mais puro e o mais velho — 12 anos de projeto, majoritariamente Shell, ~32 mil estrelas, roda em 1 GB de RAM [213][214] — e mantém compatibilidade com os buildpacks do Heroku [216]. O CapRover constrói sobre Docker Swarm e entrega algo que o Dokku não tem por design: múltiplos nós nativos [217][218][220]. E o Coolify é o mais moderno, com dashboard completo, mais de 280 serviços de um clique e um diferencial que nenhum concorrente self-hosted oferece pronto: preview deployments por pull request [208][211][212]. Seu requisito oficial é 2 CPU, 2 GB de RAM e 30 GB de disco, com recomendação prática de 4 vCPU, 8 GB e 100 GB NVMe [209][210].

O balanço final — a terceira peça deste capítulo — é uma conta que a maioria pula. Ninguém compara o preço da assinatura com o preço real da operação própria. A referência de mercado em 2026: Hetzner é o melhor custo-benefício em VPS, com o CPX22 na faixa de US\$ 9,49 por mês [221][222]; e o Slack Pro custa cerca de US\$ 8,75 por usuário por mês [190]. Dez pessoas no Slack Pro custam mais de US\$ 87 mensais — quase dez VPSs como a que hospeda a oficina inteira, com todos os serviços dos oito capítulos. A conta completa, com as ressalvas corretas, é o assunto da seção 6.

3. Ilustra

A imagem mental deste capítulo é a pilha da oficina vista de lado. No topo está a VPS — o terreno onde tudo fica, alugado do provedor. Sobre o terreno, o Docker forma a primeira camada de chão. Sobre o Docker, a camada de hospedagem estilo Heroku — Coolify, Dokku, CapRover, com o Portainer como mero vigia dos containers — e sobre ela, finalmente, todas as peças montadas nos capítulos anteriores: o Mattermost da sala de equipe, o n8n da automação, o Metabase da parede de status, a estante do Nextcloud. No modelo alugado, essa pilha inteira é fragmentada em dezenas de assinaturas mensais, cada uma numa loja diferente. Na oficina, a pilha é uma só, suas, com uma conta única de energia.

A segunda lente, como em todos os capítulos, revela o custo invisível — e aqui ele é o mais claro de todo o livro. Quando o servidor cai, a assinatura resolve: você abre um ticket e a loja faz o serviço. Na oficina, o ciclo é seu: o servidor alerta, o Artesão diagnostica, o Artesão corrige, o servidor volta ao ar. Não há SLA — há o seu plantão. É exatamente essa diferença que o diagrama abaixo contrasta, retomando a lacuna honesta que os capítulos anteriores marcaram.

```
%% legenda: O ciclo de manutencao na Oficina Digital vs a assinatura
sequenceDiagram
    participant S as Servidor
    participant A as Artesao
    participant L as Loja de aluguel
    Note over S,L: Oficina propria
    S->>A: Alerta: servico fora do ar
    A->>A: Diagnostica o container
    A->>A: Corrige e reinicia
    A->>S: Servico volta ao ar
    Note over S,L: Assinatura
    S->>L: Alerta: servico fora do ar
    L->>L: Engenheiro da loja corrige
    L->>S: Servico volta ao ar
```

4. Técnica

Esta seção segue o padrão do material-fonte: tabela de decisão, manifesto Docker Compose real, sessões de terminal com comando e saída esperada — sem código de programação, porque o que você precisa aqui é operar a oficina, não desenvolvê-la.

A Tabela de Decisão da Sala de Equipe

A escolha entre as três peças de comunicação consolida o que a seção 2 levantou [186][192][197]:

Critério	Mattermost	Rocket.Chat	Zulip
Stack	Go + React	Node.js + MongoDB	Python + React
Licença	Open source	MIT (core)	Apache 2.0
RAM mínima	2 GB (time pequeno)	~2 GB	~2 GB (produção exige mais)
Instalação	Docker Compose oficial	Docker Compose	Nativa recomendada
Diferencial	Segurança, on-premise	Compliance	Threading por tópico
Onde sente a diferença	Vídeo básico, menos integrações	Documentação inconsistente	Self-hosting mais trabalhoso

A Bancada Que Hospeda Tudo: Mattermost + Docker

A peça que fecha a sala de equipe é um manifesto Docker Compose do Mattermost — aplicação e PostgreSQL, a base mínima de produção [188]:

```
# docker-compose.yml – Mattermost (time pequeno)
version: "3.9"

services:
  mm-postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: mattermost
      POSTGRES_USER: mmuser
      POSTGRES_PASSWORD: troque-esta-senha
    volumes:
      - pgdata:/var/lib/postgresql/data

  mattermost:
    image: mattermost/mattermost-team-edition:latest
```

```

restart: unless-stopped
ports:
  - "8065:8065"
environment:
  MM_SQLSETTINGS_DRIVERNAME: postgres
  MM_SQLSETTINGS_DATASOURCE: postgres://mmuser:troque-esta-senha@mm-
postgres:5432/mattermost?sslmode=disable
  MM_SERVICESETTINGS_SITEURL: http://chat.suaoficina.com
volumes:
  - mmdata:/mattermost/data
depends_on:
  - mm-postgres

volumes:
  pgdata:
  mmdata:

```

Uma observação sobre o manifesto vale a pena registrar: ele usa uma variável de ambiente para cada segredo — a senha do banco é declarada no arquivo, mas em produção a prática recomendada é substituí-la por uma referência ao cofre de segredos da sua infraestrutura [188] [189]. O `docker compose` aceita a interpolação `${VAR}` no lugar do valor literal, e é isso que separa um manifesto de exemplo de uma configuração de produção. O padrão deste livro é didático: os manifestos mostram o caminho completo com valores trocáveis, e a seção de segurança do capítulo 8 fecha essa lacuna com a rotina de atualização e o ritual de rollback.

Subir a sala e conferir o serviço de pé:

```

$ docker compose up -d
[+] Running 2/2
 ✓ Container mm-postgres Started
 ✓ Container mattermost Started
$ curl -sI http://localhost:8065 | head -1
HTTP/1.1 200 OK

```

O Deploy na Camada PaaS: Dokku

Com o dashboard do Coolify ou o comando do Dokku, a oficina ganha o deploy estilo Heroku. A sessão abaixo mostra o fluxo completo no Dokku — o PaaS mais enxuto da prateleira [214] [216]:

```

$ dokku apps:create oficina-chat
Creating oficina-chat... done

$ dokku postgres:create chat-db

```

```

Waiting for container to be ready... done
$ dokku postgres:link chat-db oficina-chat

$ dokku domains:add oficina-chat chat.suaoficina.com
-----> Added chat.suaoficina.com to oficina-chat

$ git push dokku main
-----> Building oficina-chat from buildpacks...
-----> Deploying oficina-chat to dokku
=====> Application deployed:
http://chat.suaoficina.com

```

O Armário com Pannel: Coolify para Quem Quer Ver Tudo

O Dokku é poderoso e rápido, mas é 100% linha de comando — e o Artesão Digital que prefere ver a oficina num painel vai encontrar no Coolify a peça que faltava. O dashboard do Coolify lista cada aplicação, cada banco e cada domínio num único lugar, com botões para subir, reiniciar, atualizar e ver logs de cada serviço [208]. O fluxo de deploy muda do `git push` do Dokku para o formulário do painel: você cola a URL do repositório, escolhe o serviço ou o buildpack e clica em salvar — o Coolify cuida do clone, do build e do proxy reverso com HTTPS automático [209]. A recomendação prática da comunidade Hetzner — que roda o Coolify justamente na VPS que este livro usa como referência de custo — é de uma máquina com 4 vCPU, 8 GB de RAM e 100 GB NVMe, folga que segura os builds sem travar a oficina inteira [210].

```

$ sudo curl -fsSL https://coolify.io/install.sh | bash
[INFO] Installing Docker Engine...
[INFO] Installing Coolify v4.x...
[INFO] Coolify is ready: http://192.168.0.10:8000
$ curl -sI http://192.168.0.10:8000 | head -1
HTTP/1.1 200 OK

```

O comando acima é o ritual de instalação oficial do Coolify: um script que instala o Docker e o próprio Coolify numa VPS limpa [209]. A partir daí, o painel de configuração inicial define o domínio da oficina e o e-mail do administrador, e a prateleira de serviços ganha mais de 280 opções de um clique — do n8n do capítulo 4 ao Metabase do capítulo 7 [208][211]. O comparativo da Ownkube resume a hierarquia da prateleira em uma frase: CapRover é o equilíbrio entre simplicidade e múltiplos nós via Swarm, Dokku é a máxima pureza single-node e Coolify é a experiência moderna de painel com o diferencial dos previews por pull request [220]. A escolha entre os três é a escolha entre o terminal, o equilíbrio e o dashboard — nenhuma delas cobra por usuário, e todas entregam a mesma base Docker que o livro inteiro usou [217][218][219].

A Segurança da Oficina: Atualização e Acesso

Fechar a infraestrutura exige encarar a parte que nenhuma assinatura mostrava na fatura: a segurança. Quando o serviço era da loja, a loja aplicava o patch e você nem ficava sabendo. Na oficina, o patch é seu — e o custo de não aplicá-lo é mensurável. O Mattermost publica correções de segurança nos próprios canais oficiais e recomenda a atualização imediata das instâncias expostas à internet [190][191], e o Docker Engine, que sustenta toda a pilha, recebe atualizações críticas mensais que corrigem vulnerabilidades de container escape e rede [203]. A rotina mínima do plantão tem três comandos e uma verificação: atualizar a base do sistema, atualizar as imagens e reiniciar os containers, e conferir se os serviços voltaram ao ar por HTTPS [205][209]:

```
$ sudo apt update && sudo apt upgrade -y
$ cd /srv/oficina && docker compose pull
$ docker compose up -d
$ curl -sI https://chat.suaoficina.com | head -1
HTTP/1.1 200 OK
```

Não existe sla que cubra a sua oficina — existe a sua auditoria. A boa notícia é que ela cabe em um script e num horário fixo na agenda: a atualização mensal, o backup verificado do capítulo 6 e o teste de restauração trimestral formam o tripé que mantém a prateleira de pé [215][216]. Quem pula esse tripé não está economizando — está apostando que o desastre nunca vem, e o desastre sempre vem, mais cedo ou mais tarde.

O Ritual de Rollback: Quando a Atualização Quebra a Bancada

Toda atualização tem o potencial de quebrar a bancada — e a diferença entre o amador e o Artesão Digital aparece exatamente nesse momento. O amador atualiza, quebra, e passa a madrugada googleando o erro; o Artesão Digital atualiza com um plano de rollback já definido, porque a infraestrutura Docker dá essa garantia de graça: a imagem antiga continua no registro local, e voltar é um comando [204][205]. O ritual começa antes da atualização, com o registro da versão atual — um snapshot do estado da oficina:

```
$ docker compose ps --format "table {{.Name}}\t{{.Image}}\t{{.Status}}"
NAME                IMAGE                                STATUS
oficina-mattermost  mattermost/mattermost-team-edition:9.11  Up 42 days
oficina-db          postgres:16-alpine                    Up 42 days
$ docker images | grep mattermost
mattermost/mattermost-team-edition  9.11      3f9a1c2d8e4b  42 days ago
812MB
mattermost/mattermost-team-edition  9.10      8b7d5e2f1a3c  70 days ago
805MB
```

As duas linhas do `docker images` são o seguro da oficina: a imagem 9.10 ainda está lá, pronta para voltar. Com o registro feito, a atualização segue o fluxo padrão — e se algo quebrar, o rollback é o mesmo fluxo invertido:

```
$ docker compose pull
$ docker compose up -d
$ curl -sI https://chat.suaoficina.com | head -1
HTTP/1.1 503 Service Unavailable # projeto quebrou na atualizacao
$ docker compose up -d --no-deps mattermost=9.10
$ docker compose restart mattermost
$ curl -sI https://chat.suaoficina.com | head -1
HTTP/1.1 200 OK # bancada de volta ao ar
```

A sessão mostra o ciclo completo em quatro comandos: atualizar, testar, perceber o 503, voltar a imagem anterior [204][205][216]. O tempo de inatividade da oficina foi de minutos, não de horas — e a lição registrada (a 9.11 quebrou, a 9.10 é estável) entra no caderno de manutenção para guiar a próxima atualização. Esse ritual é o que transforma a manutenção de medo em rotina: o desastre deixa de ser o fim do mundo e vira um procedimento de dez minutos, documentado e testado.

5. Aplica

A jornada prática deste capítulo é a primeira semana da oficina completa — o momento em que todas as peças dos capítulos anteriores passam a viver sob um mesmo teto. O cenário: o time de três pessoas já usa o `n8n` do capítulo 4, o `Metabase` do capítulo 7 e a estante do capítulo 6, mas cada um numa VPS separada, com senhas diferentes e sem dono único. O primeiro passo não é técnico — é o inventário: listar cada serviço, o domínio, a versão e quem responde por ele. Esse inventário é o ponto de partida de tudo o que vem na sequência, e é também o que a assinatura nunca exigiu porque o serviço vivia na nuvem da loja.

Erro comum: você decide hospedar tudo na mesma VPS que já roda metade dos serviços, sem camada de gerenciamento. O primeiro sintoma é o “mas funciona na minha máquina” inverso: o serviço funciona até o dia em que dois containers brigam pela mesma porta, ou uma atualização de segurança quebra o que estava estável, e não há registro de qual versão estava rodando antes. O diagnóstico é o mesmo sempre: infraestrutura sem camada de hospedagem é uma oficina sem armário — as peças estão lá, mas ninguém sabe onde cada uma está, quem atualizou a última vez ou como voltar atrás.

Prática correta: a jornada começa pelo armário. Você instala o `Coolify` ou o `Dokku` na VPS dedicada da infraestrutura — a seção 4 mostrou os dois caminhos — e migra, um por um, os serviços com os manifestos que este livro já construiu: o `docker-compose` do `Mattermost` de hoje, o do `Metabase` do capítulo 7, o do `Nextcloud` do capítulo 6. Cada migração registra no

mesmo lugar o domínio, o backup e a política de atualização. A porta 8065 do chat não é mais um endereço solto na memória: é um app no armário, com domínio próprio e deploy rastreável. A régua de segurança entra na mesma rotina — atualização mensal agendada, backup verificado com o restic do capítulo 6, e o plantão de quem responde quando o alerta dispara. E vale repetir a ressalva de hardware que o capítulo 5 marcou: requisito de infraestrutura não é sugestão — o Coolify pede 4 vCPU e 8 GB de RAM com folga, e uma VPS de 2 GB que tentar hospedar a oficina inteira vai travar no primeiro build [210].

O balanço final — a decisão que fecha o livro: com a oficina inteira sob um teto, chega a hora da conta que a seção 2 preparou. Some o custo da VPS da infraestrutura (o CPX22 da Hetzner na faixa de US\$ 9,49 mensais [221][222]), as horas de manutenção e o risco assumido de segurança — contra a soma das assinaturas evitadas nos oito capítulos: Notion, Mailchimp, Salesforce, Zapier, Canva, Adobe, Drive e Slack Pro. O Slack sozinho, para o time de dez, passaria de US\$ 87 por mês [190]. A conta fecha a favor da oficina — mas o checklist abaixo é a régua que decide quando ela não fecha, e é essa régua que você leva para qualquer categoria futura.

Checklist de decisão do Artesão Digital — reutilizável para qualquer ferramenta:

1. A assinatura cobra por usuário? Some o custo anual de todo o time.
2. A alternativa open source tem manutenção ativa? Olhe o repositório e o changelog — a lição do Redash do capítulo 7.
3. A equipe depende de integrações prontas ou recursos exclusivos da loja? Video nativo, conectores, marketplace.
4. Alguém assume o plantão de atualizações, backup e segurança? Sem dono, não há troca.
5. A soma VPS + manutenção + risco fica abaixo da assinatura? Só então a peça é sua.

6. Conclusão

Este capítulo fechou a jornada inteira. Você substituiu a sala de equipe alugada por peças reais da prateleira — Mattermost, Rocket.Chat e Zulip — aprendendo onde a paridade é real e onde a equipe sente a diferença [186][197][200]. Montou o armário que sustenta tudo: Docker na base, e a camada estilo Heroku com Dokku, CapRover e Coolify — descobrindo que o deploy dos serviços dos sete capítulos anteriores é o mesmo fluxo que a loja vendia como exclusividade [208][214][217]. E fechou o balanço com a régua que vale para o resto da vida profissional: a troca só é real quando a soma de VPS, manutenção e risco fica abaixo da assinatura — e o checklist de decisão que este capítulo entrega é a ferramenta que aplica essa régua a qualquer categoria futura [221][222].

O Artesão Digital que termina este livro não é o que trocou oito ferramentas. É o que aprendeu a reconhecer, em qualquer nova ferramenta que aparecer amanhã, a diferença entre a peça alugada e a peça própria — e a calcular, com os olhos abertos, quando a troca vale

o esforço de montar, manter e garantir a própria oficina. A bancada está montada, cada peça dos oito capítulos tem dono, backup e dono do plantão, e a régua de decisão está na sua mão para a próxima ferramenta que a loja tentar vender. O próximo passo — e a próxima peça — é decisão sua.

7. Referências Bibliográficas

- [186] MATTERMOST. *mattermost/mattermost*. Disponível em: <https://github.com/mattermost/mattermost>. Acesso em: 19 ago. 2026. [187] MATTERMOST. *The Mattermost server repo surpasses 20,000 stars on GitHub*. Disponível em: <https://mattermost.com/blog/mattermost-server-surpasses-20000-stars-on-github/>. Acesso em: 19 ago. 2026. [188] MATTERMOST. *Deploy Mattermost using Containers*. Disponível em: <https://docs.mattermost.com/deployment-guide/server/deploy-containers.html>. Acesso em: 19 ago. 2026. [189] MATTERMOST. *Install Docker — Mattermost documentation*. Disponível em: <https://docs.mattermost.com/deployment-guide/server/containers/install-docker.html>. Acesso em: 19 ago. 2026. [190] SOURCEFORGE. *Mattermost vs. Microsoft Teams vs. Slack Comparison*. Disponível em: <https://sourceforge.net/software/compare/Mattermost-vs-Microsoft-Teams-vs-Slack/>. Acesso em: 19 ago. 2026. [191] BRIGHTSCOUT. *Mattermost vs. Microsoft Teams: Which is Better?*. Disponível em: <https://www.brightscout.com/insight/mattermost-vs-microsoft-teams-which-is-better>. Acesso em: 19 ago. 2026. [192] ROCKET.CHAT TECHNOLOGIES CORP. *RocketChat/Rocket.Chat*. Disponível em: <https://github.com/RocketChat/Rocket.Chat>. Acesso em: 19 ago. 2026. [193] ROCKET.CHAT. *Deploy with Docker & Docker Compose*. Disponível em: <https://docs.rocket.chat/deploy/prepare-for-your-deployment/docker-and-docker-compose>. Acesso em: 19 ago. 2026. [194] LINUXHANDBOOK. *Complete Guide to Self-hosting Rocket.Chat With Docker*. Disponível em: <https://linuxhandbook.com/rocket-chat-docker/>. Acesso em: 19 ago. 2026. [195] ITSFOSS. *Rocket.Chat vs. Slack: Choosing the Perfect Team Collaboration App*. Disponível em: <https://itsfoss.com/rocket-chat-vs-slack/>. Acesso em: 19 ago. 2026. [196] ALPHAEFFICIENCY. *Rocket Chat vs Slack: The Battle of the Chat Platforms*. Disponível em: <https://alphaefficiency.com/rocket-chat-vs-slack>. Acesso em: 19 ago. 2026. [197] ZULIP. *zulip/zulip*. Disponível em: <https://github.com/zulip/zulip>. Acesso em: 19 ago. 2026. [198] ZULIP. *Install a Zulip server — Zulip documentation*. Disponível em: <https://zulip.readthedocs.io/en/stable/production/install.html>. Acesso em: 19 ago. 2026. [199] ZULIP. *Self-host Zulip*. Disponível em: <https://zulip.com/self-hosting/>. Acesso em: 19 ago. 2026. [200] MARKAICODE. *Zulip vs Slack: Open Source Team Chat Performance and Feature Analysis*. Disponível em: <https://markaicode.com/vs/zulip-vs-slack/>. Acesso em: 19 ago. 2026. [201] ZULIP. *Issue #11119 — Is there a way to set resource usage limitations?*. Disponível em: <https://github.com/zulip/zulip/issues/11119>. Acesso em: 19 ago. 2026. [202] HACKER NEWS. *“You can install a Zulip server on a system with 2G of RAM, but for production...”*. Disponível em: <https://news.ycombinator.com/item?id=10280901>. Acesso em: 19 ago. 2026. [203] PORTAINER.IO. *portai-*

ner/portainer. Disponível em: <https://github.com/portainer/portainer>. Acesso em: 19 ago. 2026.

[204] PORTAINER. *Requirements and prerequisites*. Disponível em: <https://docs.portainer.io/start/requirements-and-prerequisites>. Acesso em: 19 ago. 2026.

[205] PORTAINER. *Install Portainer CE with Docker on Linux*. Disponível em: <https://docs.portainer.io/start/install-ce/server/docker/linux>. Acesso em: 19 ago. 2026.

[206] SFEIR INSTITUTE. *Rancher vs Lens vs Portainer: Kubernetes Dashboard Comparison*. Disponível em: <https://institute.sfeir.com/en/kubernetes-training/rancher-vs-lens-vs-portainer-dashboard-kubernetes/>. Acesso em: 19 ago. 2026.

[207] NORTHFLANK. *5 best Portainer alternatives for enterprise Kubernetes and Docker management*. Disponível em: <https://northflank.com/blog/portainer-alternatives>. Acesso em: 19 ago. 2026.

[208] COOLLABSIO. *coollabsio/coolify*. Disponível em: <https://github.com/coollabsio/coolify>. Acesso em: 19 ago. 2026.

[209] COOLIFY. *Installation | Coolify Docs*. Disponível em: <https://coolify.io/docs/get-started/installation>. Acesso em: 19 ago. 2026.

[210] HETZNER COMMUNITY. *Install and Configure Coolify on Linux*. Disponível em: <https://community.hetzner.com/tutorials/install-and-configure-coolify-on-linux/>. Acesso em: 19 ago. 2026.

[211] GETAUTONOMA. *Coolify vs Vercel: The Self-Hosting Tax Nobody Mentions*. Disponível em: <https://getautonoma.com/blog/coolify-vs-vercel>. Acesso em: 19 ago. 2026.

[212] GETAUTONOMA. *Open-Source Vercel Alternatives: Coolify, Dokku, Kamal, and CapRover Compared*. Disponível em: <https://getautonoma.com/blog/open-source-alternatives-vercel>. Acesso em: 19 ago. 2026.

[213] DOKKU. *dokku/dokku*. Disponível em: <https://github.com/dokku/dokku>. Acesso em: 19 ago. 2026.

[214] DOKKU. *Getting Started with Dokku — Dokku Documentation*. Disponível em: <https://dokku.com/docs/getting-started/installation/>. Acesso em: 19 ago. 2026.

[215] BLOG LOCALOPS. *Self-Hosted Heroku Alternatives in 2026: Build vs. Buy for Platform Engineering Teams*. Disponível em: <https://blog.localops.co/p/self-hosted-heroku-alternatives-build-vs-buy>. Acesso em: 19 ago. 2026.

[216] SLIPLANE. *Dokku: The Self-Hosted Heroku Alternative in 2026*. Disponível em: <https://sliplane.io/blog/dokku-self-hosted-heroku-alternative>. Acesso em: 19 ago. 2026.

[217] CAPROVER. *caprover/caprover*. Disponível em: <https://github.com/caprover/caprover>. Acesso em: 19 ago. 2026.

[218] CAPROVER. *Getting Started · CapRover*. Disponível em: <https://caprover.com/docs/get-started.html>. Acesso em: 19 ago. 2026.

[219] CAPROVER. *caprover/one-click-apps*. Disponível em: <https://github.com/caprover/one-click-apps>. Acesso em: 19 ago. 2026.

[220] OWNKUBE. *Self-hosted PaaS in 2026: Coolify vs Dokku vs CapRover vs Ownkube*. Disponível em: <https://ownkube.io/blog/self-hosted-paas-comparison-2026>. Acesso em: 19 ago. 2026.

[221] BETTERSTACK COMMUNITY. *DigitalOcean vs. Hetzner Cloud: a side-by-side comparison for 2026*. Disponível em: <https://betterstack.com/community/guides/web-servers/digitalocean-vs-hetzner/>. Acesso em: 19 ago. 2026.

[222] VPSBENCHMARKS. *DigitalOcean vs Hetzner: performance, features and prices*. Disponível em: https://www.vpsbenchmarks.com/compare/docean_vs_hetzner. Acesso em: 19 ago. 2026.

